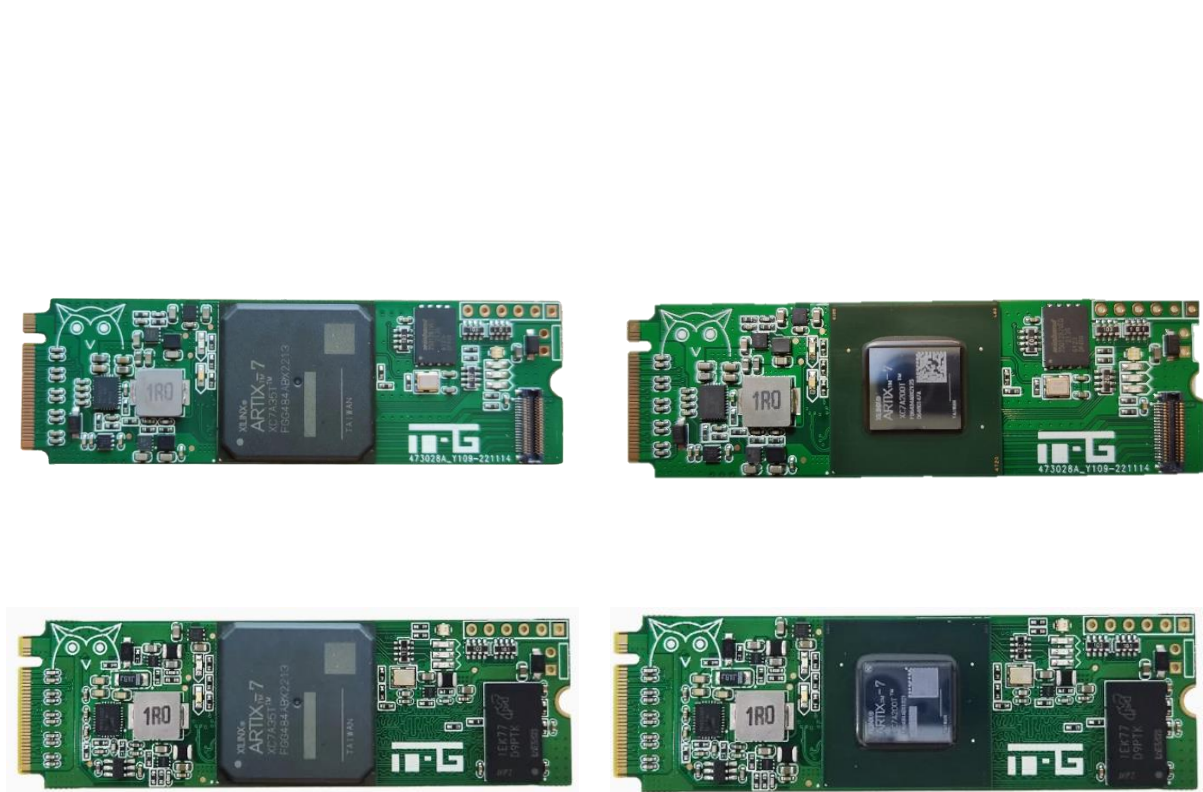


Xilinx Artix-7 M.2 PCIE 加速卡例程指导(V1.1)



2024 年 5 月 13 日

联系方式

官方淘宝店：极客堂电子

官方闲鱼：ITANGTANGI



TEL: 13755322372 (微信同号)



文件修订记录

2023.03.13 (V1.0)

创建文件和目录，添加基础教程，Riffa PCIE，XDMA PCIE 和光通信 IBERT 例程。

2023.03.13 (V1.01)

修改 Vivado2021.1 下载链接为百度网盘。

2024.05.13 (V1.1)

修改增加 Winbond Flash 支持方式，变成直接从 txt 文件中 copy，增加一些 IO 底板和转接板资料。

目录

文件修订记录.....	I
目录.....	I
第一章 基础介绍及软件准备	1
1.1 硬件介绍	1
1.2 软件安装	2
1.2.1 Vivado 软件安装	2
1.2.2 XDMA 配套 WDK, WSDK, VS2015 安装（编译驱动和应用）	7
1.2.3 Qt 软件安装（Riffa 使用, XDMA 也可使用）	9
1.3 程序固化	11
1.3.1 添加 winbond W25Q128 的支持	11
1.3.2 Vivado 固化程序到 Flash.....	11
第二章 PCIE XDMA 测试	15
2.1 XDMA 简介.....	15
2.2 XDMA 特性.....	15
2.3 XDMA 测试工程创建.....	16
2.4 XDMA IP 配置	18
2.4.1 Basic 项	18
2.4.2 PCIE ID 配置	19
2.4.3 PCIE BAR 配置	20
2.4.4 PCIE 中断配置	21
2.4.5 PCIE DMA 配置	22
2.4.6 剩余配置	24
2.5 生成 Bin 文件和烧录	28
2.6 XDMA 驱动和应用编译（也可以不编译）	30
2.7 上机测试准备.....	32
2.8 xdma_rw BRAM 读写测试	35
2.9 simple_dma 简单测速	37
2.10 xdma_test 基础测试	37
2.11 后记	38
第三章 Riffa 数据环回	39
3.1 Riffa 概述.....	39
3.2 RIFFA 体系结构.....	40
3.3 RIFFA 驱动安装.....	42

3.4 RIFFA 库使用	44
3.5 RIFFA Vivado 工程搭建.....	45
3.6 PCIE IP 配置.....	48
3.6.1 Basic	48
3.6.2 IDs	49
3.6.3 BARs	50
3.6.4 Core Capabilities	50
3.6.5 Link Registers	51
3.6.6 Interrupts	52
3.6.7 Power Management.....	53
3.6.8 Ext Capabilities	54
3.6.9 Ext Capabilities-2.....	55
3.6.10 TL Settings	55
3.6.11 DL&PL Settings	56
3.6.12 Shared Logic	56
3.6.13 Core Interface Parameters	57
3.6.14 Add.Debug Options.....	57
3.6.15 Soft Reset 配置	59
3.6.16 PCIE 通道修改	59
3.7 Riffa RTL 源文件及约束文件添加	60
3.8 上机测试.....	65
3.8.1 Flash 烧录	65
3.8.2 安装加速卡	65
3.8.3 Qt 软件工程编译和运行测试.....	66
3.9 后记.....	67
第四章 光通信 ibert 测试.....	68
4.1 硬件准备.....	68
4.2 创建 Vivado 测试工程	69
4.3 ibert 测试.....	75

第一章 基础介绍及软件准备

1.1 硬件介绍

基于 Xilinx Artix-7 系列 FPGA 芯片设计的 M.2 M-Key FPGA 加速卡，引出 Artix7-484 脚芯片的 4 条高速 GT，最高支持 PCIE2.0*4 速率。高功率 12A 核心电源设计，可支持 Artix7-XC7A35T, Artix7-XC7A50T, Artix7-XC7A75T, Artix7-XC7A100T 和 Artix7-XC7A200T 芯片。加速卡正面如图 1-1 所示。



图 1-1 M2 PCIE 加速卡正面

加速卡板载硬件资源如图 1-2 所示。

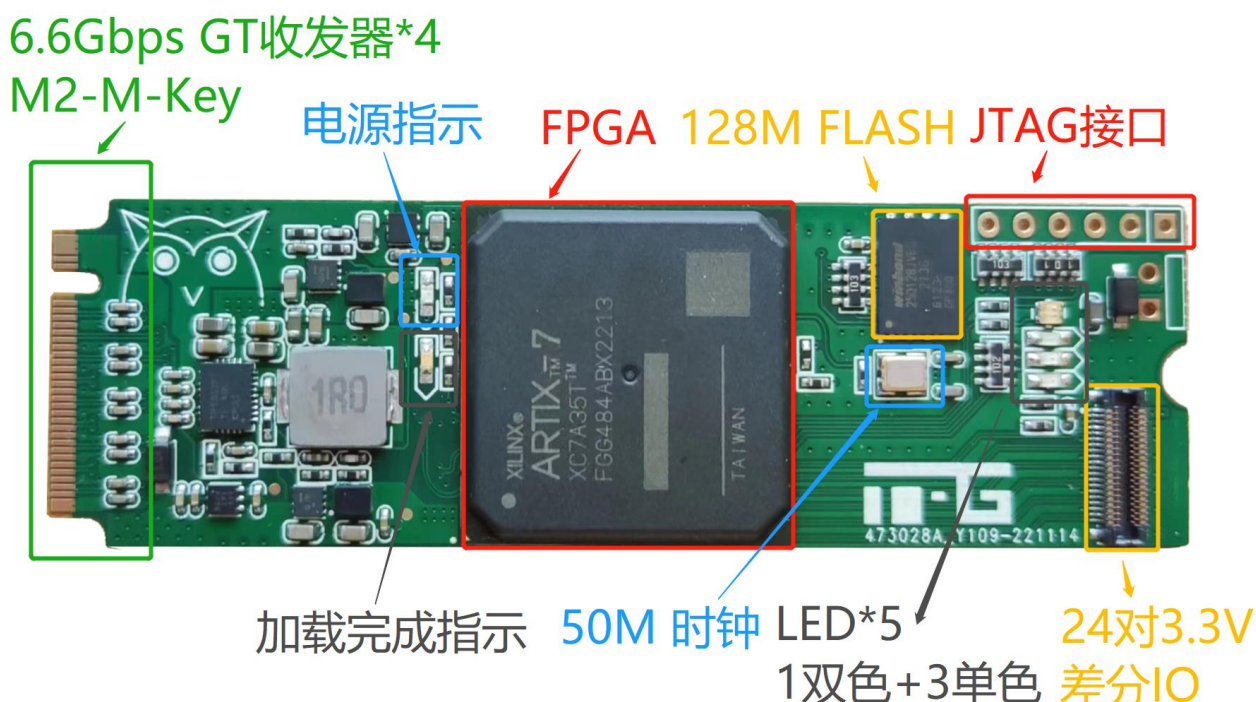


图 1-2 加速器板载硬件资源图

由于本 PCIE 加速卡只是将 GT 收发器以 M2 接口形式引出，所以可以通过 M2 接口座子转接出不同类型的应用底板，不局限于 PCIE 应用。例如 SFP, USB3.0 或者 HDMI 等。所以设计并制作了如图 1-3 所示的 4 路 SFP 光通信底板，可搭配用于光通信测试或是用于

FPGA 加速卡的供电。

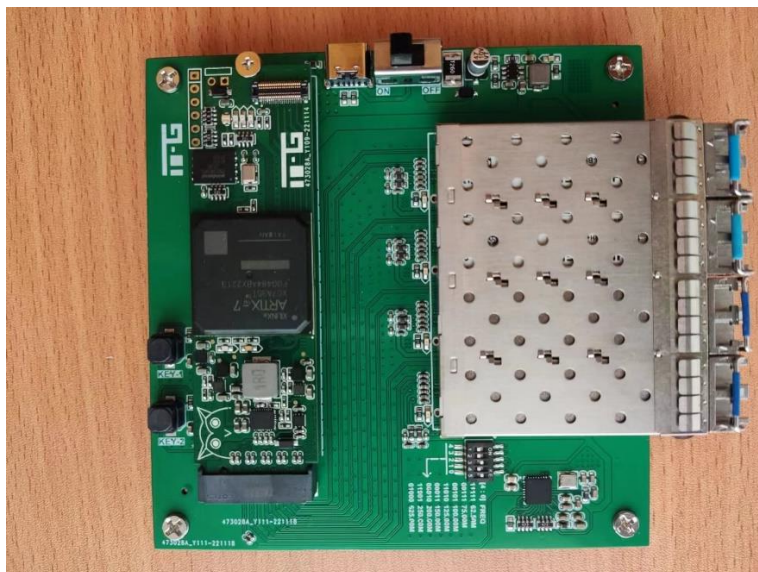


图 1-3 光通信底板

1.2 软件安装

本指导书使用的软件版本为：

FPGA 开发：Vivado 2021.1

PC Windows 软件开发（XDMA）：WDK，WINSDK，VS2015

PC Windows 软件开发（Riffa）：Qt5.9.9

开发平台：Windows10

1.2.1 Vivado 软件安装

Vivado Design Suite 是 Xilinx 公司的综合性 FPGA 开发软件，可以完成从设计输入到硬件配置的完整 FPGA 设计流程。本节将学习如何安装 Vivado 软件。

Vivado 2021.1 下载链接：

链接：<https://pan.baidu.com/s/15fkuISY8JP5FxbqwIgl8jg>

提取码：4q2r

下载得到名为 Xilinx_Unified_2021.10610,2318.tar 的压缩包，安装过程如下：

将压缩包解压出来（注意，解压目录的路径名称只能够包含字母、数字、下划线，否则安装程序有可能出问题），双击解压出来的文件夹下的“xsetup.exe”，如图 1-4 所示：

文件 (1)			
 xsetup	2019/11/7 13:10	文件	3 KB
文件夹 (6)			
 bin	2020/5/12 13:34	文件夹	
 data	2020/5/12 13:34	文件夹	
 lib	2020/5/12 13:34	文件夹	
 payload	2020/5/12 14:02	文件夹	
 scripts	2020/5/12 14:02	文件夹	
 tps	2020/5/12 14:04	文件夹	
应用程序 (1)			
 xsetup	2019/11/7 13:28	应用程序	439 KB

图 1-4 双击“xsetup.exe”安装

欢迎界面点击“Next”，进入协议许可界面，三个勾都选上，点击下一步进入版次选择，这里选择最上面一项 Vitis，这个就包括了所有的开发工具，如图 1-5 所示。

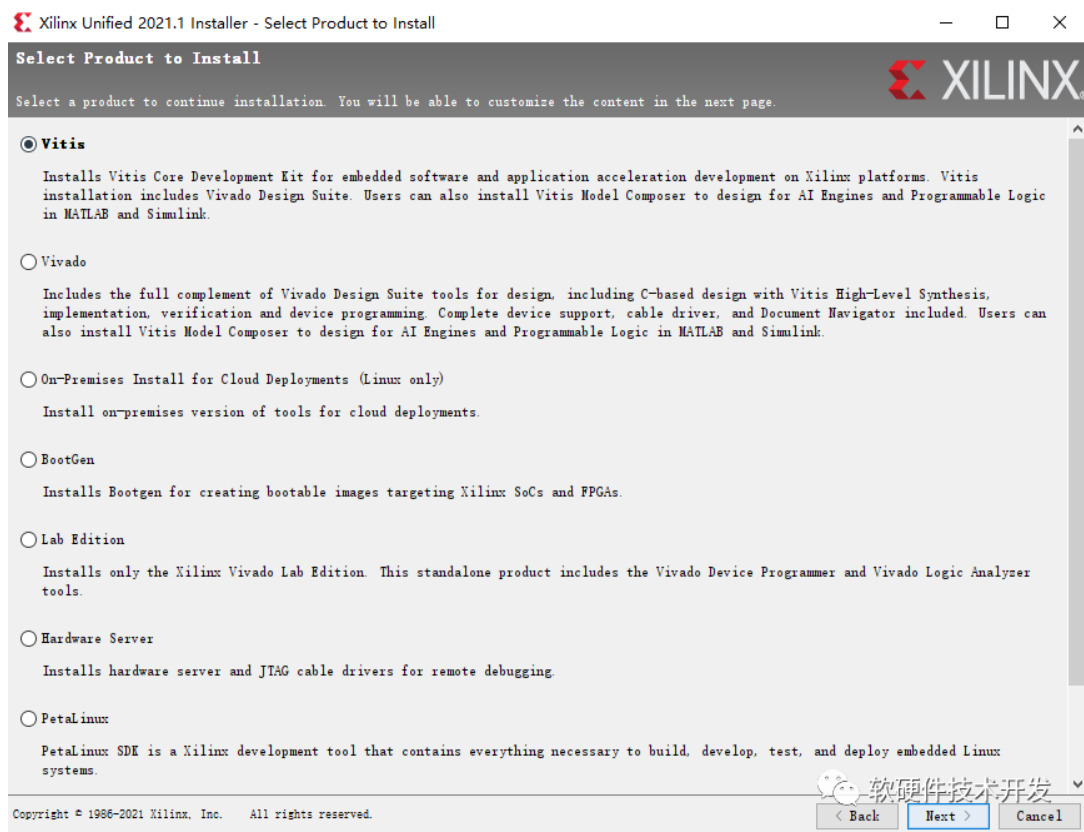


图 1-5 版式选择

接下来是选择工具组件和器件库。本加速卡只需要用到 7Series，但各位可以根据自己需要节省存储空间，可以将用不到的工具组件和器件库去掉，如图 1-6 所示。

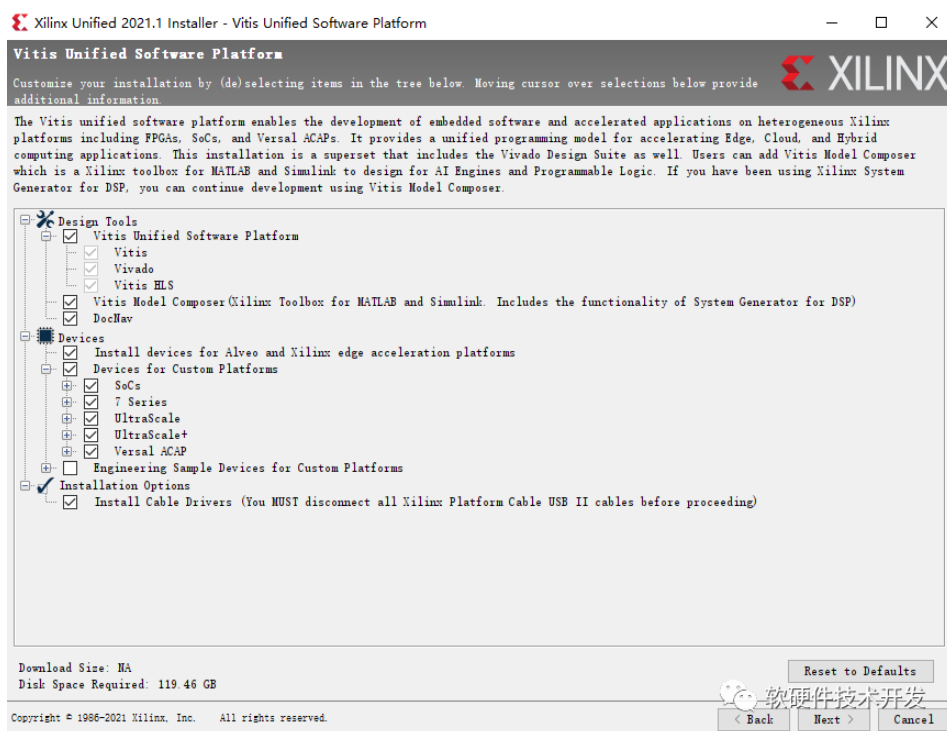


图 1-6 器件库和组件选择

点击“Next”，进入安装目录设置页面，如图 1-7 所示。

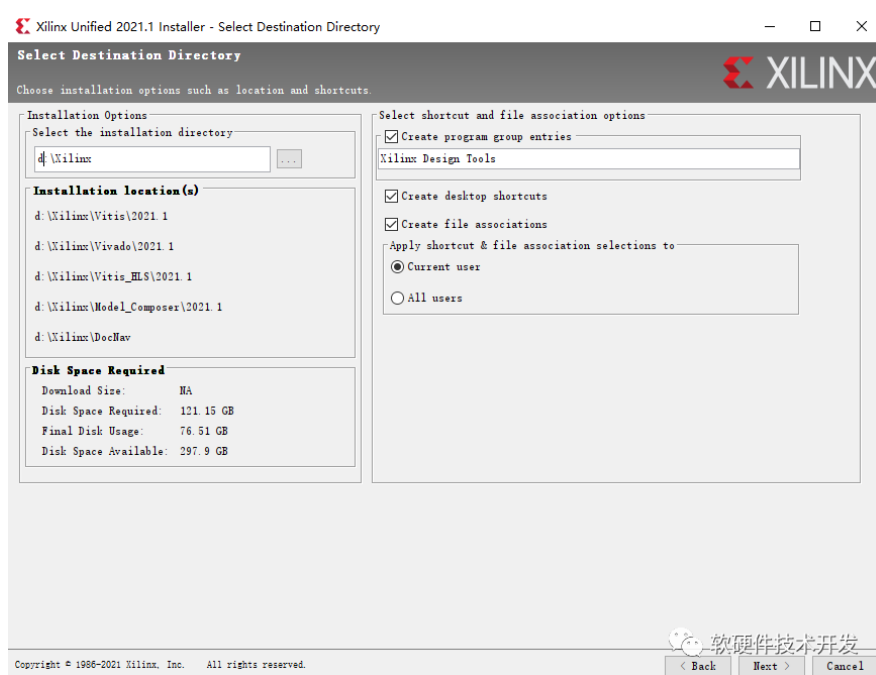


图 1-7 选择安装目录和创建快捷方式

点击“Next”，出现 Summary 界面，如图 1-8 所示，该界面是对先前所有安装配置信息的总结，供用户浏览确认。确认无误后，点击“Install”会开始 Vivado 设计套件的安装。如图 1-9 所示：（由于 Vivado 在安装期间会占用大量的电脑 CPU 资源和内存资源，所以这里建议大家在开始安装之前，尽量关闭电脑中其他的不必要的应用软件）

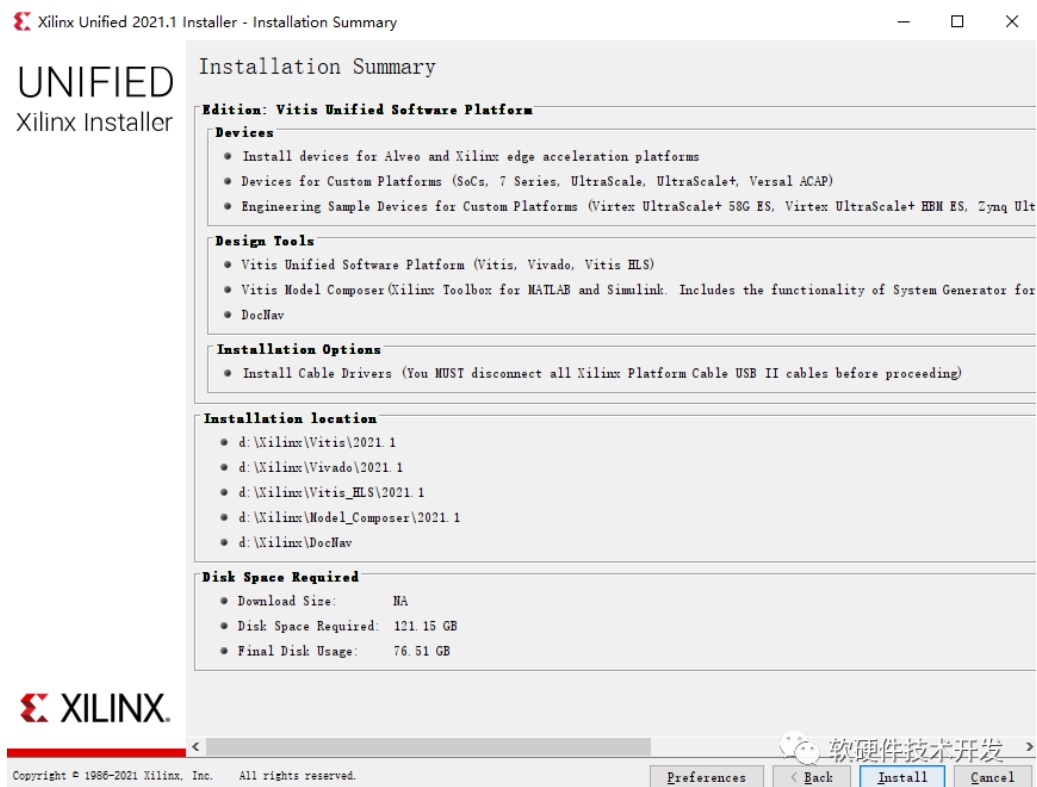


图 1-8 安装信息汇总



图 1-9 安装过程

安装过程中会弹出安装 PinPcap 的界面，此时一直点击“Next”完成安装即可，如图 1-10 所示。

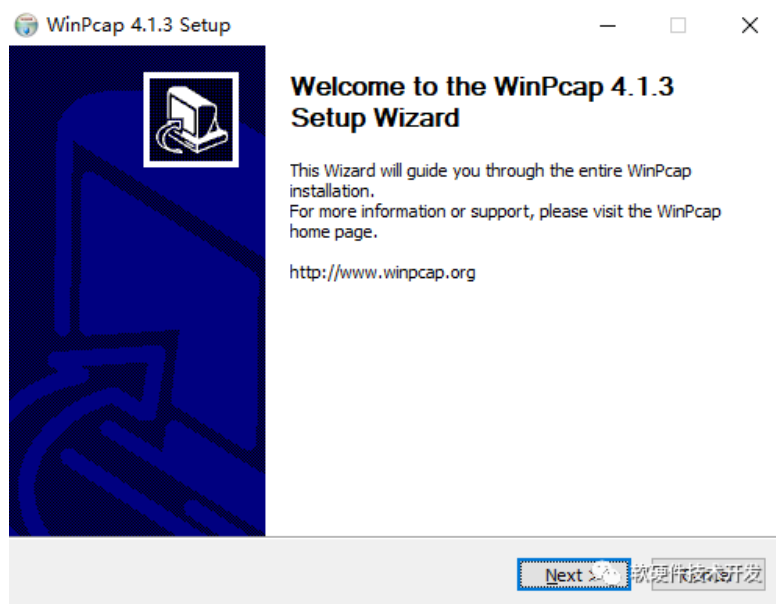


图 1-10 WinPcap 安装

安装完成会提示，如图 1-11 所示，同时会弹出“Vivado License Manager”窗口，在该窗口内点击“Copy license”，在资源管理器中选择提供的 License 文件并加载，如图 1-12 所示，至此便完成了 Vivado 软件的安装。

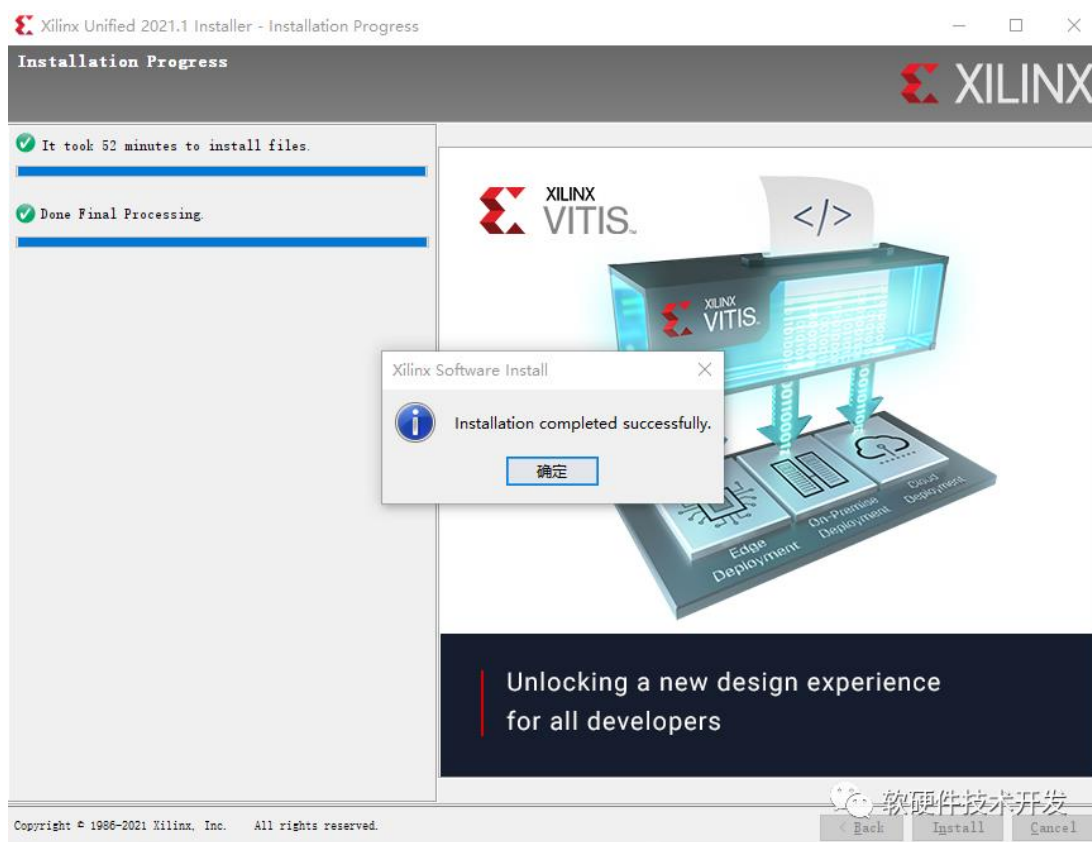


图 1-11 Vivado 安装完成

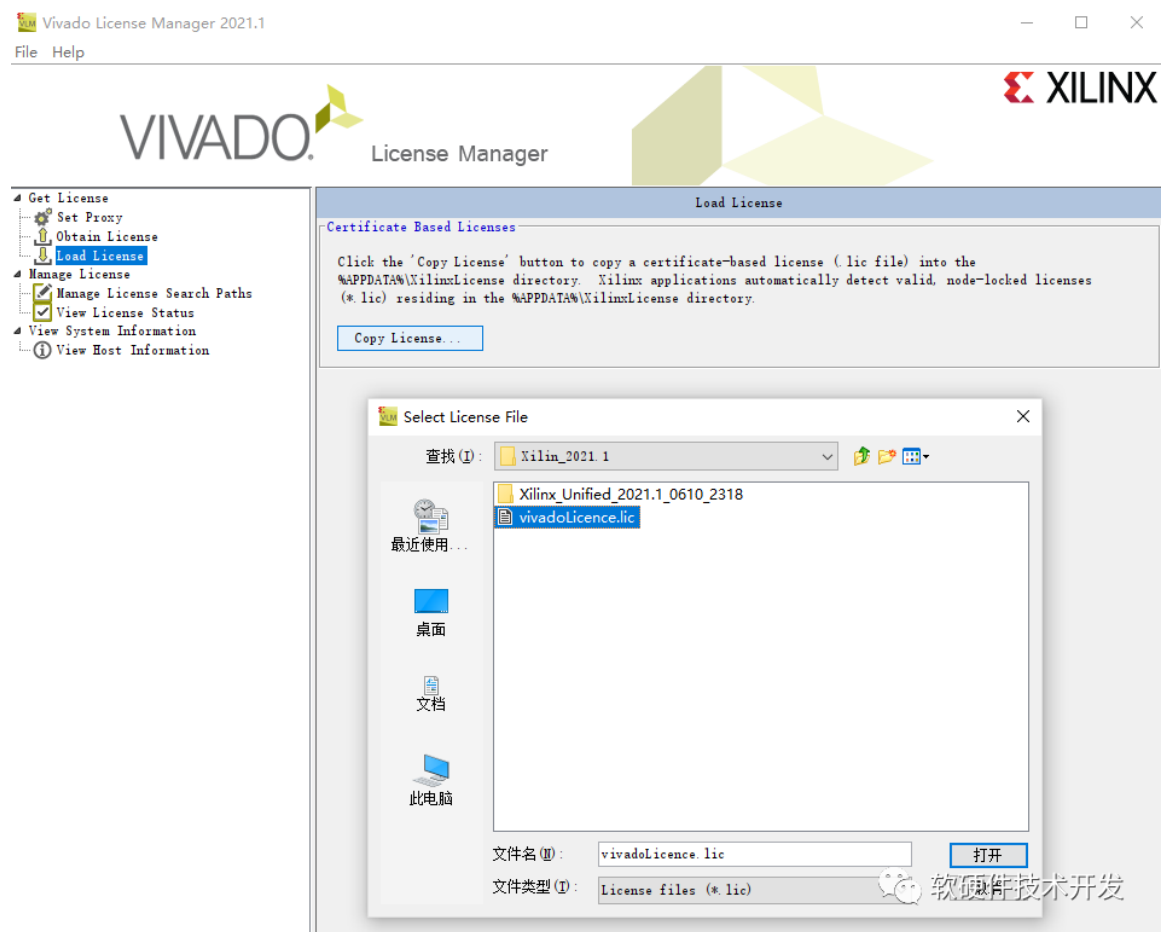


图 1-12 加载 License 文件

1.2.2 XDMA 配套 WDK, WSDK, VS2015 安装（编译驱动和应用）

WDK, WSDK, VS2015 下载地址:

链接: <https://pan.baidu.com/s/1nIOSw68EwMelhI34WZWEDA>

提取码: 4f2p

请注意软件的安装顺序一定要是先安装 vs2015 再安装 WINSDK, 最后安装 WDK。如果一不小心顺序错了, 卸载干净后重新安装, 否则肯定不会成功的。

首先是 VS2015 的安装, 如果已经安装了 VS2010, 或者其他版本, 建议先进行卸载, 卸载过程不要使用 360 等软件工具卸载, 而是使用 VS 自带的卸载工具 (注意: 操作过程中可能会出现问题, 导致该系统无论如何都无法编译程序, 本教程不对此情况负责。

双击打开 VS2015 软件安装包。如图 1-13 所示, 选择自定义安装, 之后勾选 Visual C++ 选项以及 Visual Studio 2015 更新 3 以避免编译过程报错。报错原因基本上是由于版本匹配问题引起的。之后一直点击下一步和同意相关协议完成安装即可, 安装完成后填入注册用的序列号: HM6NR-QXX7C-DFW2Y-8B82K-WTYJV。完成 VS2015 的安装。

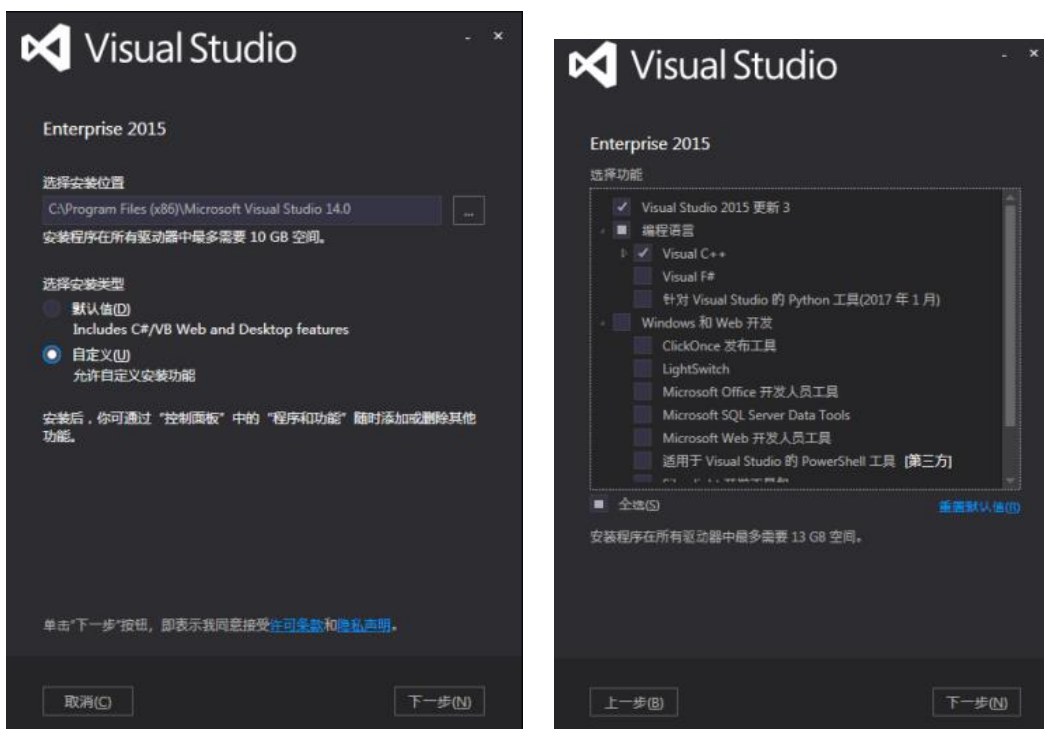


图 1-13 VS2015 安装配置

WSDK 和 WDK 的安装界面分别如图 1-14 和图 1-15 所示，无需配置，直接按照默认配置一直下一步即可。

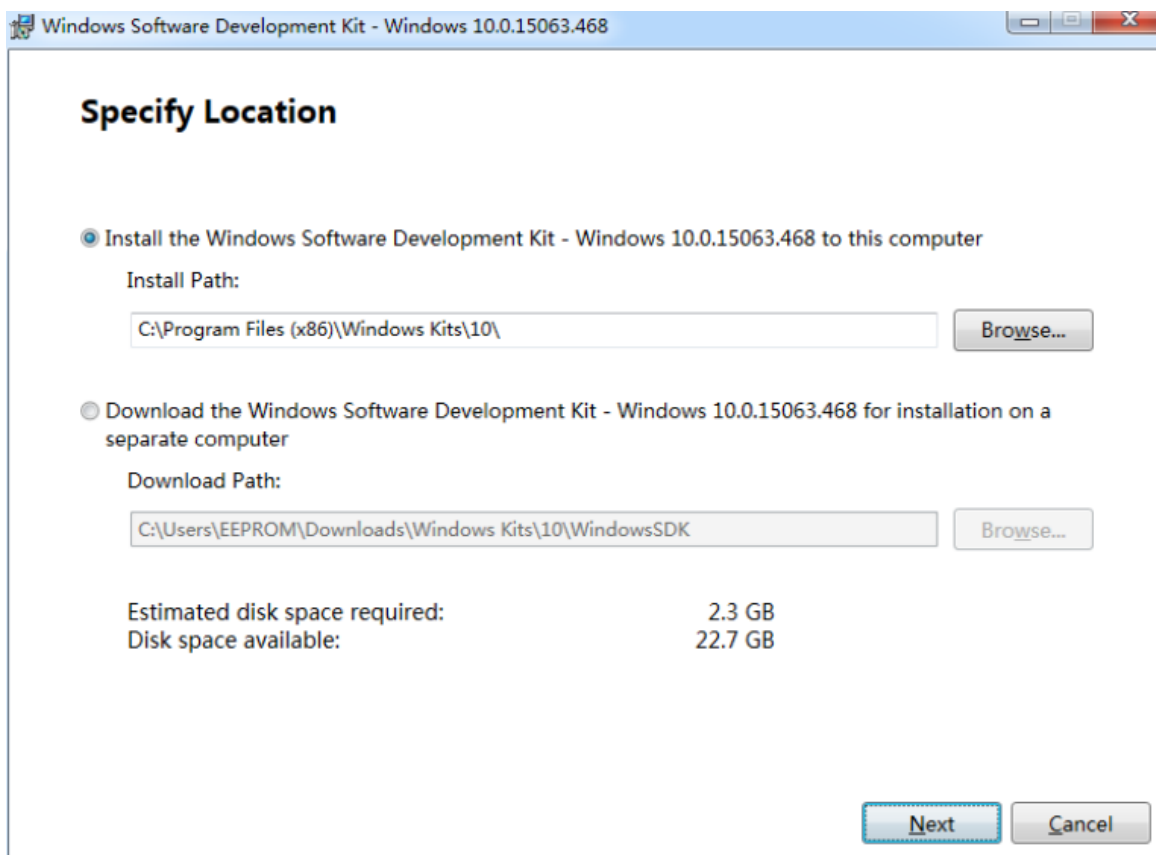


图 1-14 WSDK 安装界面

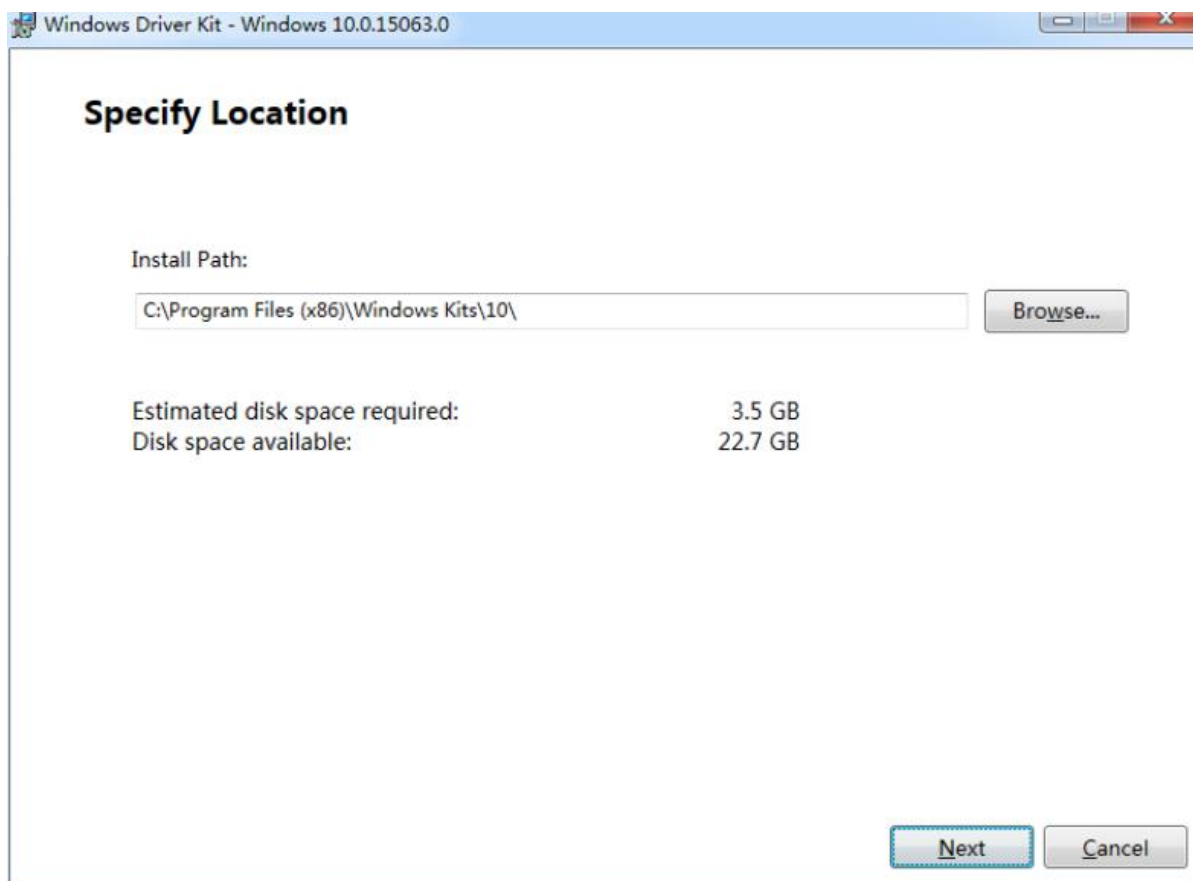


图 1-15 WDK 安装界面

至此完成 XDMA 所需 PC 端软件的安装，环境配置将在第二章给出。

1.2.3 Qt 软件安装（Riffa 使用，XDMA 也可使用）

Qt 软件用于 Riffa 开发 PC 端的软件工程用以和加速卡交互，其下载地址为：

<https://download.qt.io/archive/qt/5.9/5.9.9/>

选择 qt-opensource-windows-x86-5.9.9.exe 进行下载。

下载完成后，先关闭电脑的网络连接，否则 Qt 安装时会要求登陆账号，双击软件安装包打开，欢迎界面点击“Next”进入配置界面，如图 1-16 所示选择安装位置。

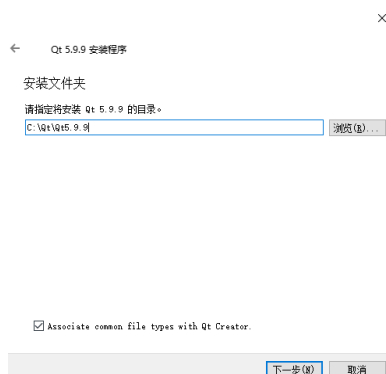


图 1-16 Qt 选择安装位置

勾选如图 1-17 所示的 MinGW 5.30 开发组件和 Qt Creator 组件，之后点击下一步。

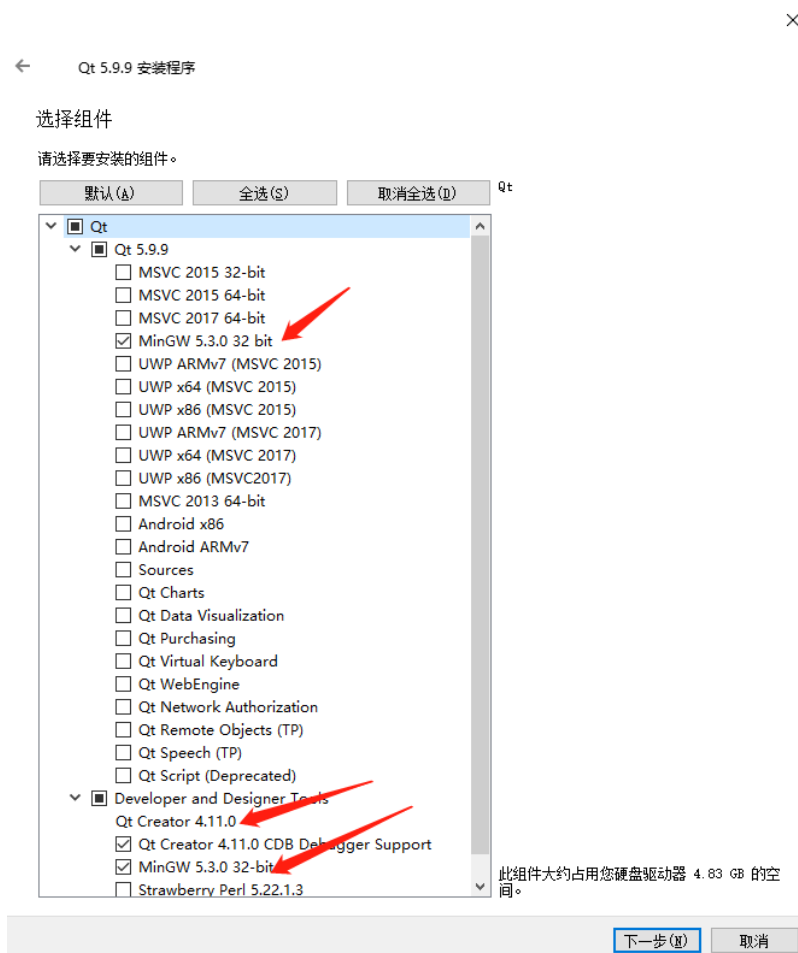


图 1-17 安装组件选择

如图 1-18，选择同意许可和确认接下来的安装信息，一直点击下一步完成安装。

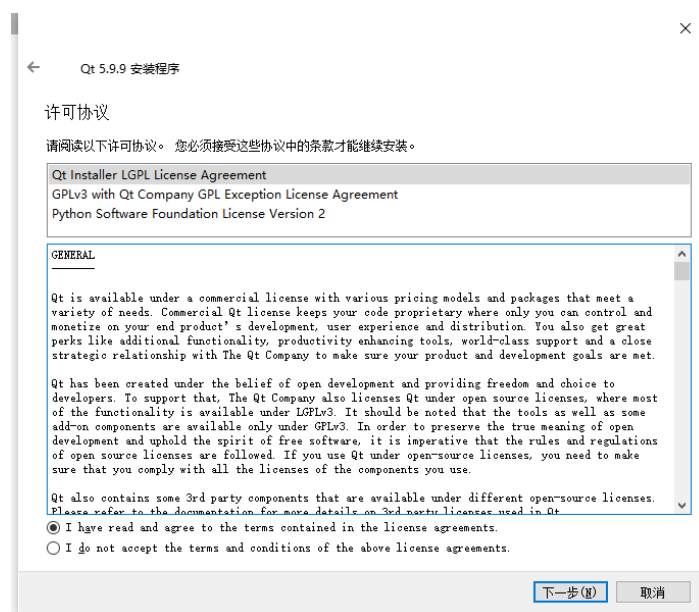


图 1-18 同意许可协议

1.3 程序固化

1.3.1 添加 winbond W25Q128 的支持

参考链接：<https://xilinx.eetrend.com/blog/2021/100112739.html>

由于本加速卡上使用的用于存储 FPGA 启动程序的 FLASH 型号为华邦的 W25Q128，Vivado2021.1 软件默认不支持，需通过在 Xilinx 的 Vivado 软件数据库中做修改，具体操作如下。

使用**文本编辑软件**（例如 Notepad++）（**不要使用 Excel 或者 WPS**）打开
安装目录\Xilinx\Vivado\2021.1\data\xicom\xicom_cfgmem_part_table.csv 文件
再打开**资料\EXAMPLES\winbond FLASH 增加内容.txt** 文件

如图 1-19 所示，将 txt 文件第二行复制到 csv 文件最后一行以获得支持。

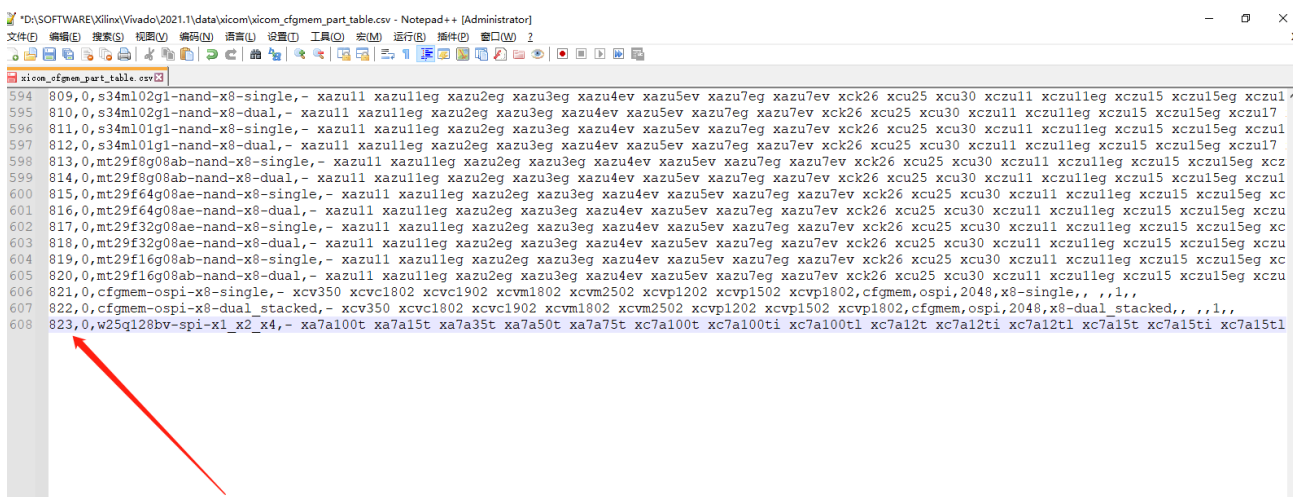


图 1-19 加入 Winbond W25Q128 支持语句

添加完成后，保存 CSV 文件即可，如果 Vivado 在运行的话需要关闭并重启 Vivado。

注：如果使用的 Vivado 版本不是 Vivado2021.1，则需要根据 Vivado 版本选择 txt 文件中对应行粘贴到 CSV 文件中，并修改最左边的序号，使之与前一编号相衔接。

1.3.2 Vivado 固化程序到 Flash

在工程确认测试无误的情况下，在 Generate Bitstream 选项上右键选择 Bitstream Setting，勾选如图 1-20 所示的 -bin_file 选项，再次点击 Generate Bitstream 则会生成烧录 Flash 所使用的 bin 文件。

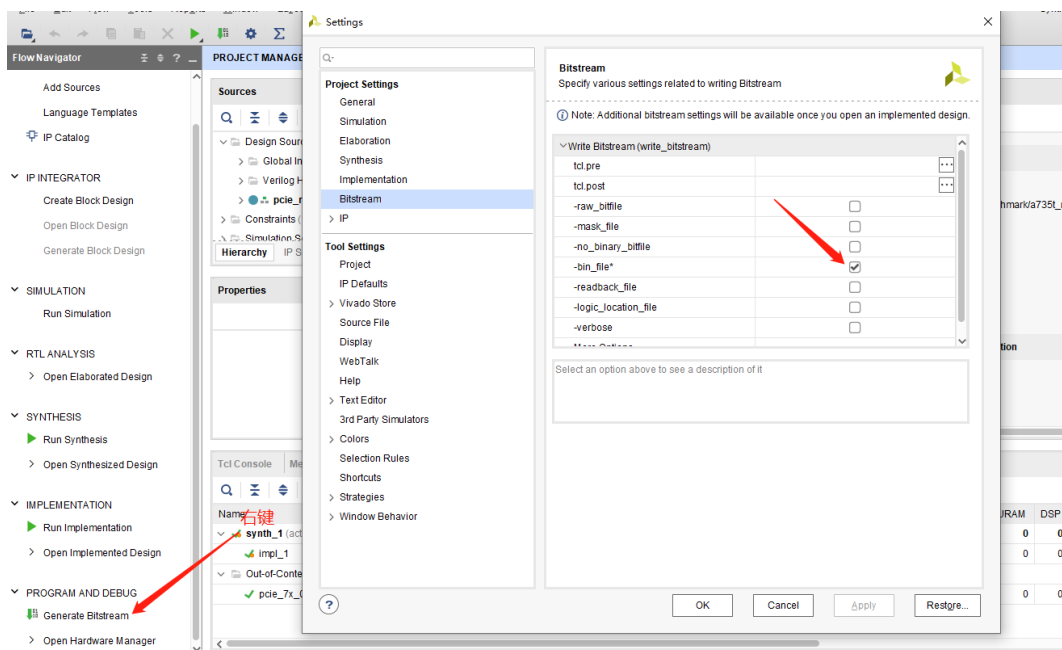


图 1-20 生成烧录 Flash 的 bin 文件

开发板连接好 JTAG 后上电,如图 1-21 所示,在 Hardware Management 里面 Open Target 下选择 Auto Connect 选项,连接上 FPGA。

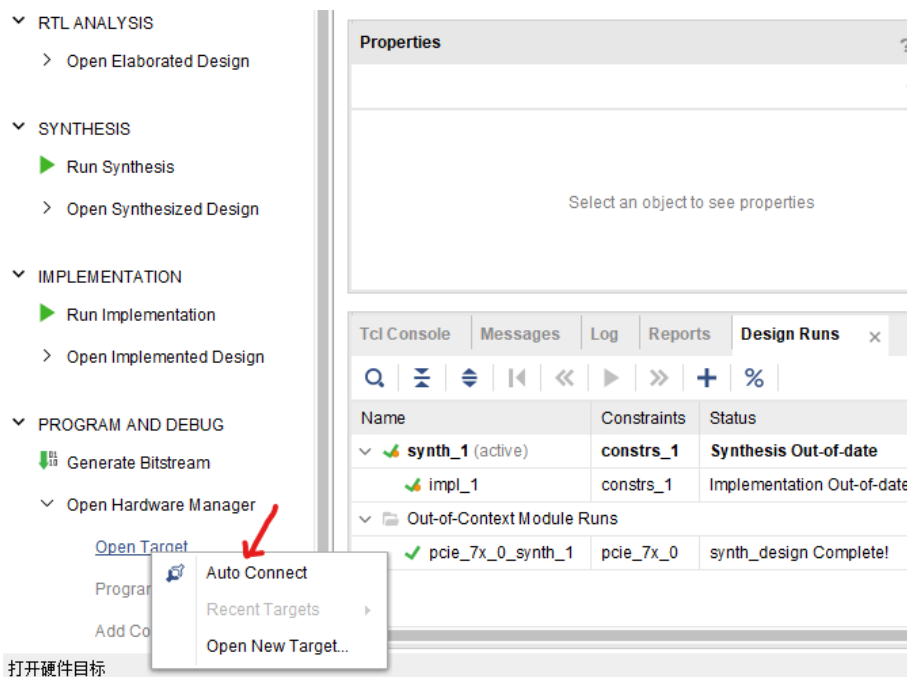


图 1-21 连接到 FPGA

连接上 FPGA 之后,右键选择添加配置存储器设备,如图 1-22 所示。

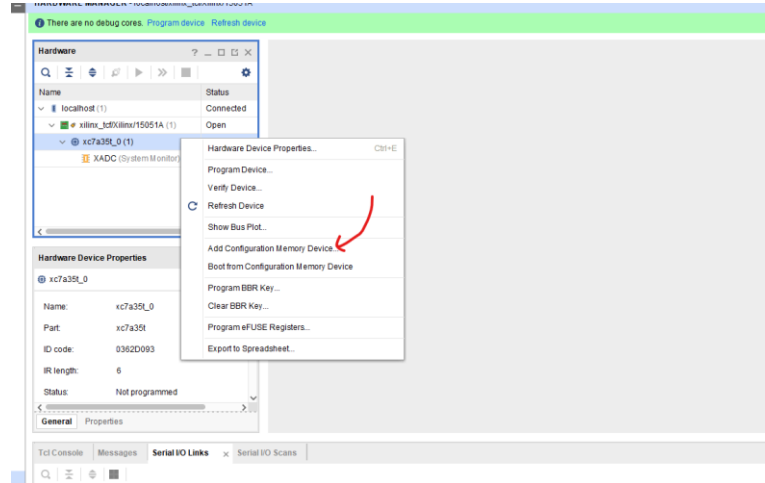


图 1-22 添加配置存储器设备

弹出界面中，搜索框中输入 w25q，出现如图所示的 W25Q128 芯片，选中后点击 OK 完成选择，如图 1-23 所示。在选择的存储器芯片右键，选择 Program memory device。

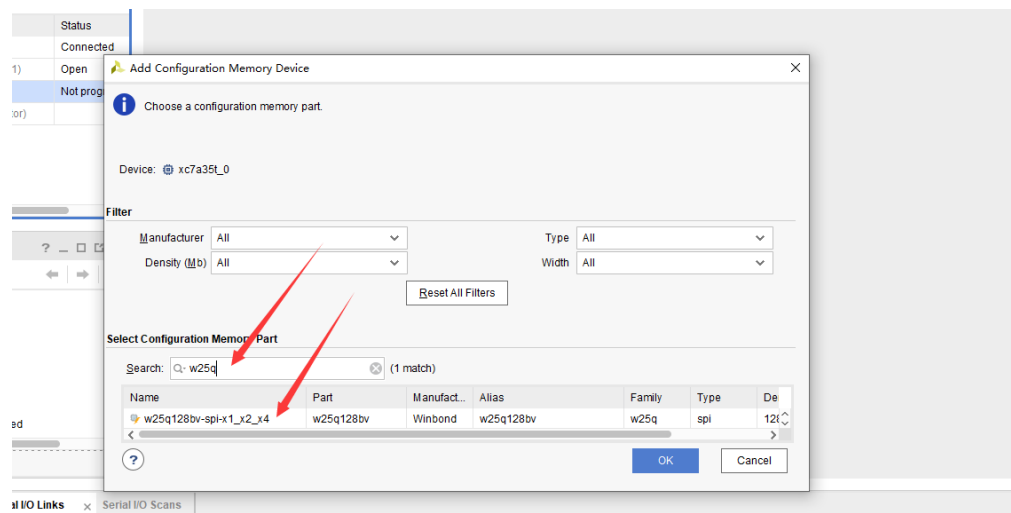


图 1-23 选择 W25Q128 芯片

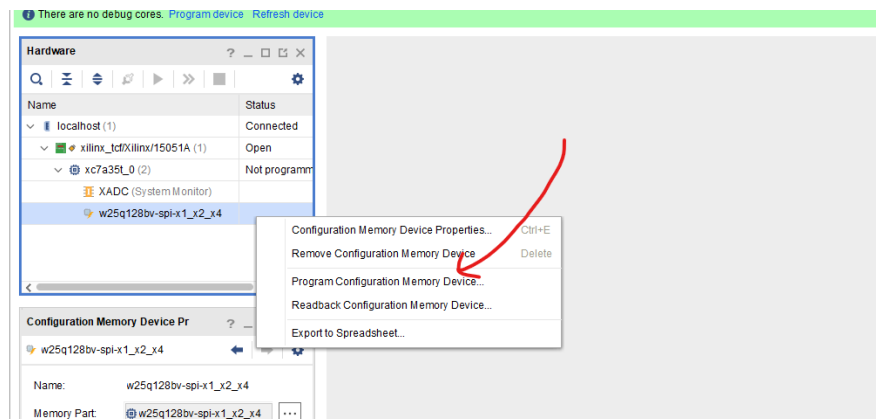


图 1-24 选择烧录存储器

弹出界面中，选择 工程名.runs/impl_1/xx.bin 文件，然后点击 OK 开始烧录，如图 1-25 所示。

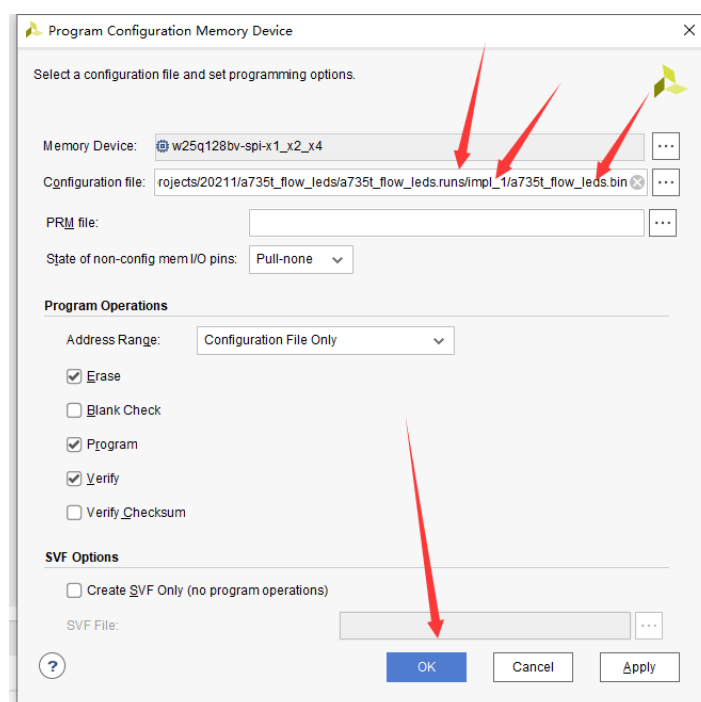


图 1-25 选择烧录 bin 文件

之后会开始对 Flash 进行烧录，直到烧录完成，如图 1-26 所示。

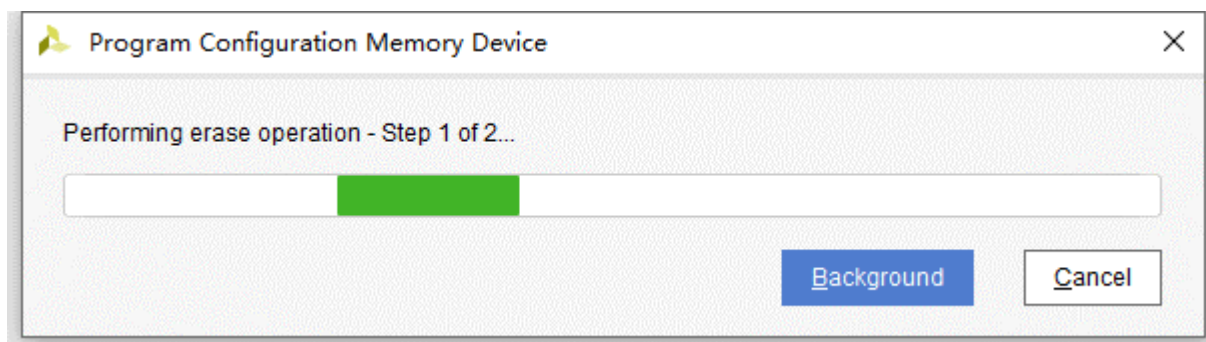


图 1-26 烧录过程

断电后重启，FPGA 会自动从 Flash 中加载程序运行。

第二章 PCIE XDMA 测试

2.1 XDMA 简介

Xilinx 提供的 DMASubsystem for PCIeExpressIP 是一个高性能，可配置的适用于 PCIE2.0, PCIE3.0 的 SG 模式 DMA，提供用户可选择的 AXI4 接口或者 AXI4-Stream 接口。一般情况下配置成 AXI4 接口可以加入到系统 总线互联，适用于大数据量异步传输，通常情况都会使用到 DDR，AXI4-Stream 接口适用于低延迟数据流 传输。

XDMA 是 SGDMA，并非 Block DMA，SG 模式下，主机会把要传输的数据组成链表的形式，然后将链表 首地址通过 BAR 传送给 XDMA，XDMA 会根据链表结构首地址依次完成链表所指定的传输任务。XDMA 的结构如图 2-1 所示。

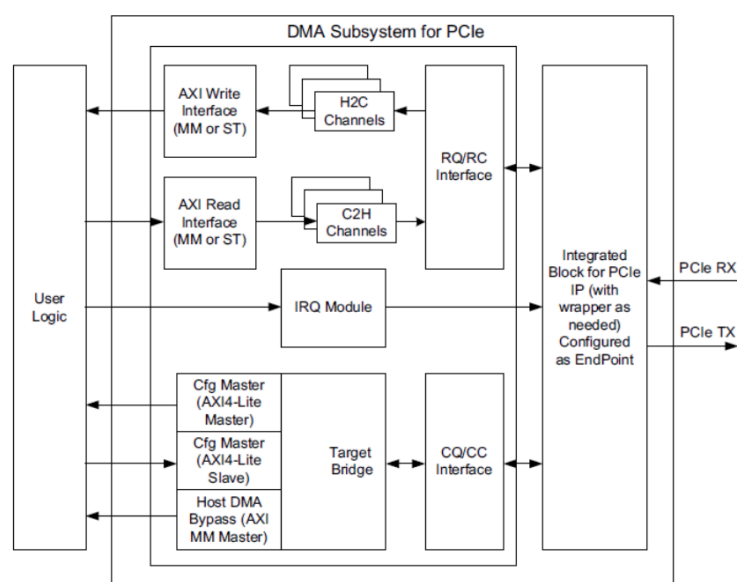


图 2-1 XDMA 结构图

2.2 XDMA 特性

XDMA 提供如下接口：

AXI、AXI4-Stream，必须选择其中一个，用来数据传输。

AXI4-Lite Master，可选，用来实现 PCIE BAR 地址到 AXI4-Lite 寄存器地址的映射，可用来读写用户逻辑寄存器。

AXI4-Lite Slave，可选，用来将 XDMA 内部寄存器开放给用户逻辑，用户逻辑可以通过此接口访问 XDMA 内部寄存器，不会映射到 BAR。

AXI4 Bypass 接口，可选，用来实现 PCIE 直通用户逻辑访问，可用于低延迟数据传输。

2.3 XDMA 测试工程创建

打开 Vivado 软件，选择 Create Project 并点击 Next。如图 2-2 所示。

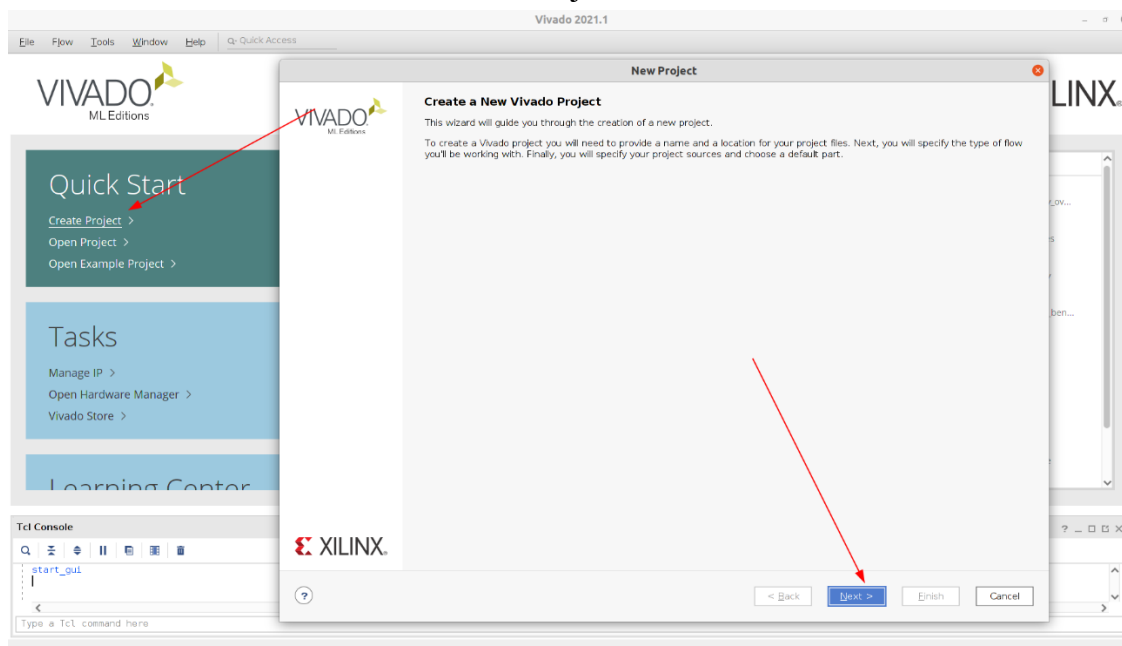


图 2-2 创建 Vivado 工程

输入工程名称和工程存放路径,以 m2_artix7_xdma 为例,如图 2-3 所示.

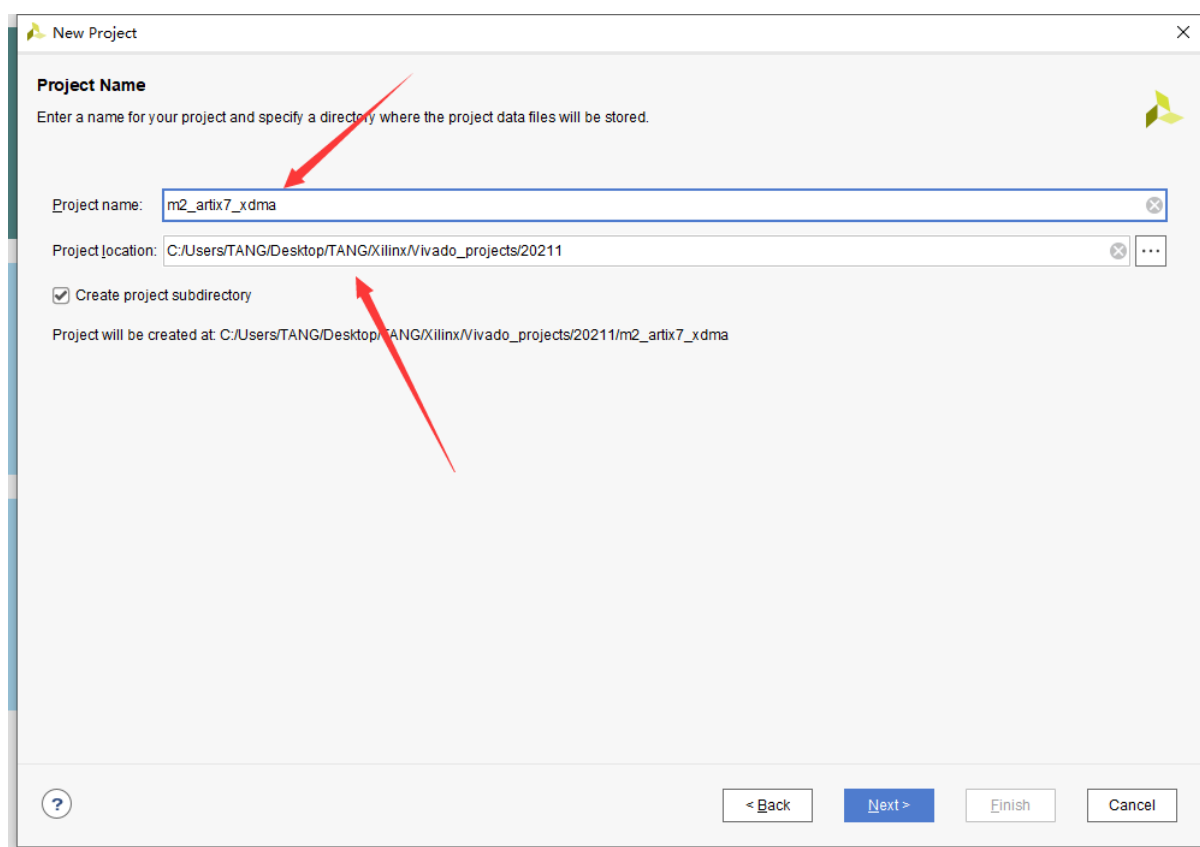


图 2-3 输入工程名称和路径

选择 RTL Project 并且勾选”当前不指定源文件”,如图 2-4 所示.

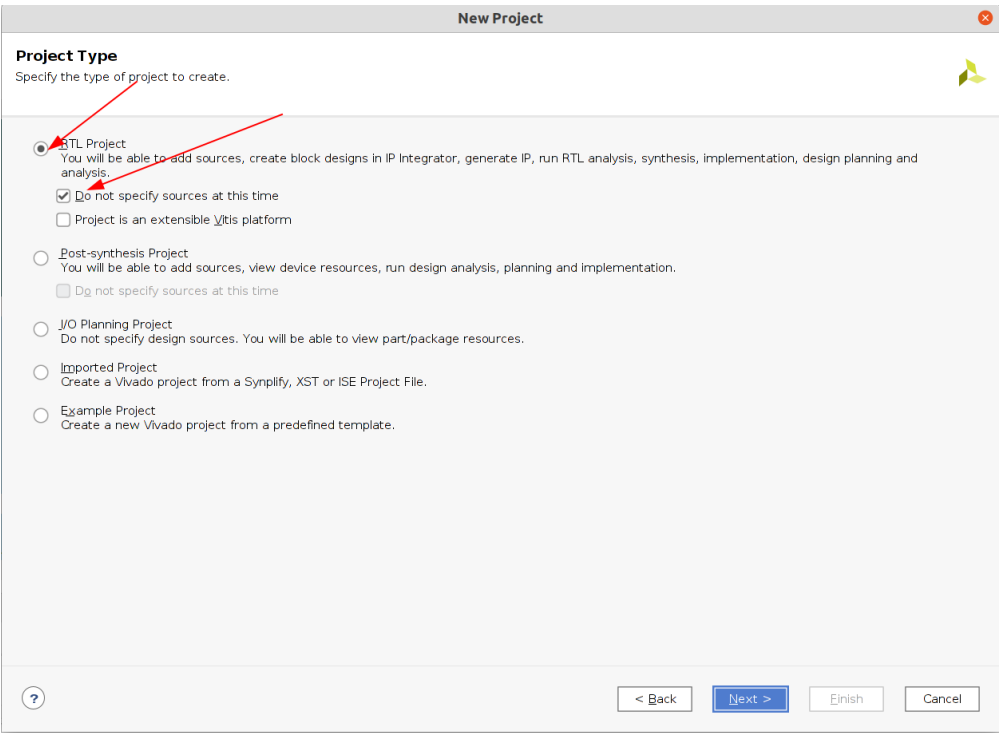


图 2-4 选择 RTL Project

根据核心板芯片型号选择芯片后点击 Next,以 XC7A35T-2FGG484I 为例,如图 2-5 所示.

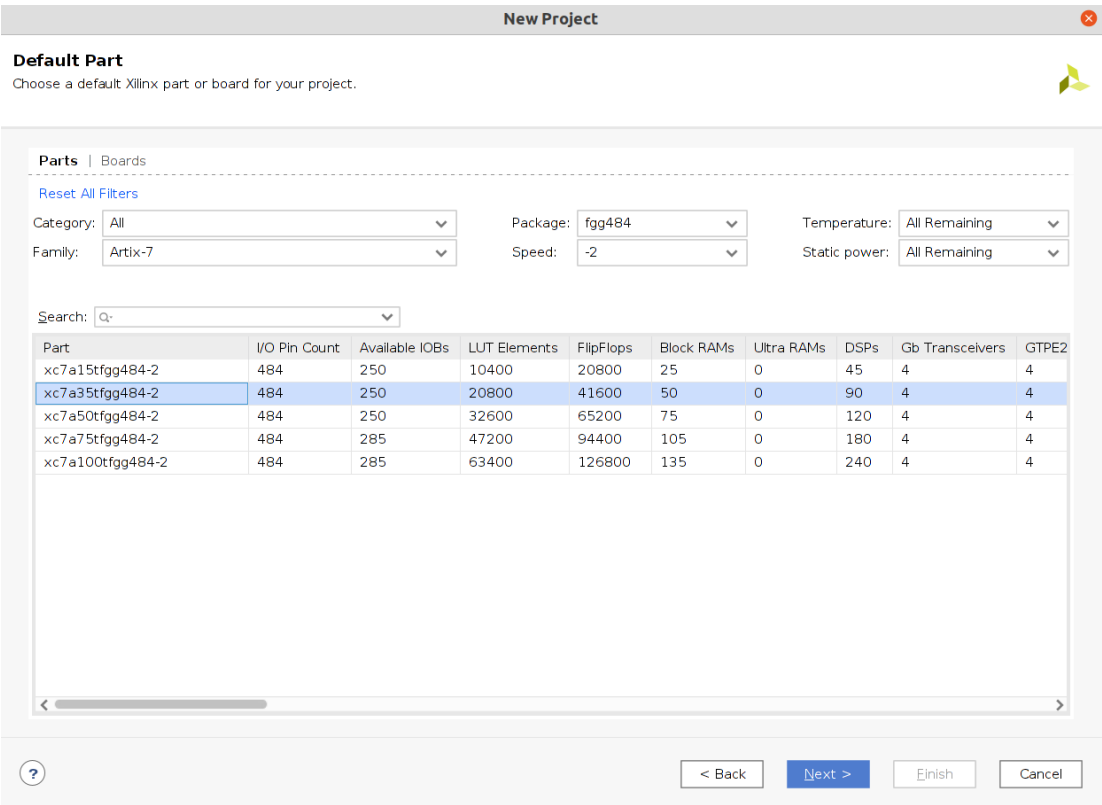


图 2-5 选择芯片型号

点击 Create Block Design，输入 Block Design 名称后点击 OK，如图 2-6 所示。

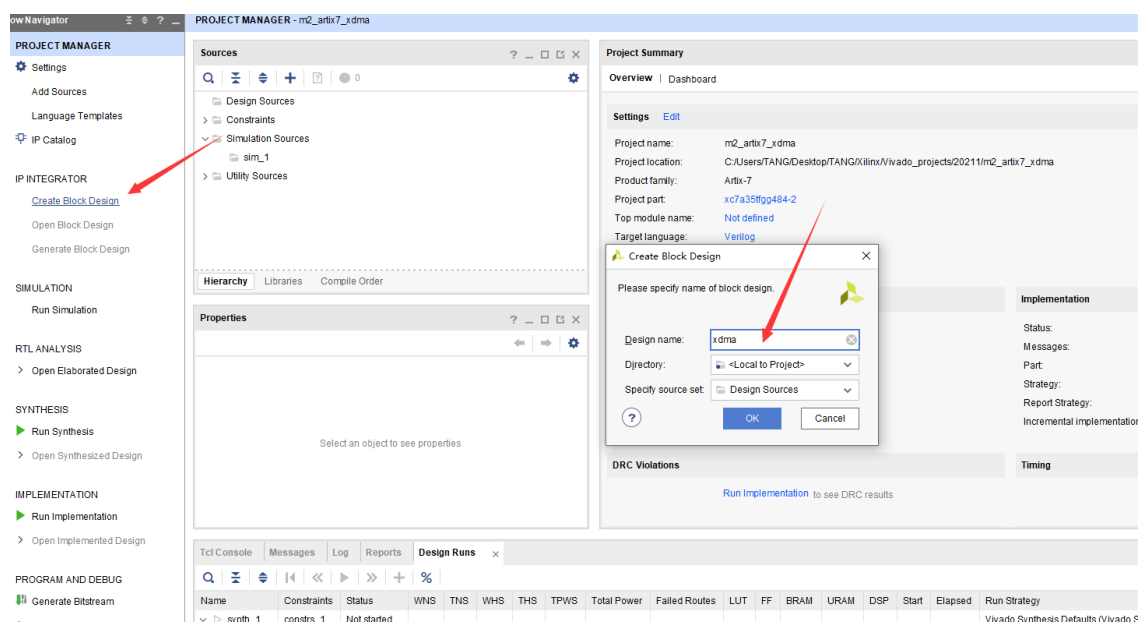


图 2-6 创建 Block Design

点击+号添加 IP，搜 xdma，点击添加 xdma IP，如图 2-7 所示。

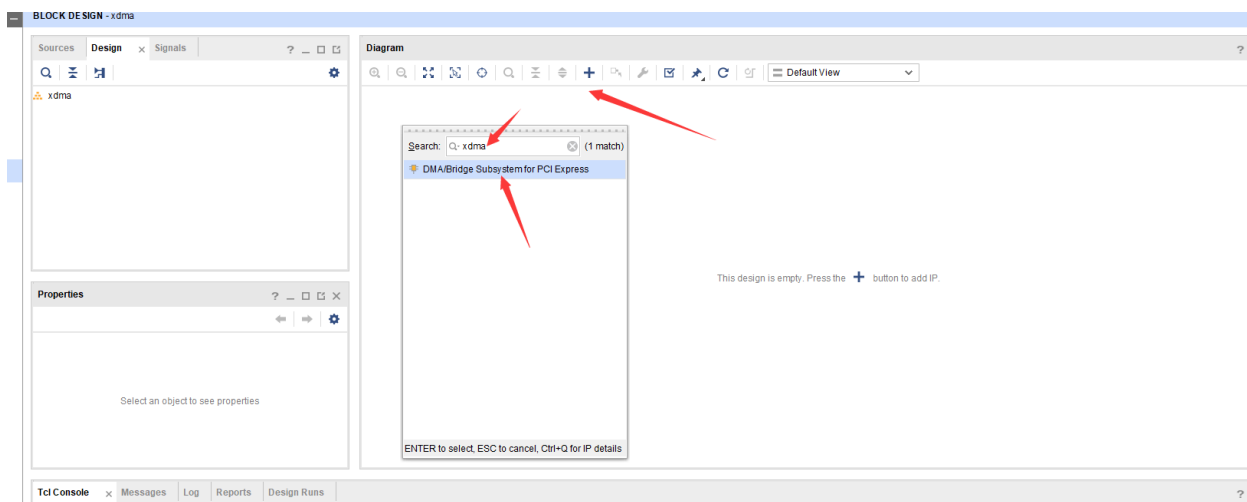


图 2-7 添加 XDMA IP

2.4 XDMA IP 配置

2.4.1 Basic 项

Mode: 配置模式，选择 BASE 配置

Lane Width: 选择 PCIE 的通道数量，M2 加速卡最多为 4 条，通道数量越多通信速度越快，用户需要根据硬件的实际通达数量选择正确的通道数。（35T 由于资源原因只能选择 *2，非 35T 可以选择为 *4），本工程以 A7-35T 为例。

Max Link Speed: 选择 5.0GT/s 即 PCIE2.0。

Reference Clock : 100MHZ, 参考时钟 100M

DMA Interface Option: 接口选择 AXI4 接口

AXI Data Width: 64bit, 即 AXI4 数据总线宽度为 64bit

AXI Clock: 125M, 即 AXI4 接口时钟为 125MHZ

DMA Interface option 设置为 AXI Memory Mapped 方式

如图 2-8 所示。

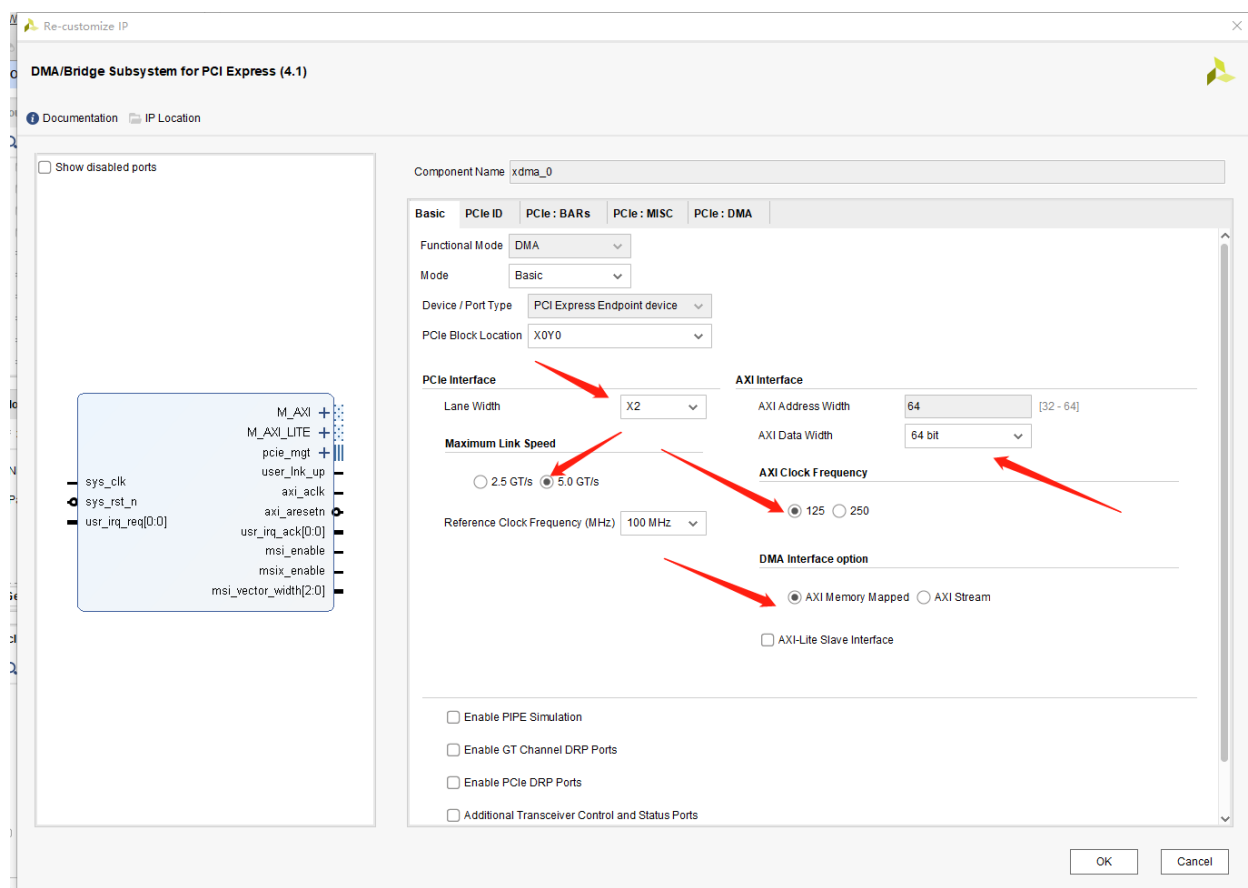


图 2-8 Basic 配置

2.4.2 PCIE ID 配置

PCIE ID 配置, 这里选择默认的配置就可以, 默认的设备类型是 Simple communication controllers。如图 2-9 所示。

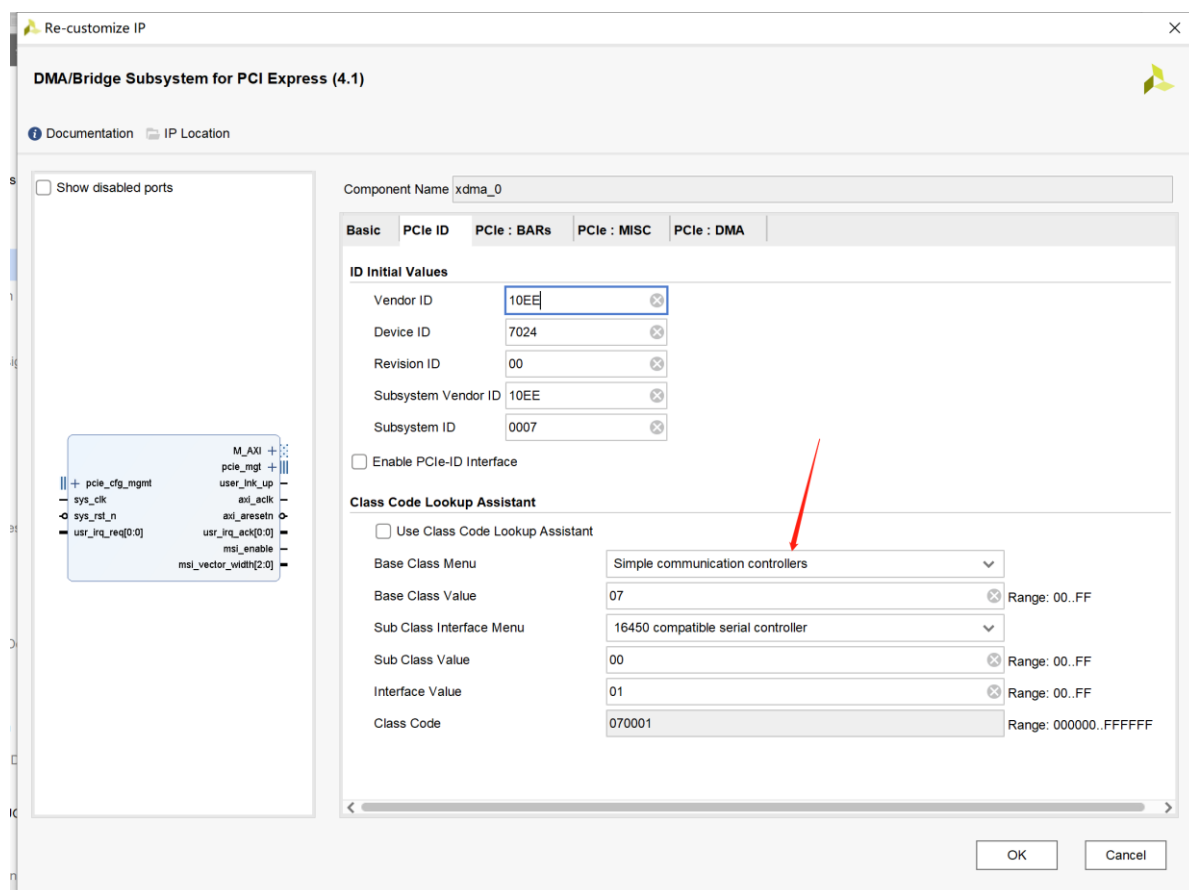


图 2-9 PCIE ID 配置

2.4.3 PCIE BAR 配置

PCIE BAR 配置，这里的配置比较重要，首先使能 PCIE to AXI Lite Master Interface，这样可以在主机一侧通过 PCIE 来访问用户逻辑侧寄存器或者其他 AXI4-Lite 总线设备映射空间选择 1M，当然用户也可以根据实际需要来自定义大小。

PCIE to AXI Translation：这个设置比较重要，通常情况下，主机侧 PCIE BAR 地址与用户逻辑侧地址是不一样的，这个设置就是进行 BAR 地址到 AXI 地址的转换，比如主机一侧 BAR 地址为 0，IP 里面转换设置为 0x40000000，则主机访问 BAR 地址 0 转换到 AXI Lite 总线地址就是 0x40000000

PCIE to DMA Interface：选择 64bit 使能

DMA Bypass 暂时不用。

PCIE BAR 设置如图 2-10 所示。

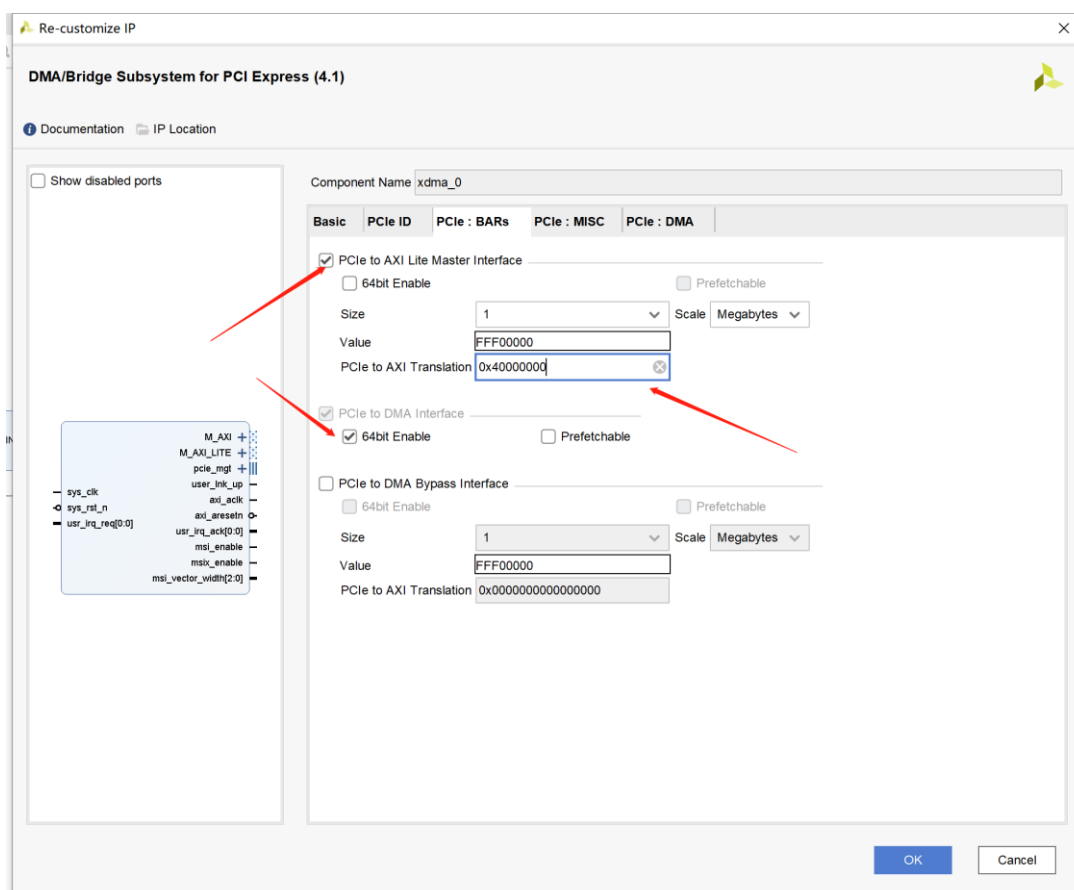


图 2-10 PCIE BAR 配置

2.4.4 PCIE 中断配置

Legacy interrupt Settings: 传统中断设置，设置为 NONE，即不使用传统中断；

Enable MSI Capability Structure: 不勾选以禁用 MSI 中断功能；

Enable MSI-X Capability Structure: 不勾选以禁用 MSI-X 中断功能；

Configuration Management Interface: PCIe 配置管理接口一般用不到，所以取消勾选。

配置后的 PCIe:Misc 标签页如图 2-11 所示。

关于 PCIe:Misc 标签页的部分选项介绍如下：

Number of User Interrupt Request: 用户中断请求数，最多可以选择 16 个用户中断请求；

Legacy Interrupt Settings: 可以选择传统中断之一：INTA，INTB，INTC 或 INTD；

MSI Capabilities: 默认情况下，启用 MSI 功能，并且启用 1 个向量，最多可以选择 32 个向量。通常，Linux 仅将 1 个向量用于 MSI，这里开启此选项；

MSI-X Capabilities: 勾选以使用 MSI-X 功能；

Finite Completion Credits (Advanced 配置模式下有): 在支持有限完成信用的系统上，可以启用此选项以获得更好的性能；

Extended Tag Field: 扩展标签字段，默认情况下，使用 6 位完成标签。对于 UltraScale 和 Virtex-7 器件，扩展标签选项提供了 64 个标签。对于 UltraScale+器件，扩展标签选项提供

256 个标签。如果未选择扩展标签选项，则 DMA 将 32 个标签用于所有设备。这里使用该选项。

Configuration Management Interface: 是否使用 PCIe 配置管理接口，这里不使用；
关于 MSI-X 中断: 可以尝试使用 MSI-X 中断，而不是 MSI 或传统中断。使用 MSI-X 中断，数据速率要优于使用 MSI 或基于传统中断的设计。

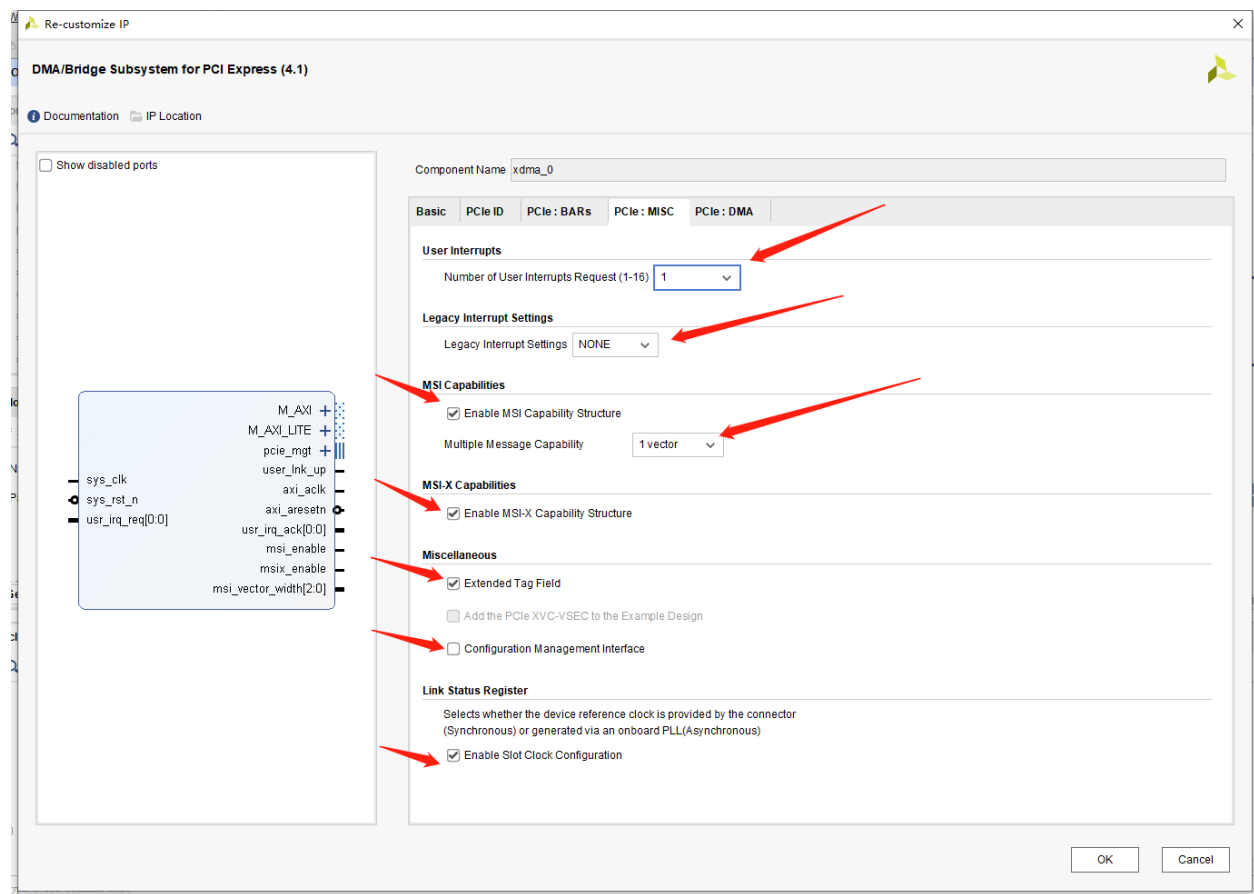


图 2-11 PCIe 中断配置

2.4.5 PCIe DMA 配置

Number of DMA Read Channel（H2C）和 Number of DMA Write Channel（C2H）通道数，对于 PCIe2.0 来说最大只能选择 2，也就是 XDMA 可以提供最多两个独立的写通道和两个独立的读通道，独立的通道对于实际应用中 有很大的作用，在带宽允许的前提前，一个 PCIe 可以实现多种不同的传输功能，并且互不影响。这里我们选择 2

Number of Request IDs for Read（Write）channel: 这个是每个通道设置允许最大的 outstanding 数量，按照默认即可。DMA 配置如图 2-12 所示。

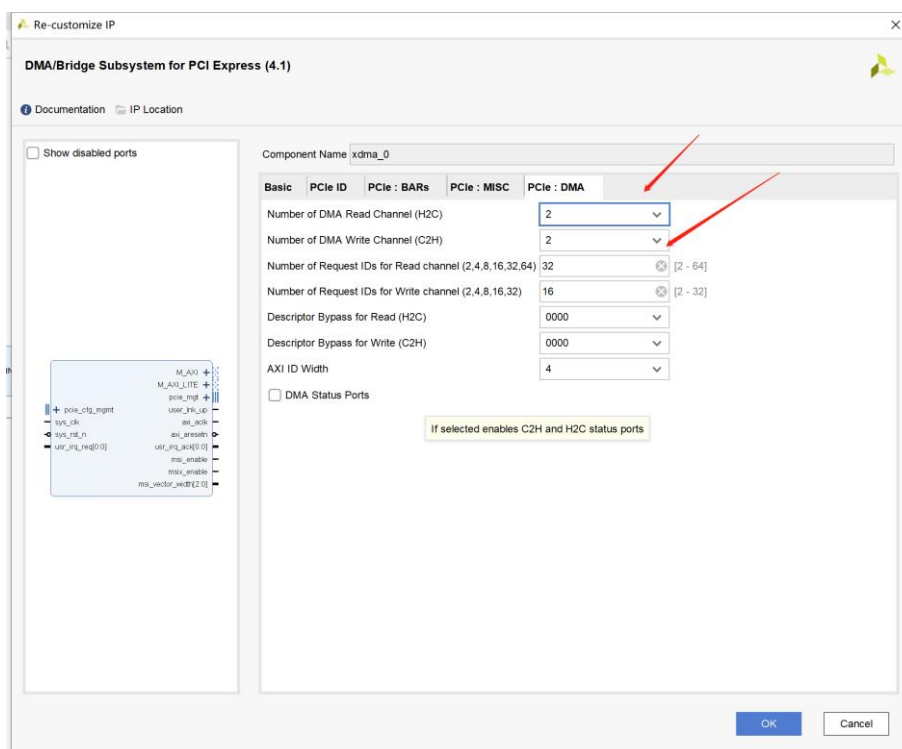


图 2-12 PCIE DMA 配置

至此完成 PCIE XDMA IP 核配置，点击 OK。

点击 Run Block Automation，在该界面中我们需要检查“Options”中的配置信息与我们刚才配置 XDMA 时的选项是否对应，如果不 对应需要改回来（这可以说是 Vivado 的一个 bug），重点注意的配置项已在上图中用红框标出。检查配置 无误后点击右下角的“OK”按钮。如图 2-13 所示。

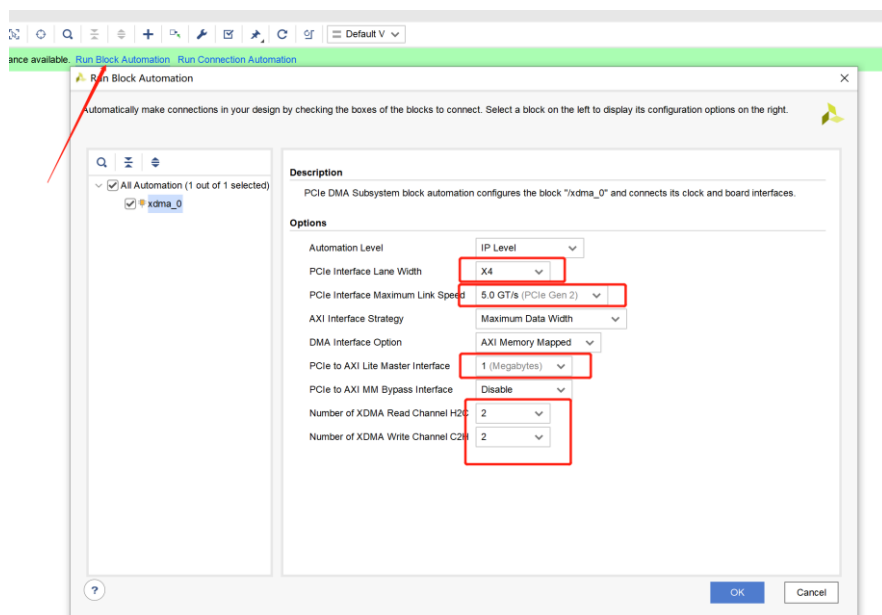


图 2-13 自动处理

自动处理完成后，得到如图 2-14 所示的结构。

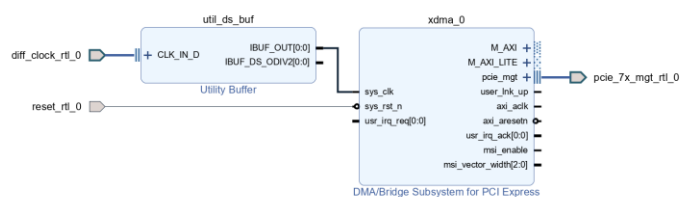


图 2-14 自动处理完成

2.4.6 剩余配置

之后在添加 IP 选项下，添加一个 AXI BRAM Controller 模块，数据宽度设置为 64，配置过程如图 2-14、 2-15 所示，按照箭头所指进行配置，其他项保持默认即可。

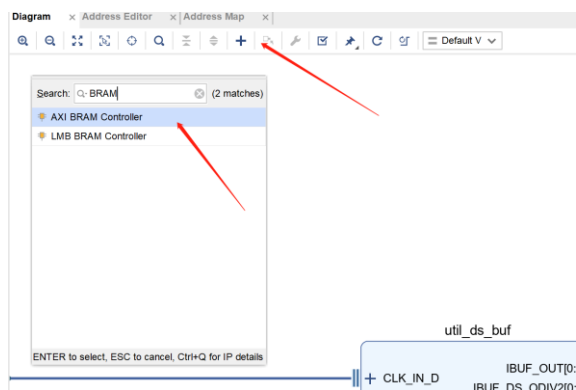


图 2-14 选择添加 AXI BRAM Controller

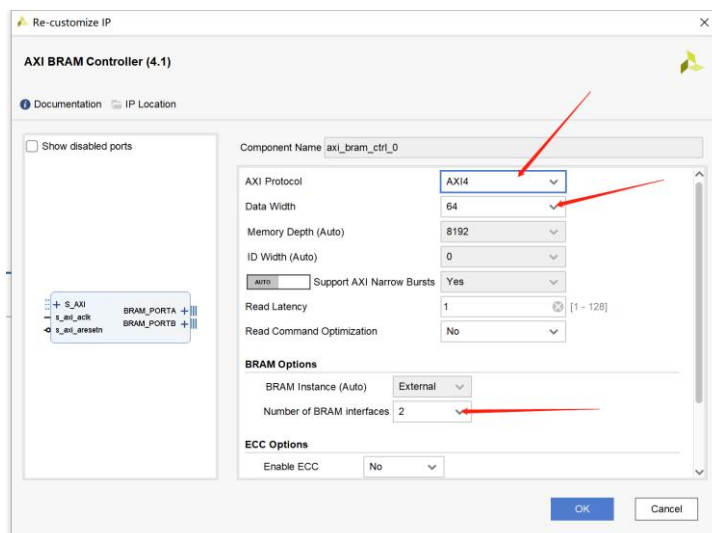


图 2-15 选择配置 AXI BRAM Controller

添加完如图 2-16 所示

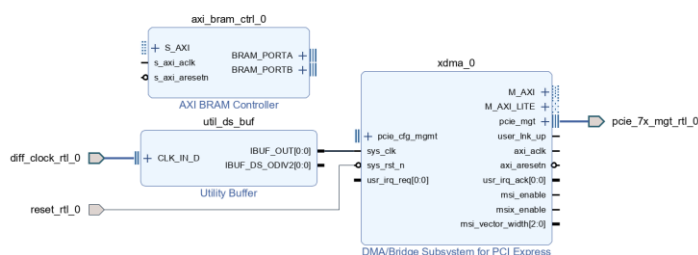


图 2-16 添加完 BRAM Controller

创建一个 AXI-GPIO IP 用于控制板子上的三个 LED 灯，IP 配置如图 2-17 所示。

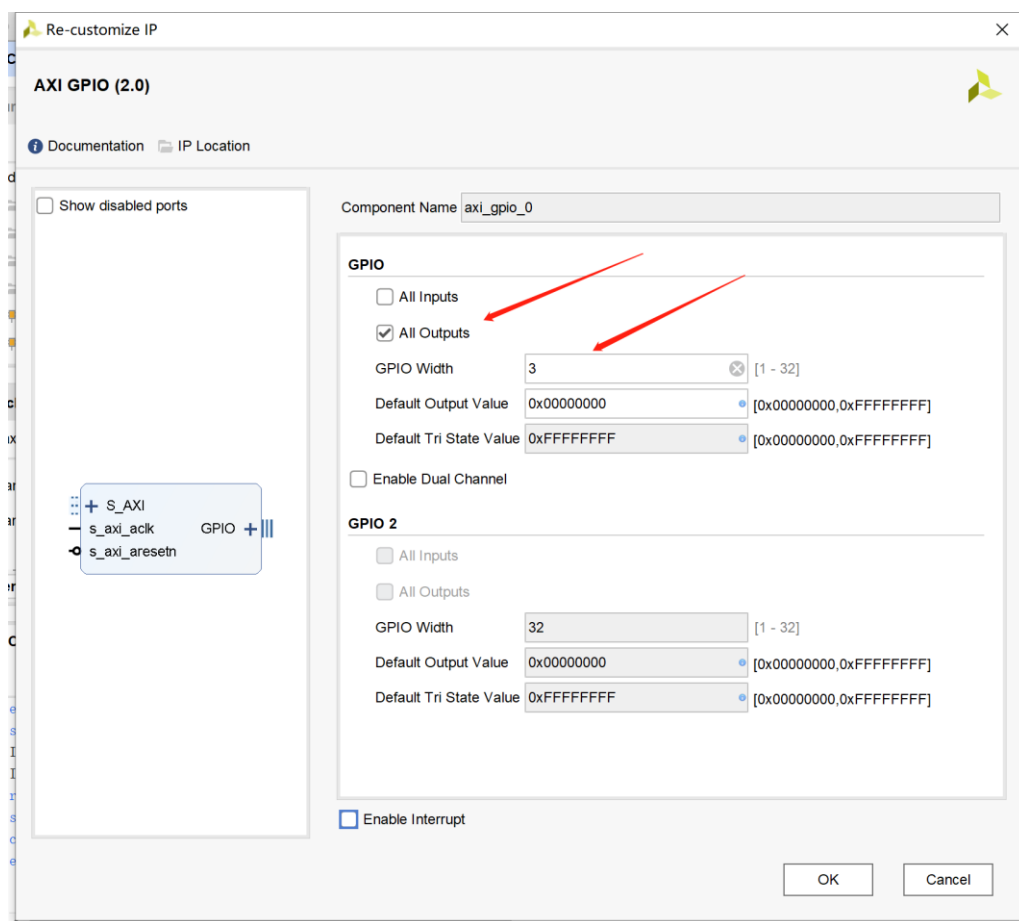


图 2-17 创建 AXI GPIO IP

点击“Run Connection Automation”，在弹出的窗口中勾选“All Automation”，让 Vivado 工具完成剩下的接口连接工作。如图 2-18 所示。可以看到，Vivado 工具自动为我们添加了互联 IP。由于 XDMA 的 M_AXI 接口连接的是内存，带宽要求高，所以 Vivado 工具选择了比 AXI Interconnect IP 性能更好的 AXI SmartConnect IP。M_AXI_LITE 接口用于驱动和控制用户自己的逻辑（如控制 LED 的流水效果等），一般带宽要求不高，所以

Vivado 工具选择了通用的 AXI Interconnect IP。

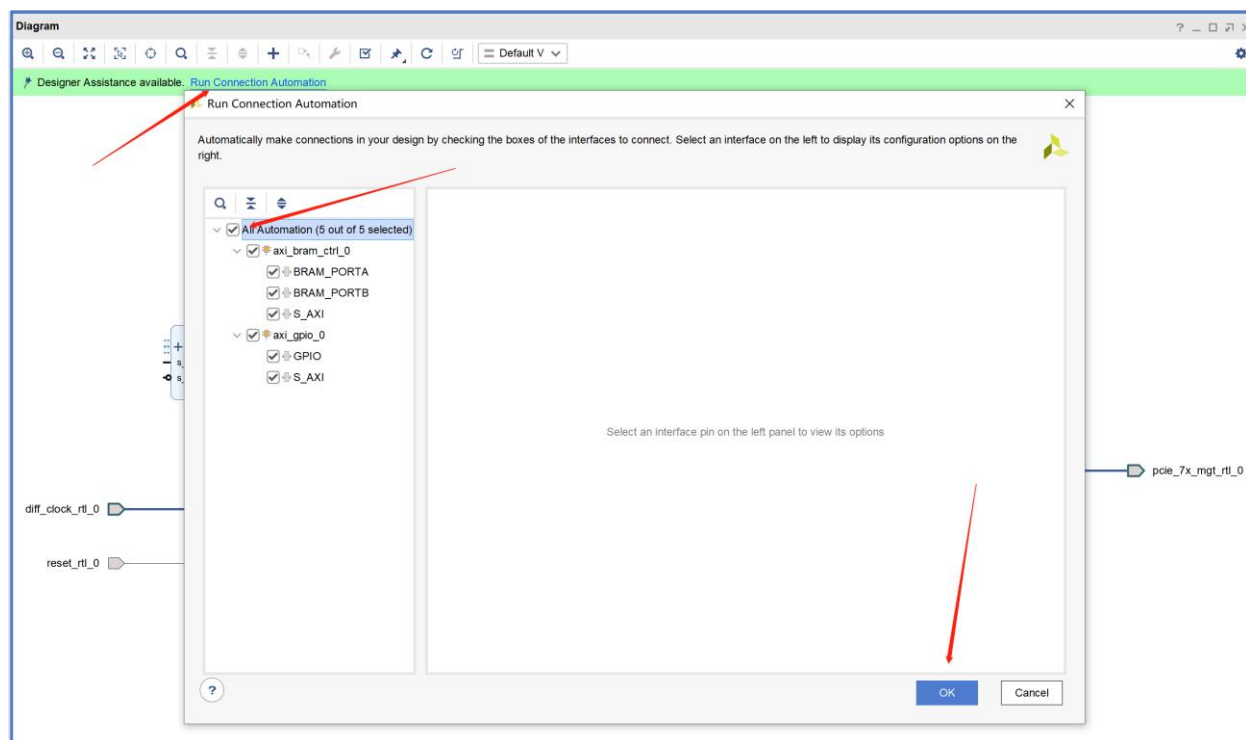


图 2-18 自动连接

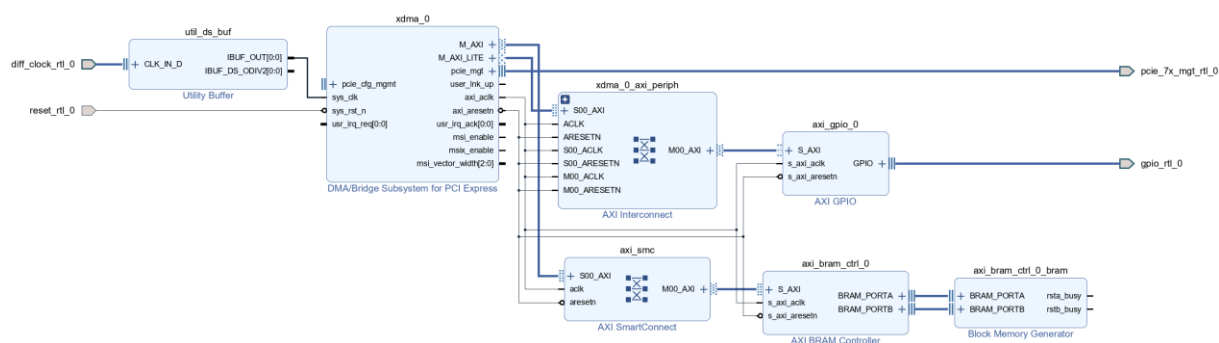


图 2-19 自动连线结果

Block Memory Generator IP 的 `rsta_busy` 和 `rstb_busy` 信号（用于 Block Memory Generator 的内部安全电路，一般不需要）我们用不到，留着影响美观，可以去掉。双击 Block Memory Generator IP，打开其配置窗口，在 **Other Options** 标签页中取消勾选“Enable Safety Circuit”，然后点击“OK”按钮即可，如图 2-20 所示：

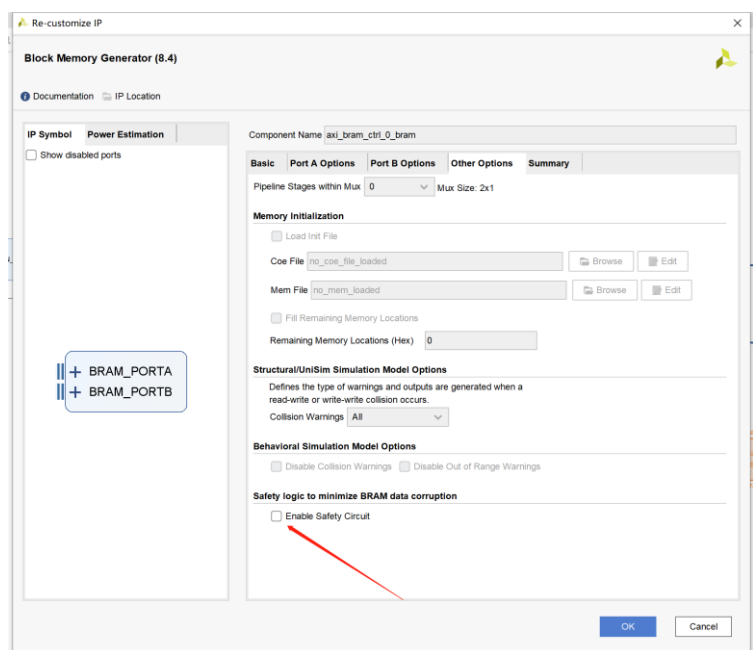


图 2-20 去除 BRAM 模块 safety circuit

系统搭建虽然已经完成了，不过还剩下关键的一步——修改地址映射。如图 2-21 所示，与 XDMA 的 M_AXI 接口连接的 BRAM 偏移地址（Offset Address）改为 0x0000，当然也可不改，笔者试过使用其默认值是可以的。重点注意的是与 XDMA 的 M_AXI_LITE 接口连接的用户设计偏移地址，该偏移地址需要与 XDMA 的配置标签页 PCIe:BARs 中的“PCIe to AXI Lite Master Interface:”配置项下的“PCIe to AXI Translation:”PCIe 到 AXI 的地址转换相同。由于 XDMA 的 M_AXI_LITE 接口的偏移地址是 Vivado 工具自动分配的，一般不会有什问题，推荐根据 XDMA 的 M_AXI_LITE 接口的偏移地址配置 XDMA 配置标签页 PCIe:BARs 中的“PCIe to AXI Translation:”。

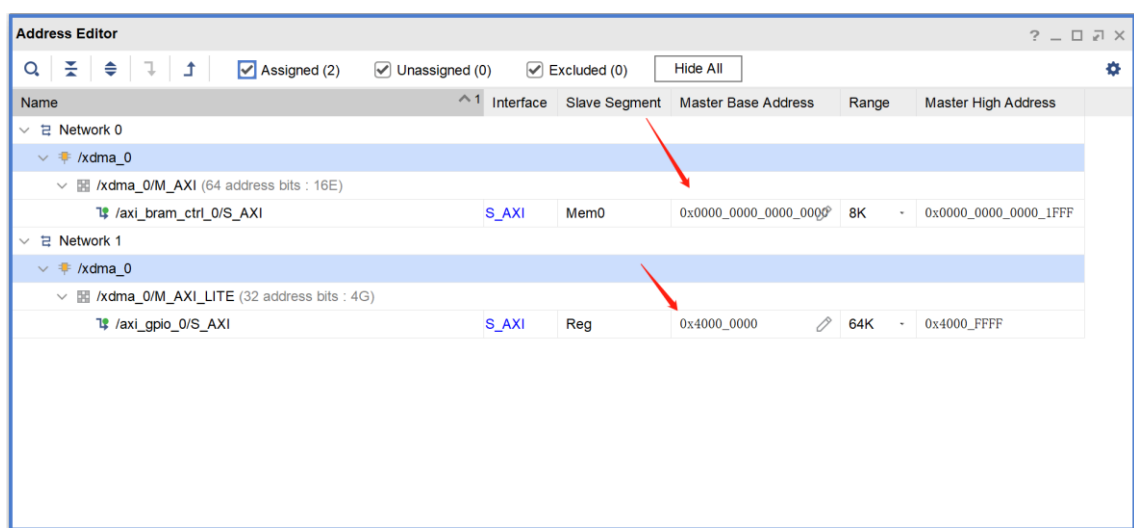


图 2-21 地址空间编辑

编辑完成后，点击如图 2-22 所示的 Valid Design，确保没有 Warning 和 Error，完成 Block

Design 配置。

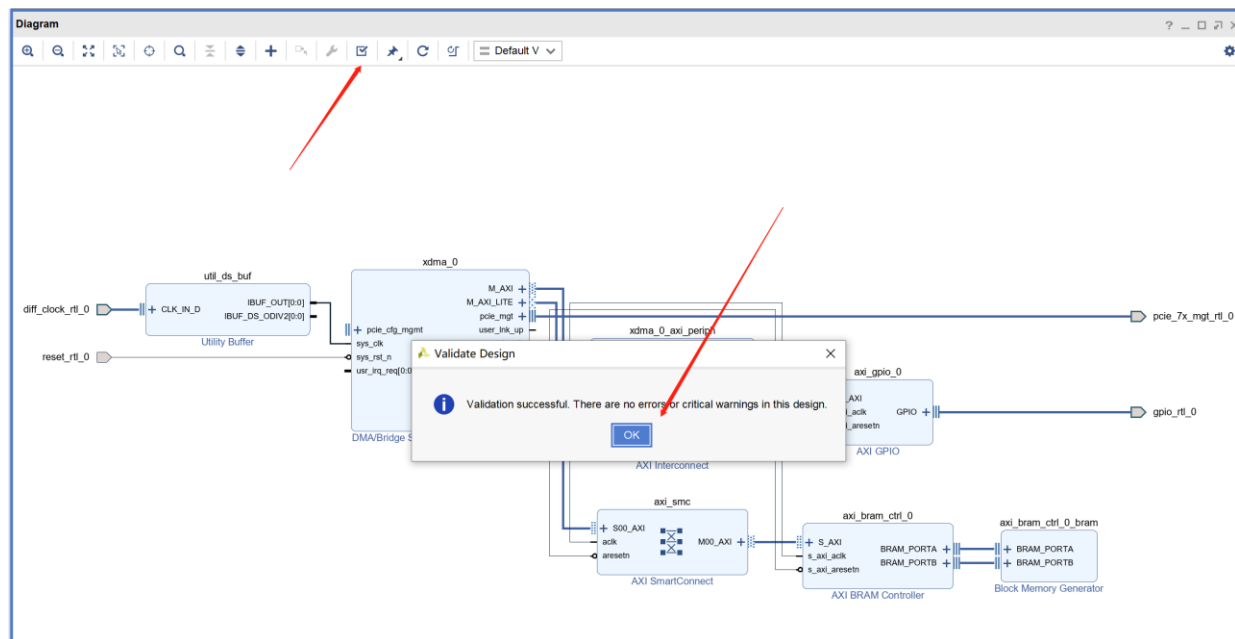


图 2-22 完成 Block Design

2.5 生成 Bin 文件和烧录

右键 Block Design，选择如图 2-23 所示生成，生成输出文件。

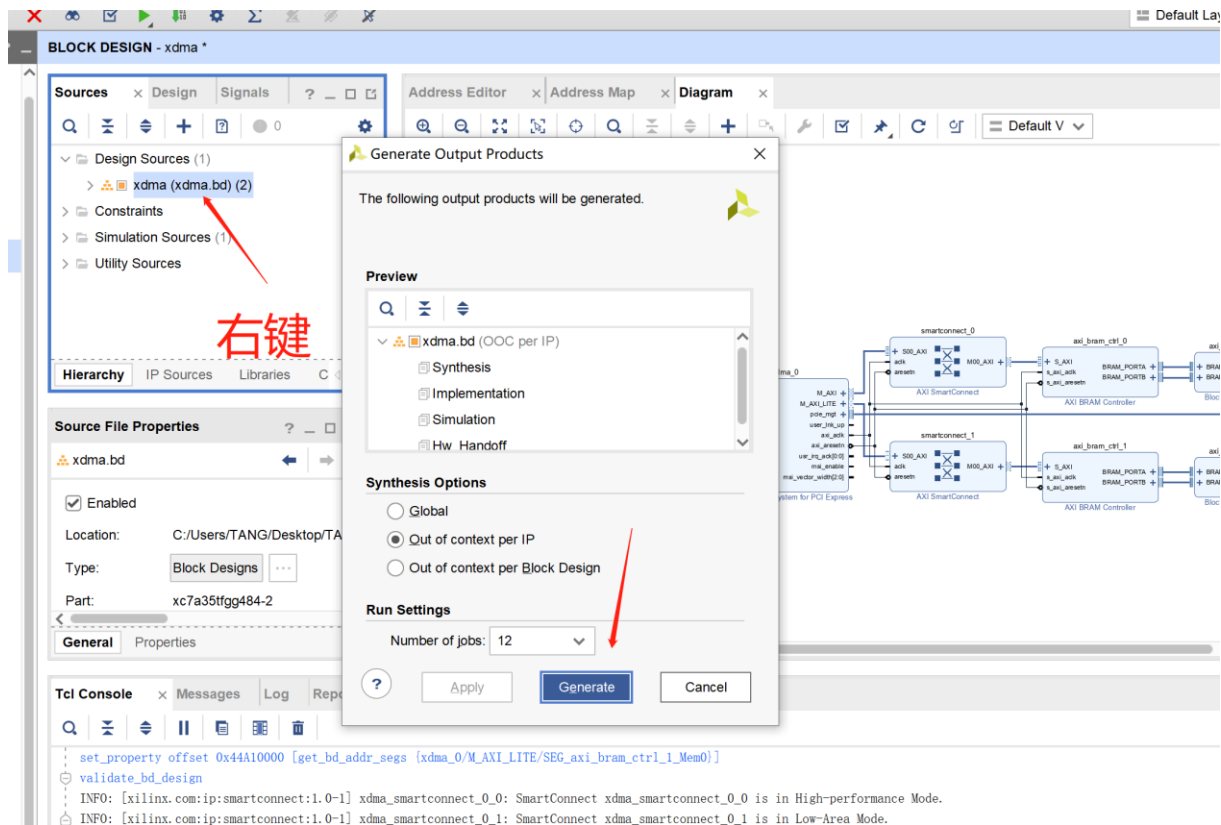


图 2-23 生成输出文件

右键 Block Design，选择如图 2-24 所示按照默认配置生成 Wrapper。

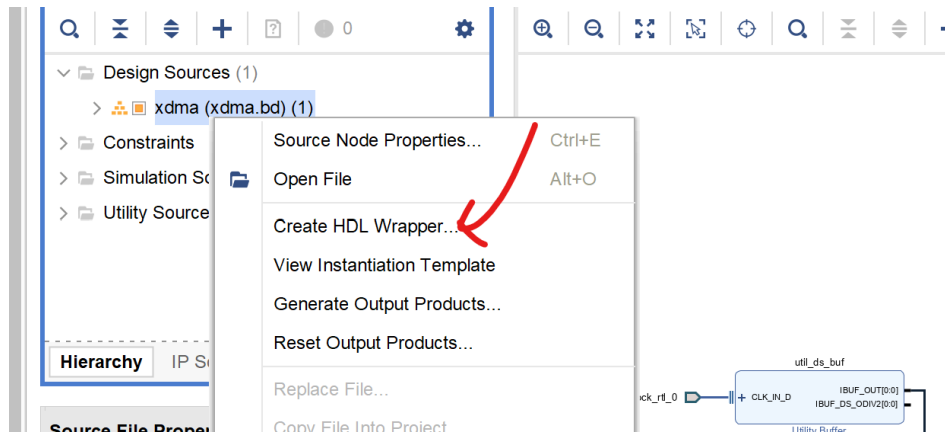


图 2-24 生成 HDL Wrapper

创建.xdc 约束文件，使用下面的约束内容。

```
#bit compress spix4 speed up
set_property CFGBVS VCC0 [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

#pcie reset_n input
set_property -dict {PACKAGE_PIN K22 IOSTANDARD LVCMOS33} [get_ports reset_rtl_0]

#led*3 outputy
set_property -dict {PACKAGE_PIN AB7 IOSTANDARD LVCMOS33} [get_ports
gpio_rtl_0_tri_o[2]]
set_property -dict {PACKAGE_PIN AA6 IOSTANDARD LVCMOS33} [get_ports
gpio_rtl_0_tri_o[1]]
set_property -dict {PACKAGE_PIN AB6 IOSTANDARD LVCMOS33} [get_ports
gpio_rtl_0_tri_o[0]]

#pcie lanes
set_property PACKAGE_PIN F10 [get_ports {diff_clock_rtl_0_clk_p[0]}]

set_property PACKAGE_PIN D9 [get_ports {pcie_7x_mgt_rtl_0_rxp[3]}]
set_property PACKAGE_PIN B10 [get_ports {pcie_7x_mgt_rtl_0_rxp[2]}]
set_property PACKAGE_PIN D11 [get_ports {pcie_7x_mgt_rtl_0_rxp[1]}]
set_property PACKAGE_PIN B8 [get_ports {pcie_7x_mgt_rtl_0_rxp[0]}]
set_property PACKAGE_PIN A8 [get_ports {pcie_7x_mgt_rtl_0_rxn[0]}]
set_property PACKAGE_PIN C11 [get_ports {pcie_7x_mgt_rtl_0_rxn[1]}]
set_property PACKAGE_PIN A10 [get_ports {pcie_7x_mgt_rtl_0_rxn[2]}]
set_property PACKAGE_PIN C9 [get_ports {pcie_7x_mgt_rtl_0_rxn[3]}]
set_property PACKAGE_PIN B4 [get_ports {pcie_7x_mgt_rtl_0_txp[0]}]
set_property PACKAGE_PIN D5 [get_ports {pcie_7x_mgt_rtl_0_txp[1]}]
set_property PACKAGE_PIN B6 [get_ports {pcie_7x_mgt_rtl_0_txp[2]}]
set_property PACKAGE_PIN D7 [get_ports {pcie_7x_mgt_rtl_0_txp[3]}]
set_property PACKAGE_PIN A4 [get_ports {pcie_7x_mgt_rtl_0_txn[0]}]
set_property PACKAGE_PIN C5 [get_ports {pcie_7x_mgt_rtl_0_txn[1]}]
set_property PACKAGE_PIN A6 [get_ports {pcie_7x_mgt_rtl_0_txn[2]}]
set_property PACKAGE_PIN C7 [get_ports {pcie_7x_mgt_rtl_0_txn[3]}]
```

由于本加速卡的 PCIE Lane 顺序和 Xilinx 官方推荐不一致，生成的 PCIE IP Lane 约束和用户约束冲突，所以在 Implementation 中会产生 Critical Warning，不影响功能使用，如确有需要，可以按照如下步骤解除。

打开开如下文件：

\m2_artix7_xdma.gen\sources_1\bd\xdma\ip\xdma_xdma_0_0\ip_0\source\xdma_xdma_0_0_pcie2_ip-PCIE_X0Y0.xdc

将其中 PCIE Lane 的位置按照图 2-25 进行修改，之后再布局布线就不会报 Warning。

(35T 改成 X0Y0-X0Y1, 50T-100T 改成 X0Y0-X0Y3, 200T 改成 X0Y4-X0Y7)

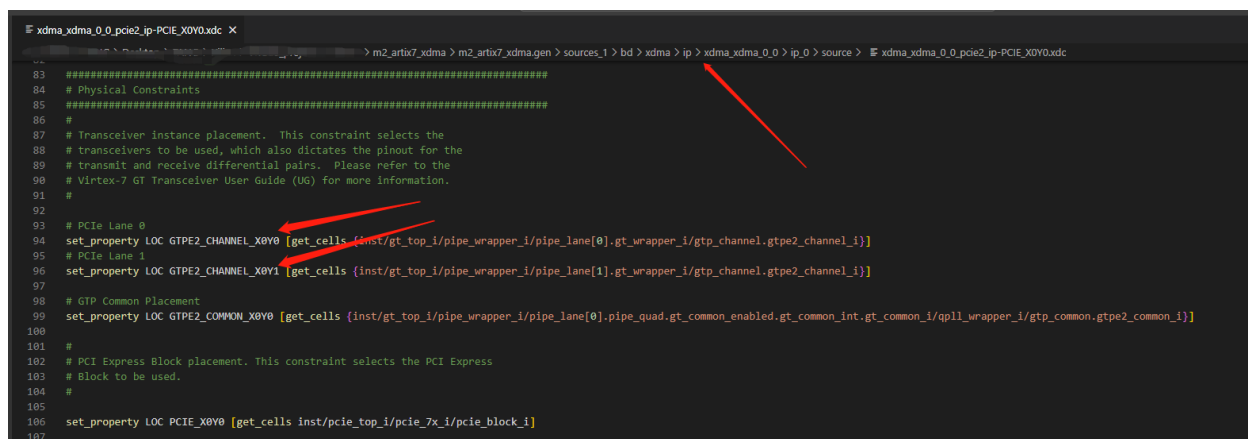


图 2-25 修改 PCIE Lane 位置

编译通过之后按照第 1-3 节生成 Bin 文件并烧录进 Flash 中，准备进行测试，这里不再赘述 Flash 固化过程。

2.6 XDMA 驱动和应用编译（也可以不编译）

打开 VS2015 软件，如图 2-26 所示，选择打开项目选项。

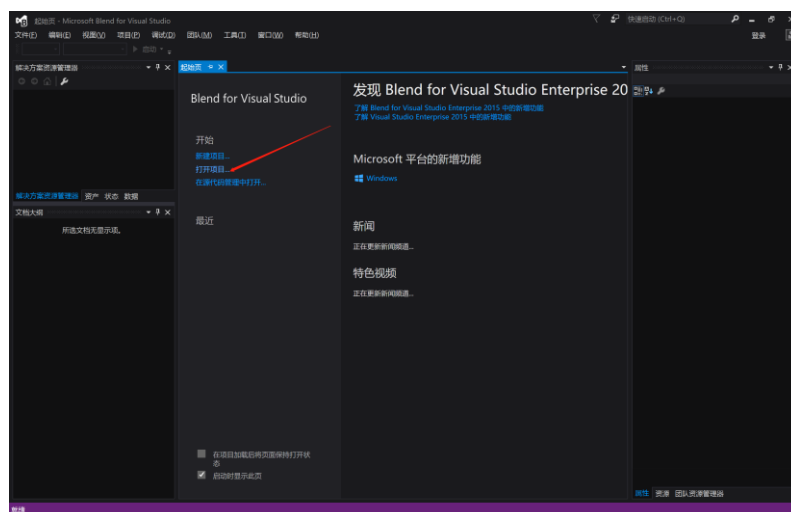


图 2-26 使用 VS2015 打开项目

解压提供的驱动和应用压缩包，打开提供的驱动和应用文件夹下的项目，如图 2-27 所示。

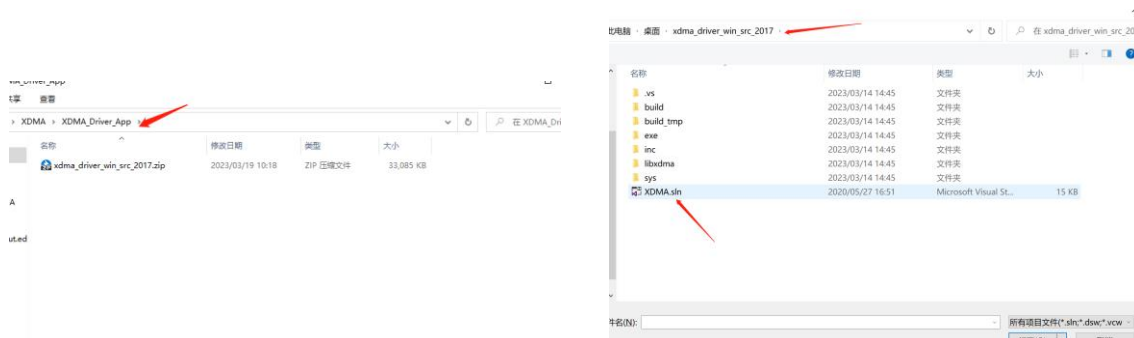


图 2-27 打开项目

清理解决方案，如图 2-28 所示。

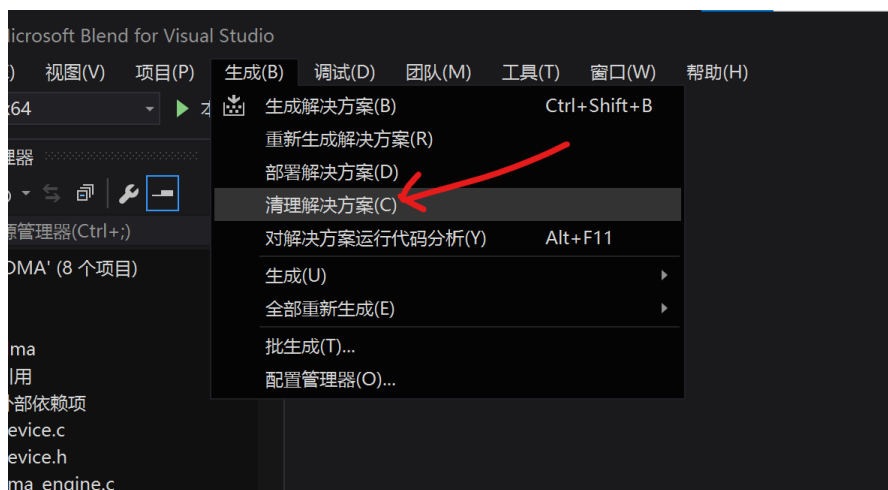


图 2-28 清理解决方案

重新生成解决方案，如图 2-29 所示，如果 1.2.2 节软件和环境安装正确的话，就会自动编译成功，并提示生成成功如图 2-30 所示。

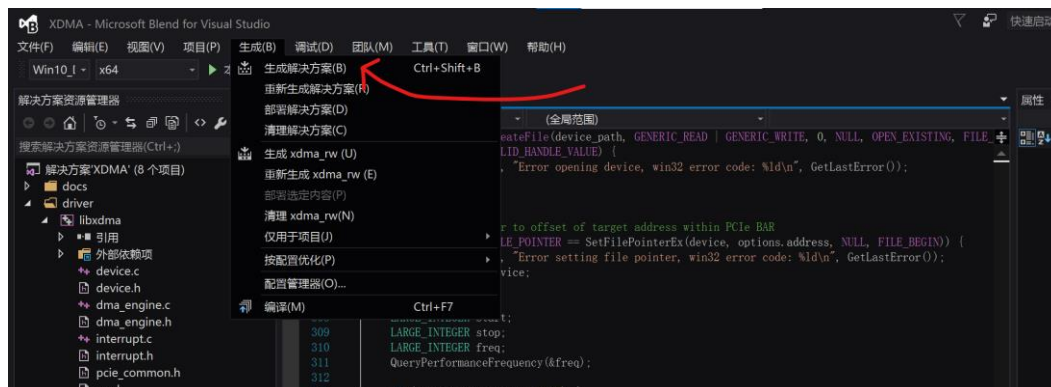


图 2-29 重新生成解决方案

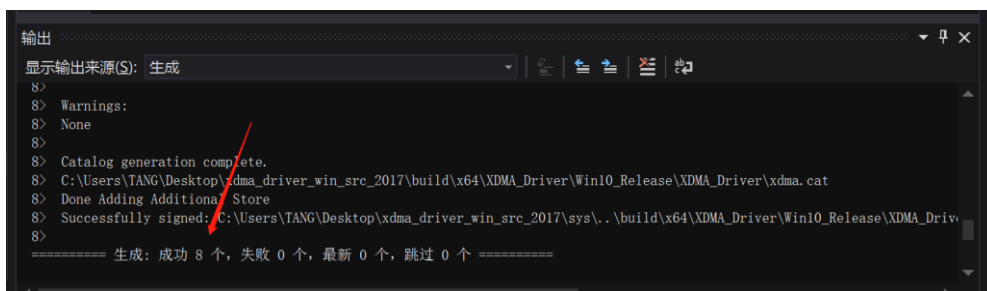


图 2-30 驱动和应用程序编译成功

至此，驱动和应用程序编译完成。

2.7 上机测试准备

接下来还有一项重要的设置，根据官方文档的说法，XDMA 的驱动没有提供一个验证过的证书，所以必须让系统进入测试模式才能安装驱动。

使用如下命令可以开关测试模式。

`bcdedit /set testsigning on` 打开测试模式

`bcdedit /set testsigning off` 关闭测试模式

在 WIN7/WIN10 系统下如图 2-31 打开终端并输入对应命令回车，一定要使用管理员权限

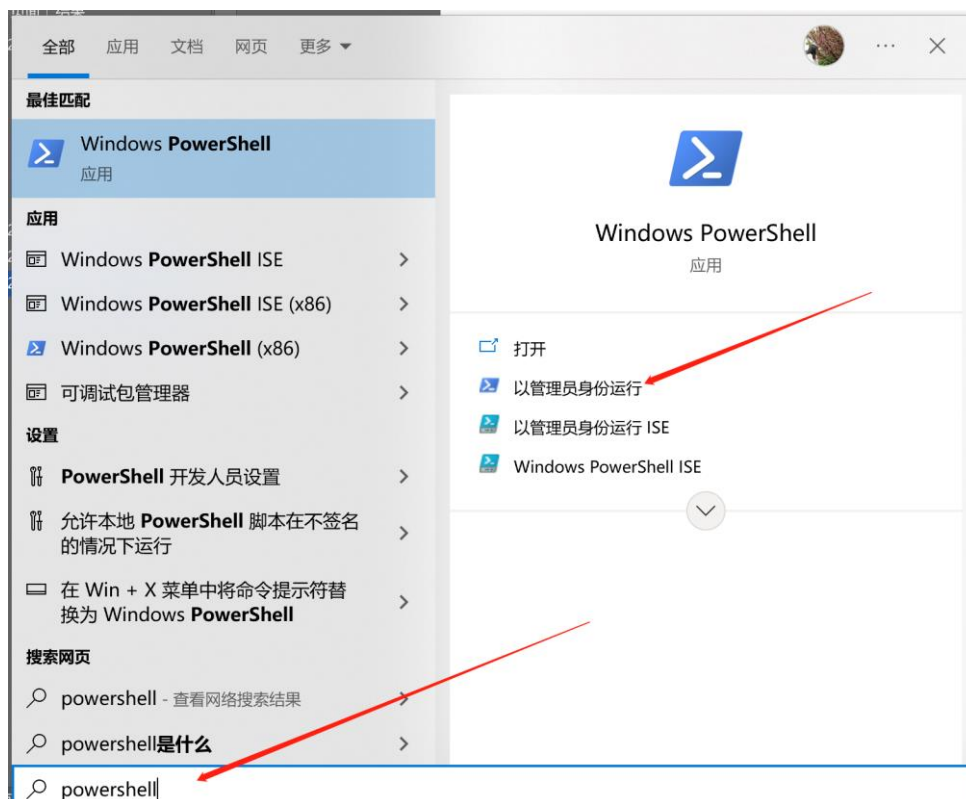


图 2-31 搜索 powershell 并以管理员身份打开

输入打开测试模式命令并回车，如图 2-32 所示。

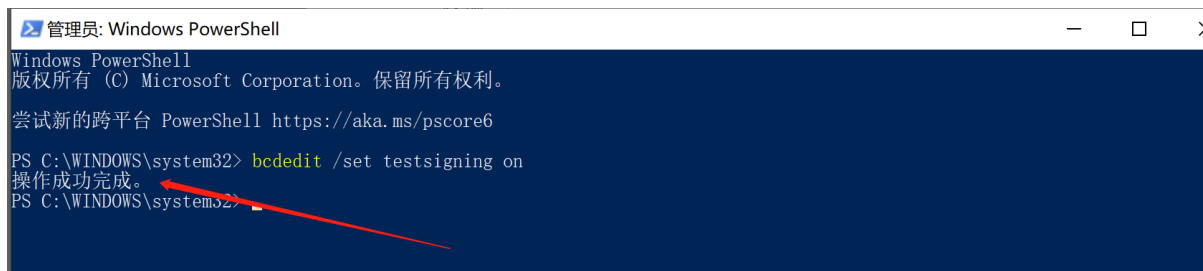


图 2-32 成功进入测试模式

设置完成后**电脑关机**，将烧录好 Flash 的 M2 加速卡安装至电脑的 M2 接口上，注意是要 M2 接口 NVME 协议的，安装完成后电脑开机，加速卡上电灯亮，如图 2-33 所示。

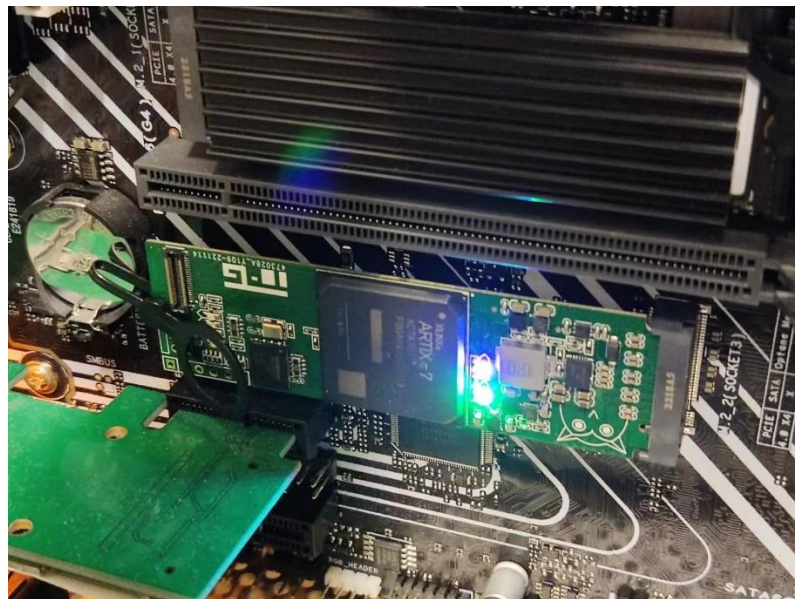


图 2-33 安装加速卡上电开机

打开电脑设备管理器，会在其他设备栏中看到“PCI 串行端口”，如图 2-34 所示。

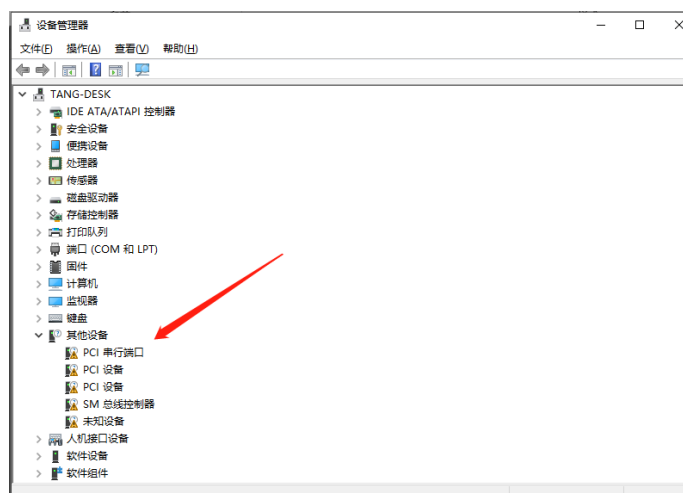


图 2-34 设备管理器识别到的未知设备

打开之前编译好的驱动文件夹 Win10_Release，如图 2-35 所示。

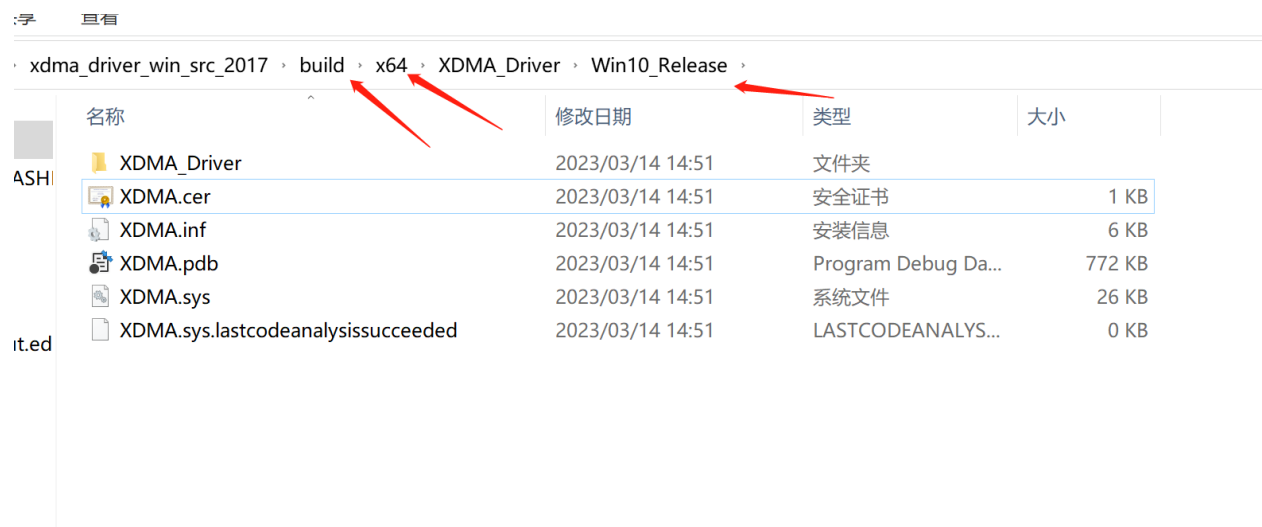


图 2-35 编译好的驱动文件夹

右键选择证书文件，点击“安装证书”，如图 2-36 所示。在弹出的向导中一直下一步完成安装即可。

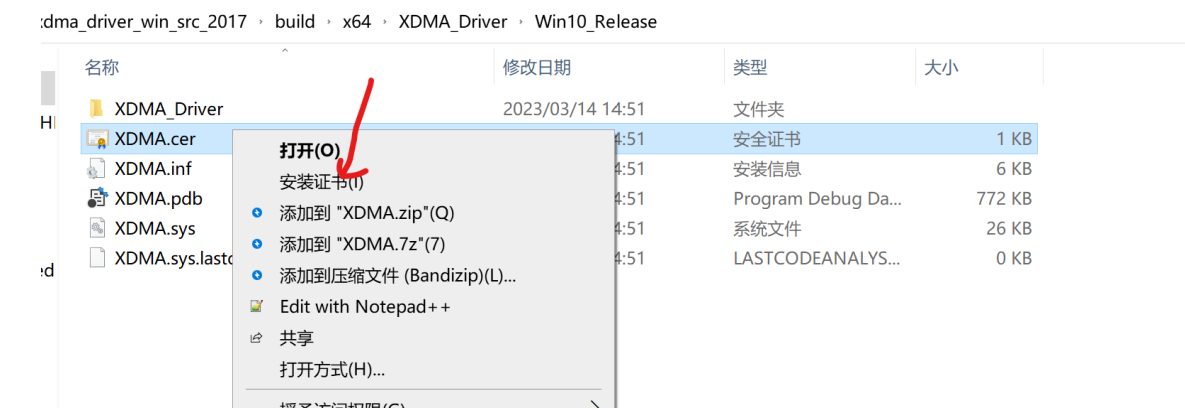


图 2-36 安装证书

在进入下一级的 XDMA_Driver 文件夹，右键点击 XDMA.inf 文件进行驱动安装，如图 2-37 所示。



图 2-37 驱动安装

安装驱动过程中会弹出 Windows 无法验证驱动发布者的警告，选择始终安装，最后完成驱动安装，如图 2-38 所示。



图 2-38 驱动安装完成

驱动安装完成后设备管理器中能够成功识别 XDMA 设备，如图 2-39 所示。

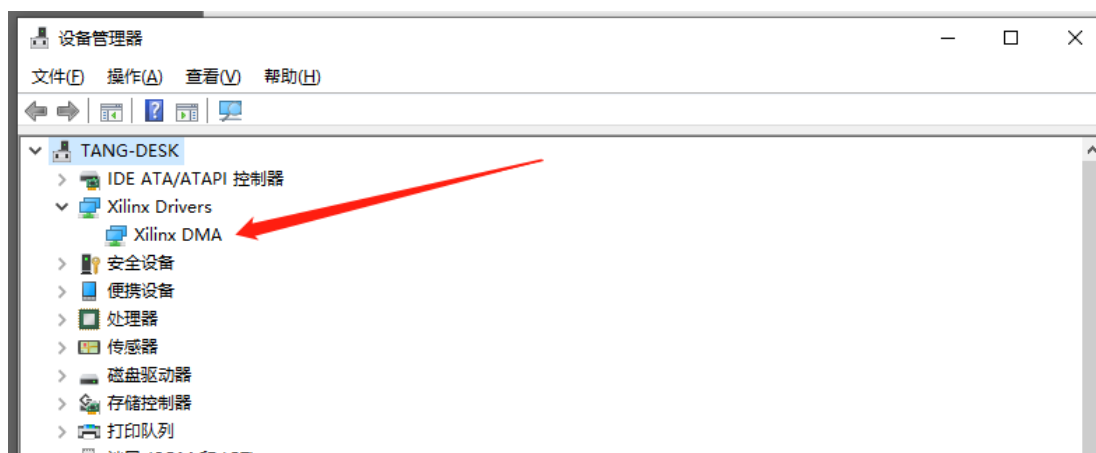


图 2-39 设备管理器识别到 XDMA 设备

2.8 xdma_rw BRAM 读写测试

我们打开一个终端（如果双击运行会很快退出来），cd 进入到编译生成的应用程序目录找到 xdma_rw.exe，这个应用程序是操作 pcie 的所有设备的，我们在终端只输入.\xdma_rw.exe，可以看到提示信息，告诉用户这个程序如何使用。如图 2-40 所示。

```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。
尝试新的跨平台 PowerShell https://aka.ms/pscore6

PS C:\Users\TANG> cd C:\Users\TANG\Desktop\xdma_driver_win_src_2017\build\x64\bin\
PS C:\Users\TANG\Desktop\xdma_driver_win_src_2017\build\x64\bin> .\xdma_rw.exe
C:\Users\TANG\Desktop\xdma_driver_win_src_2017\build\x64\bin\xdma_rw.exe usage:
C:\Users\TANG\Desktop\xdma_driver_win_src_2017\build\x64\bin\xdma_rw.exe <DEVNODE> <read|write> <ADDR> [OPTIONS] [DATA]
- DEVNODE : One of: control | user | event_* | hc2_* | c2h_*,
              where the * is a numeric wildcard (0-15 for events, 0-3 for hc2 and c2h).
- ADDR : The target offset address of the read/write operation.
           Can be in hex or decimal.
- OPTIONS :
            -a set alignment requirement for host-side buffer (default: PAGE_SIZE)
            -b open file as binary
            -f use contents of file as input or write output into file.
            -l length of data to read/write (default: 4 bytes or whole file if '-f' flag is used)
            -v more verbose output
- DATA : Space separated bytes (big endian) in decimal or hex,
           e.g.: 17 34 51 68
           or: 0x11 0x22 0x33 0x44
```

图 2-40 进入 xdma_rw.exe 文件夹并运行

可以看到在当前目录有一个 datafile4k.bin 文件，那就测试一下将这个文件传输到 FPGA 的 BRAM，然后读出来。

首先在终端输入指令：`.\xdma_rw.exe h2c_0 write 0x00000000 -b -f datafile4K.bin -l 4096` 如图 2-41 所示。

意思就是使用 h2c_0 设备以二进制的形式读取文件 datafile4k.bin 写入到 BRAM 内存地址 0x00000000 长度为 4096 字节。

```
PS C:\Users\TANG\Desktop\xdma_driver_win_src_2017\build\x64\bin> .\xdma_rw.exe h2c_0 write 0x00000000 -b -f datafile4K.bin -l 4096
4096 bytes read from file datafile4K.bin
4096 bytes written in 0.000038s
PS C:\Users\TANG\Desktop\xdma_driver_win_src_2017\build\x64\bin>
```

图 2-41 PCIE 写数据到 BRAM

接下来再读回来。

使用命令 `.\xdma_rw.exe c2h_0 read 0x00000000 -b -f datafile4K_rd.bin -l 4096`

如图 2-42 所示。

```
PS C:\Users\TANG\Desktop\xdma_driver_win_src_2017\build\x64\bin> .\xdma_rw.exe c2h_0 read 0x00000000 -b -f datafile4K_rd.bin -l 4096
4096 bytes received in 0.000031s
PS C:\Users\TANG\Desktop\xdma_driver_win_src_2017\build\x64\bin>
```

图 2-42 PCIE 从 BRAM 读回数据

使用 WinMerge 软件，打开 datafile4k.bin 和 datafile4K-recv.bin 进行比对，软件提示两个文件内容完全相等，说明 PCIE 读写成功。如图 2-43 和 2-44 所示。

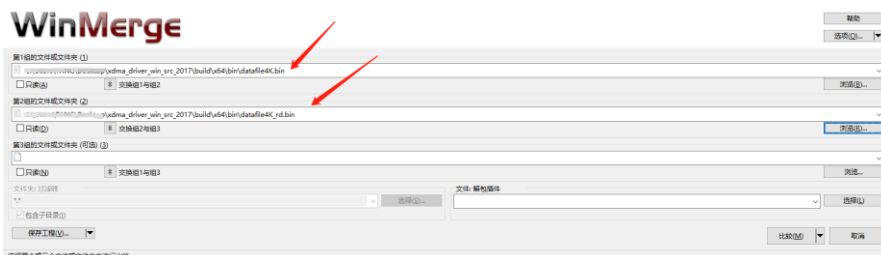


图 2-43 选择读写文件进行比对

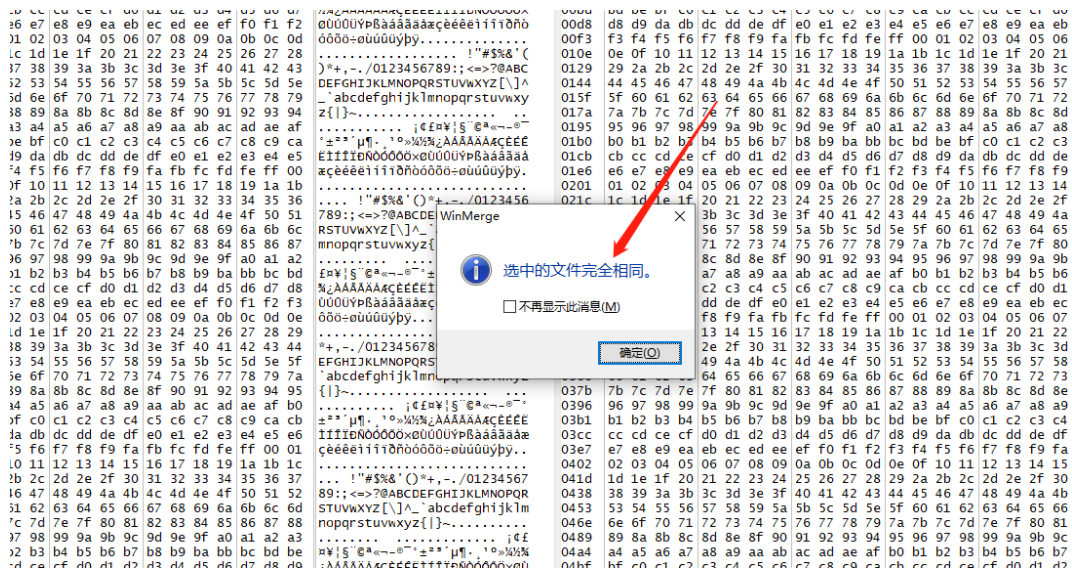


图 2-44 文件对比结果

2.9 simple_dma 简单测速

simple_dma 应用程序是简单的测速程序。使用 simple_dma 应用程序的方式很简单，直接在打开的 Powershell 窗口中输入如下命令即可。

```
.\simple_dma.exe
```

执行结果如图 2-45 所示。

```
PS C:\Users\TANG\Desktop\xdma_driver_win_src_2017\build\x64\bin> .\simple_dma.exe
Found 1 Xdma device.
Starting single XDMA Card-to-Host transfer of 8 MB.
XDMA Card-to-Host transfer has finished after 11618 us.
Starting single XDMA Host-to-Card transfer for 8 MB.
XDMA Host-to-Card transfer has finished after 9443 us.
PS C:\Users\TANG\Desktop\xdma_driver_win_src_2017\build\x64\bin>
```

图 2-45 simple_dma 测试

从执行结果中，可以计算出传输速率。

2.10 xdma_test 基础测试

xdma_test 应用程序具有以下功能：

- ✓检测 XDMA IP 中启用了多少个 h2c 和 c2h 通道
- ✓检测是否将 XDMA IP 配置为内存映射（AXI-MM）或流（AXI-ST）模式
- ✓在所有可用的 h2c 和 c2h 通道上执行数据传输
- ✓验证写入设备的数据是否与从设备读取的数据匹配
- ✓向用户报告通过或未通过的完成状态

使用 xdma_test 应用程序的方式很简单，直接在打开的 Powershell 窗口中输入如下命令即可：

.\xdma_test.exe

执行结果如图 2-46 所示。

```
PS C:\Users\TANG\Desktop\xdma_driver_win_src_2017\build\x64\bin> .\xdma_test.exe
Detected XDMA AXI-MM design.
Found h2c_0 and c2h_0:
  Initiating H2C_0 transfer of 4096 bytes...
  Initiating C2H_0 transfer of 4096 bytes...
  Transfers completed. Comparing data... OK!
Found h2c_1 and c2h_1:
  Initiating H2C_1 transfer of 4096 bytes...
  Initiating C2H_1 transfer of 4096 bytes...
  Transfers completed. Comparing data... OK!
Could not find h2c_2 and/or c2h_2
Could not find h2c_3 and/or c2h_3
Success!
PS C:\Users\TANG\Desktop\xdma_driver_win_src_2017\build\x64\bin>
```

图 2-46 xdma_test 测试结果

从执行结果可以看到，XDMA 使用的是 AXI-MM 接口模式，使用的是双通道的 h2c 和 c2h，在每个 h2c 和 c2h 通道上传输 4096 字节的数据并验证数据是匹配的。从 xdma_test 的检测结果和读写测试中可以知道我们设计的 XDMA Vivado 工程是没有问题的。

2.11 后记

由于 A7-35T 的资源限制，只能进行简单的 BRAM 读写测试，此工程 FPGA 资源占用已经达到 83%，如图 2-44 所示，这还是通过降低 PCIE 通道数改成*2 才塞下的，如果使用 *4 的话将会超过可用资源数量而无法通过编译。这时选用更大容量的 FPGA 芯片，例如 A7-50T-200T 型号的加速卡。更多的 XDMA 开发资料请参考附件中的米联客 XDMA 以及正点原子 XDMA 开发指南。

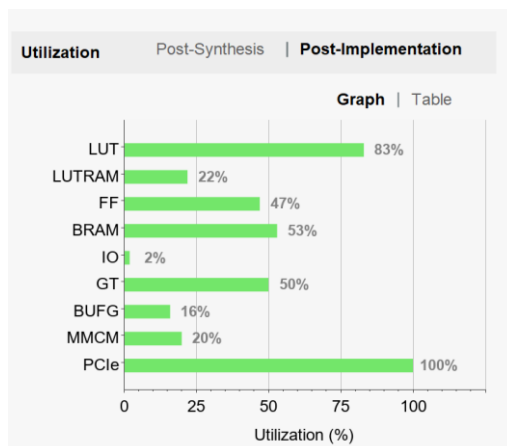


图 2-44 A7-35T XDMA 资源占用

本指导更推荐第三章的基于 Riffa PCIE DMA 读写方式，更灵活，资源占用少，环境搭建简单且开源。

第三章 Riffa 数据环回

3.1 Riffa 概述

RIFFA (Reusable Integration Framework for FPGA Accelerators) 即是 FPGA 加速器的一种可重用性集成框架, 是一个第三方开源 PCIE 框架. RIFFA 是一个通过 PCI Express 总线实现 cpu 和 FPGA 数据通信的简单框架, 该框架要求具备一个支持 PCIe 的工作站和一个带有 PCIe 连接器的 FPGA 板卡. RIFFA 支持 Windows, Linux, Altera 和 Xilinx, 可以通过 C/C++、Python、MATLAB 或 Java 驱动来实现数据的发送和接收. 驱动程序可以在 Linux 或 Windows 上运行, 每一个系统最多支持 5 个 FPGA 设备. 在用户端有独立的发送和接收端口, 用户只需编写几行简单代码即可实现与 FPGA IP 内核通信。

RIFFA 不依赖于 PCIe Bridge, 因此不受桥连实现的限制. RIFFA 使用直接存储器访问 (DMA) 传输和中断信号传送数据. 这实现了 PCIe 链路上的高带宽, 运行速率可以达到 PCIe 链路饱和点。

图 3-1 显示了使用 32 位, 64 位和 128 位接口的设计性能. 实线为理论最大带宽值, 虚线为可实现最大带宽值. PCIe Gen 1 和 2 使用 8 位/10 位编码, 将最大可实现带宽限制为理论值的 80%。我们的实验表明, 在几乎所有情况下, RIFFA 可以实现理论带宽的 80%。128 位接口达到理论最大值的 76%。

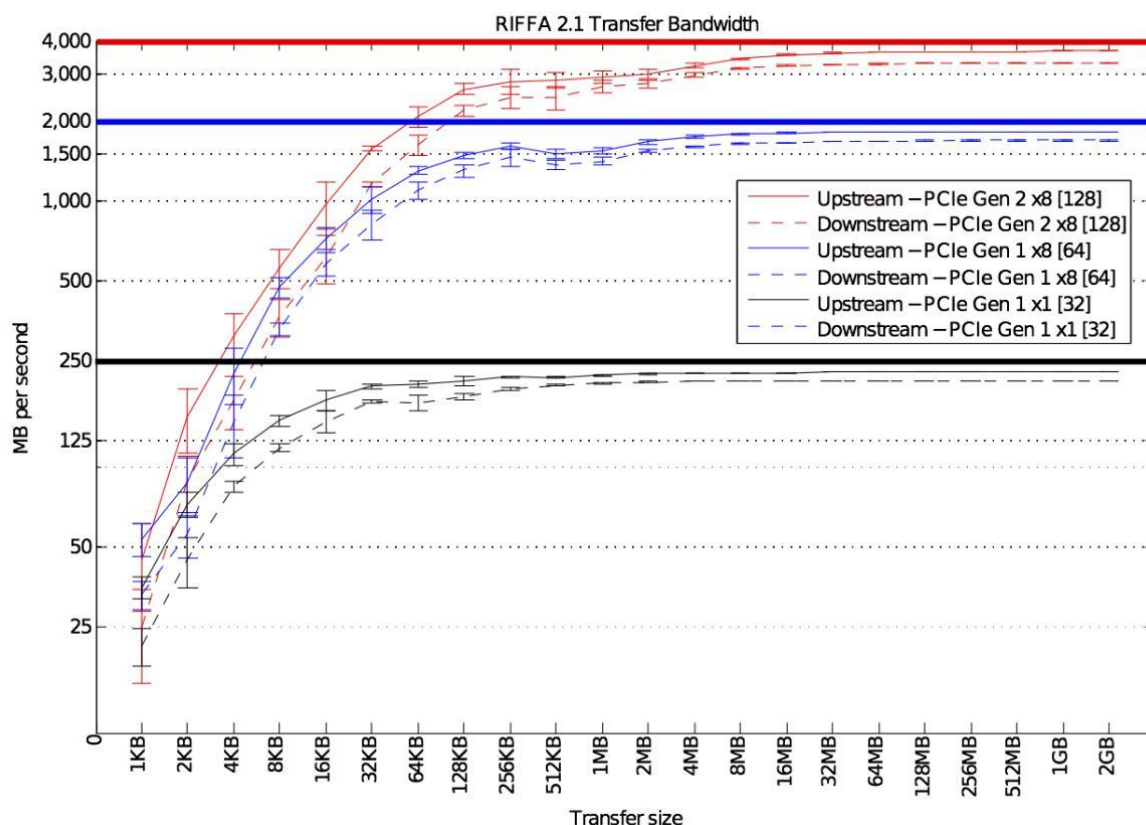


图 3-1 Riffa 理论和实际带宽

RIFFA 2.2.2 支持:

1.支持 VC707, ZC706 和用于 PCI Express Xilinx IP Core 7 系列集成块的板卡, 提供 VC707 和 ZC706 电板卡的示例设计。

2, 支持 VC709 板卡和似类有 Xilinx IP Gen3 集成块用于 PCI Express 的板卡。提供 VC709 的示例设计。

3, 支持具有 DES-Net 的 Stratix V, Cyclone V 和类似板卡, 用于 PCI Express 的硬核 (Avalon Streaming Interface)。提供了 DE5-Net 板卡的示例设计。

4. 支持 DE4 的 Stratix IV, Cyclone IV 和 Arria 11 器件。提供 DE4 板的设计示例。当前配置支持所有 64 位和 128 位 Avalon Streaming 接口。

3.2 RIFFA 体系结构

RIFFA 体系结构如图 3-2 所示。

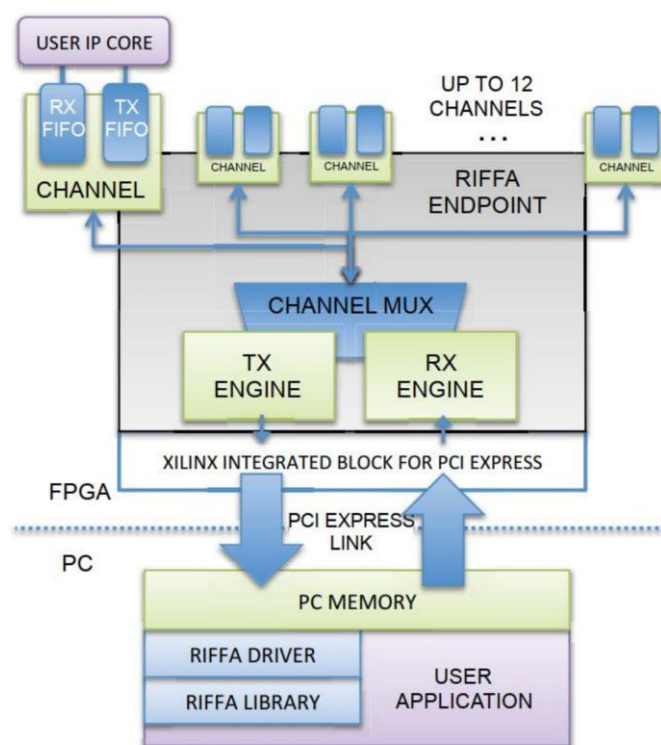


图 3-2 RIFFA 体系结构

在硬件方面, 简化了接口, 以便通过 FIFO 简便的将数据取出和存入。数据的传输由 RIFFA 的 RX 和 TX DMA Engine 模块用分散收集聚合方法来实现。RX Engine 模块收集上位机传来的有效数据, 收集完成发给 Channel 模块, TX Engine 收集 Channel 模块传来的数据, 打包发给 PCI Express 端点。根据 PCIe 链路配置, RIFFA 接口支持 32 位, 64 位和 128 位宽度, 计划为 PCIe Gen3 端点的 256 位接口提供支持。

PC 接收 FPGA 板卡数据是用户应用程序调用库函数 `fpga_recv`, 然后由 FPGA 端启

动。用户应用程序线程进入内核驱动程序，然后开始接收上游 FPGA 的读请求，将数据分包发送，如果没收到请求，将会等待它达到。

启动发送函数后，服务器将建立一个散列收集元素的列表，将数据存储地址和长度等信息放入其中，将其写入共享缓冲区。用户应用程序将缓冲区地址和数据长度等信息发送给 FPGA。FPGA 读取散列收集数据，然后发出相应地址的数据写入请求，如果散列收集元素列表的地址有多个，FPGA 将通过中断发出多次请求。

TX 搬移的数据全部写入缓存区后，驱动程序读取 FPGA 写入的字节数，确认是否与发送数据长度一致。这样就完成了传输。其过程如图 3-3 所示。

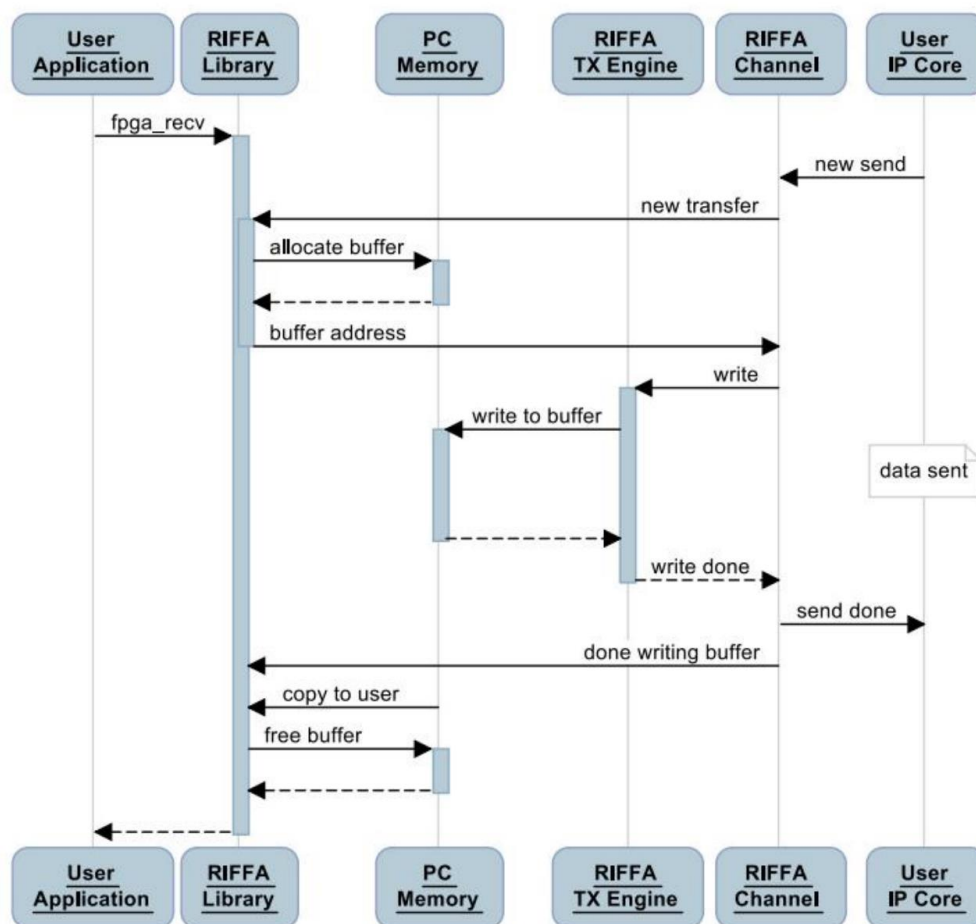


图 3-3 FPGA 传输到 PC 流程

PC 机发送数据到 FPGA 板卡过程与 PC 机接收 FPGA 板卡数据过程相似，如图 3-4 所示。刚开始也是用户应用程序调用库函数 `fpga_send`，传输线程进入内核驱动程序，然后 FPGA 启动传输。

启动 `fpga_send`，服务器将申请一些空间，将要发送的数据写入其中，然后建立一个分散收集列表，将存储数据的地址和长度放入其中，并将分散收集列表的地址和要发生的数据长度等信息发给 FPGA。FPGA 收到列表地址后，读取该列表的信息，然后发出相应地址和长度的读请求，然后将数据存储，最后一起发给 FPGA 板卡。

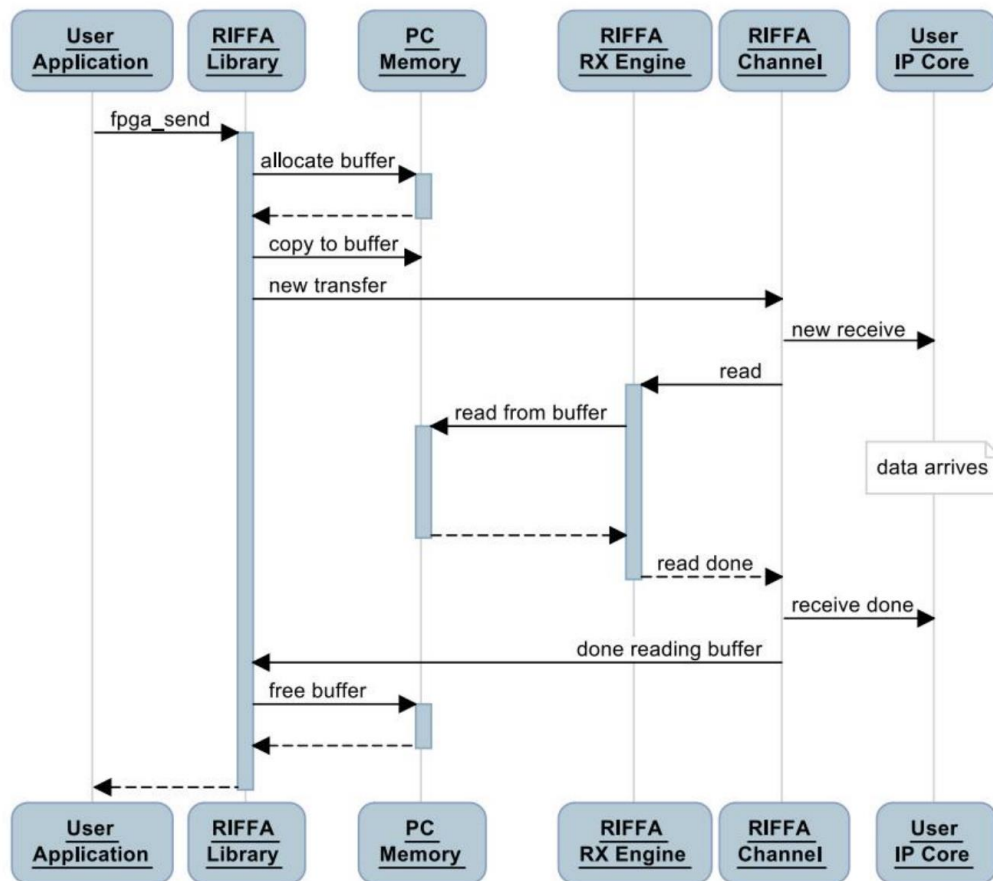


图 3-4 PC 传输数据到 FPGA

3.3 RIFFA 驱动安装

在提供的 `riffa_2.2.2/nstall/windows/install` 文件目录中运行 `setup.exe` 或 `setup_dbg.exe` 安装程序来安装内核驱动程序和 C/C++ 库。如图 3-5 所示。

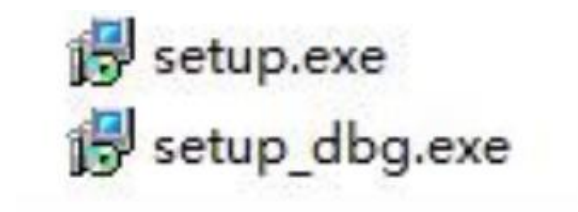


图 3-5 RIFFA Windows 驱动安装包

RIFFA 的驱动程序需要以 Windows 7 环境安装，并且需要以管理员程序进行安装，否则需要关闭驱动签名才能运行。如图 3-6 所示。

接下来按照向导一直点击下一步直到完成安装，如图 3-7 和图 3-8 所示。



图 3-6 启用兼容性和管理员身份运行

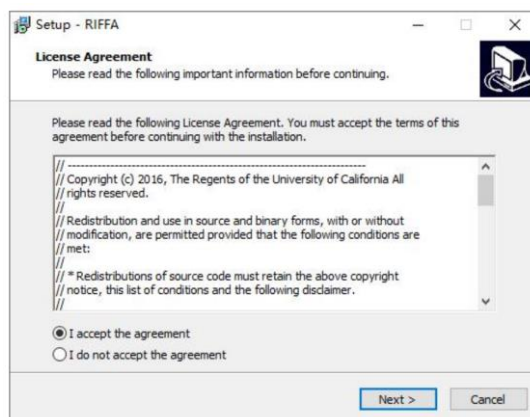


图 3-7 同意协议

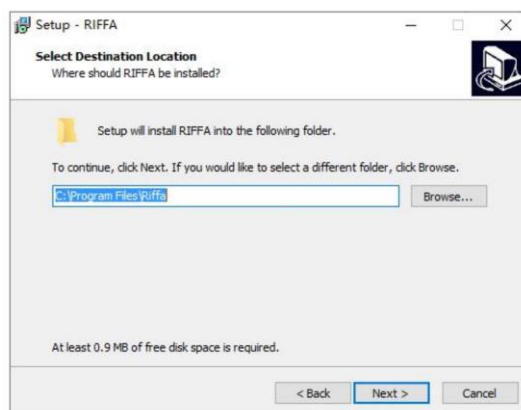


图 3-8 选择安装位置

驱动安装完成后，PC 端插上烧录了程序的 FPGA 加速卡后，即可在设备管理器中识别到 Riffa Device。

注意：对于比较新的 WIN10 或者 WIN11 的电脑，Riffa 驱动签名也已经不再长期保持有效，如果识别到 Riffa 设备但却提示该驱动未经签名。可以通过以下方式解决。

方法一：按照 2.7 节开启测试模式。

方法二：在电脑 BIOS 中关闭 Secure_Boot。

方法三：在 Windows 高级启动中选择禁用驱动程序强制签名。

3.4 RIFFA 库使用

RIFFA 库支持 java, python, C/C++, matlab。本指导以 C/C++ 举例，C/C++ 适合编写上位机，如图 3-9 所示。其他三种语言与上述两种类似，如需了解请自行研究。

RIFFA 库的使用主要由 4 个函数，与常规文件读写类似，有 open, write, read, close 这 4 个函数，读写就通过这 4 个函数进行操作。首选通过 open 打开 PCIE 设备，然后通过 write 或者 read 即可写入或者读取数据到 PC 机，如果不需要进行读写，则使用 close 关闭设备。下面使用 matlab 与 c 语言简单介绍一下 riffa 库的使用，这些库函数在驱动函数里面就有，如需了解，可自行查看源码。

C/C++ 库使用方法：

```
01 #include <stdio.h>;
02 #include <stdlib.h>;
03 #include <riffa.h>;
04 #define BUF_SIZE (1*1024*1024)
05 unsigned int buf[BUF_SIZE];
06 int main(int argc, char* argv[])
07 {
08     fpga_t * fpga;
09     int fid = 0; // FPGA id
10     int channel = 0; // FPGA channel
11     fpga = fpga_open(fid);
12     fpga_send(fpga, channel, (void *)buf, BUF_SIZE, 0, 1, 0);
13     fpga_recv(fpga, channel, (void *)buf, BUF_SIZE, 0);
14     fpga_close(fpga);
15     return 0;
16 }</riffa.h></stdlib.h></stdio.h>
17
```

图 3-9 RIFFA C/C++ 库使用实例

C 语言对应的驱动函数，首先使用 fpga_open 进行打开 pcie 设备，然后使用 fpga_send 进行发送数据，使用 fpgarecv 进行接收数据，如果后续不需要继续采集，使用 fpga_close 进行关闭 pcie 设备。

最后调试 pcie 请注意上述规范，否则轻则导致蓝屏重启，重则导致系统损坏。

3.5 RIFFA Vivado 工程搭建

本指导基于 Github 上的 Riifa 参考例程改进而来，官方 Github:

网址 <https://github.com/KastnerRG/riffa>

原始例程可参考 riffa2.2.2 文件夹下的 example 例程。

下面介绍如何搭建一个 Riffa 数据环回测试例程。

打开 Vivado 软件，选择 Create Project 并点击 Next。如图 3-10 所示。

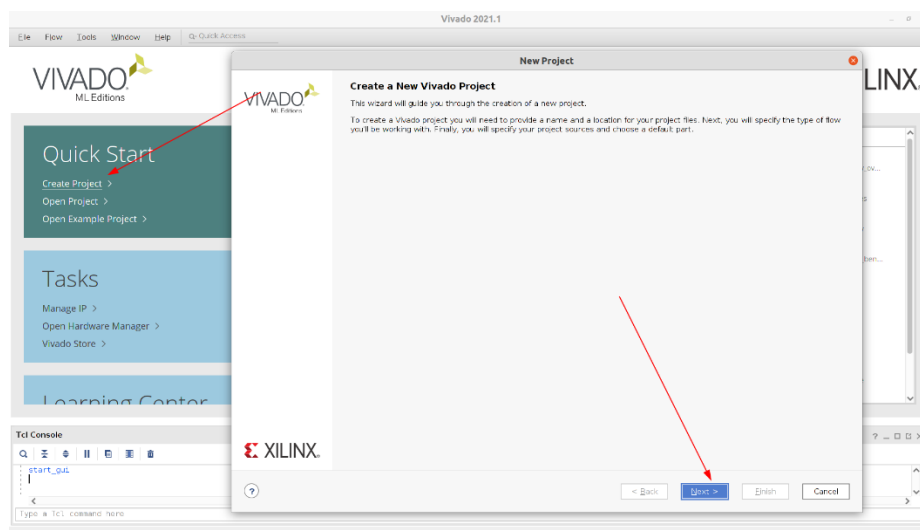


图 3-10 创建 Vivado 工程

输入工程名称和工程存放路径,以 m2_artix7_riffa 为例,如图 3-11 所示.点击下一步。

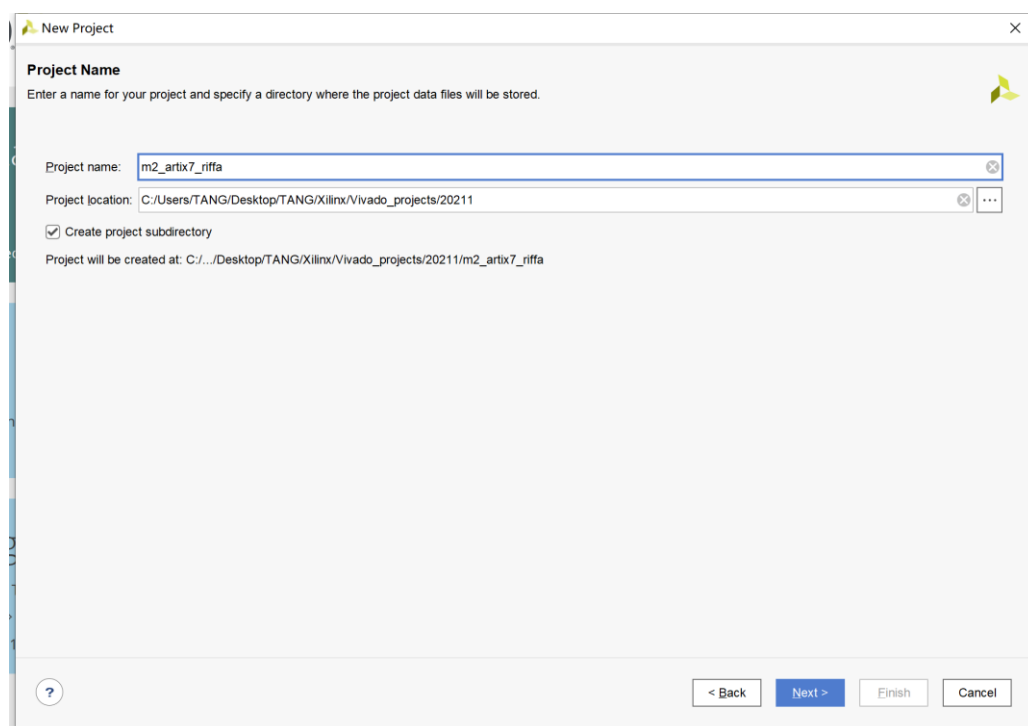


图 3-11 输入工程名称和路径

选择 RTL Project 并且勾选”当前不指定源文件”,如图 3-12 所示.

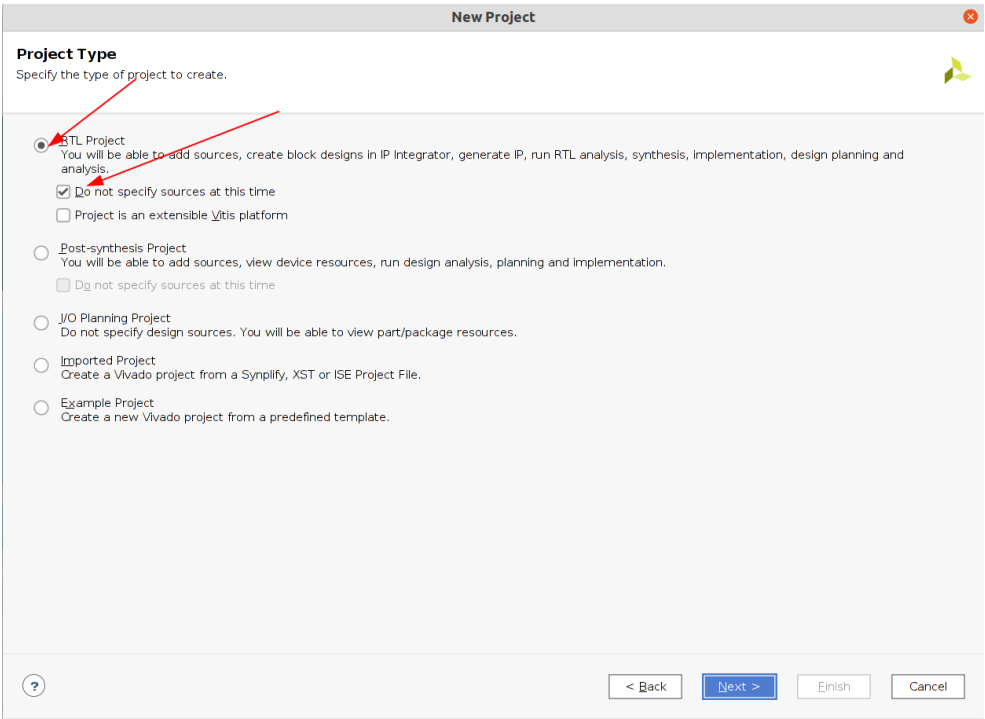


图 3-12 选择 RTL Project

根据芯片型号选择芯片后点击 Next,以 XC7A35T-2FGG484I 为例,如图 3-13 所示.

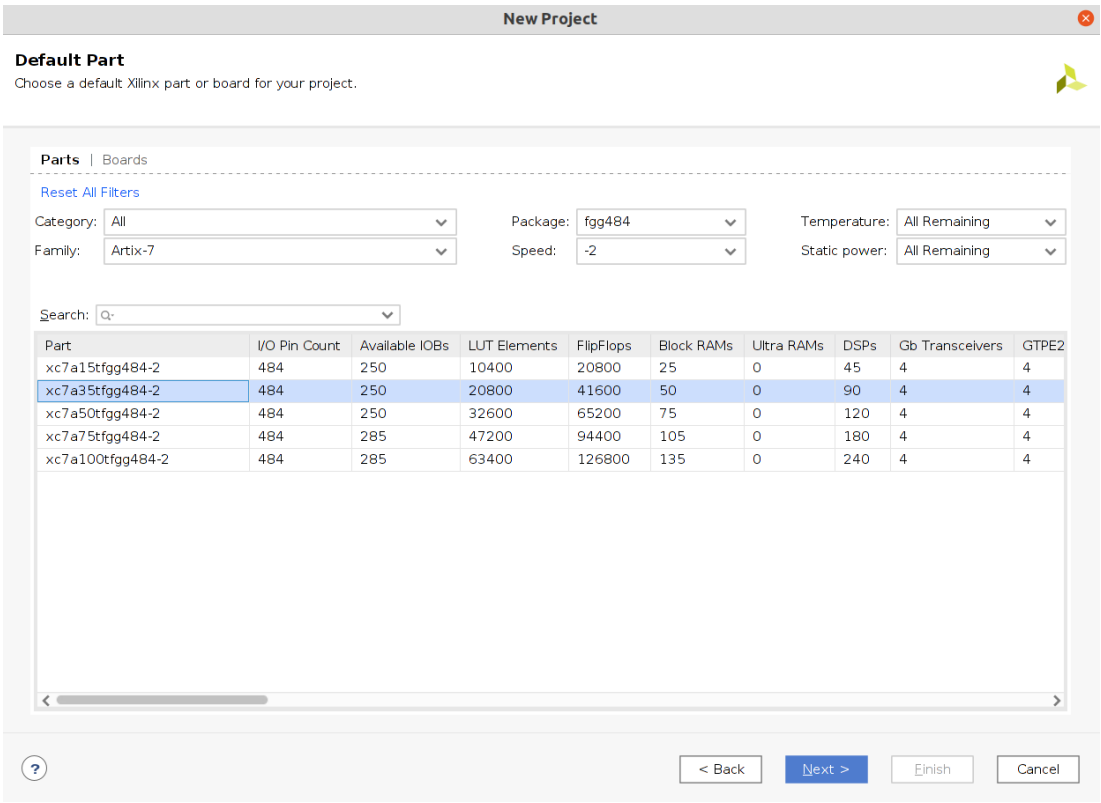


图 3-13 选择芯片型号

在新窗口中点击 Finish 完成工程创建,进入工程主界面,选择 IP catalog 如图 3-14 所示。

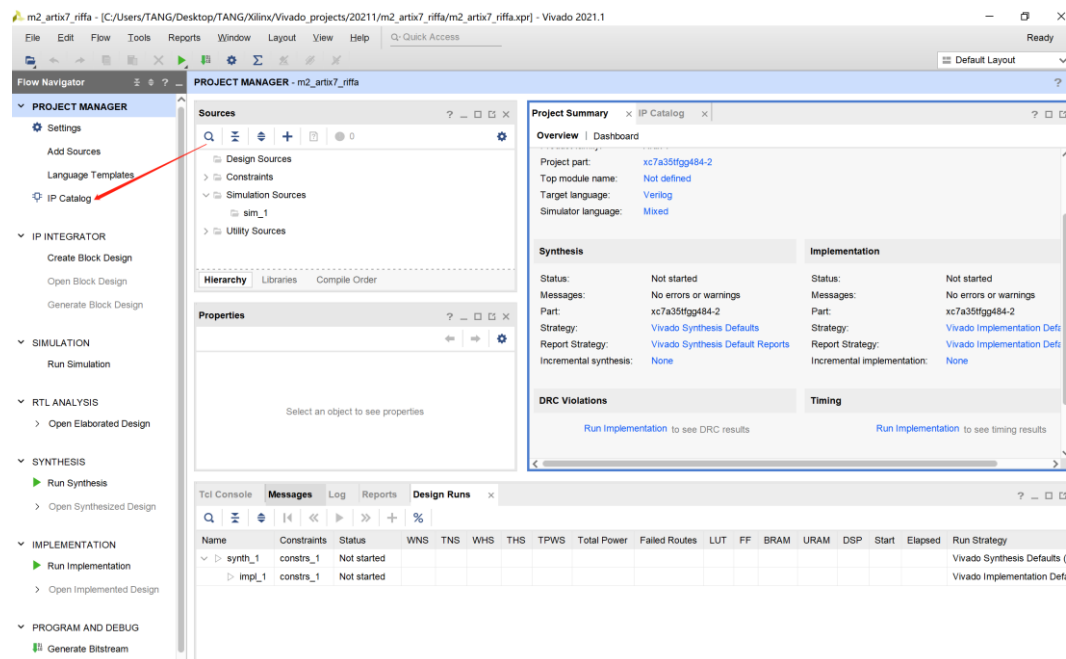


图 3-14 点击 IP Catlog

在 IP Catalog 中的 Search 框中输入 pcie,找到 7 Series Integrated Block for PCI Express 并双击进入配置界面.如图 3-15 所示。

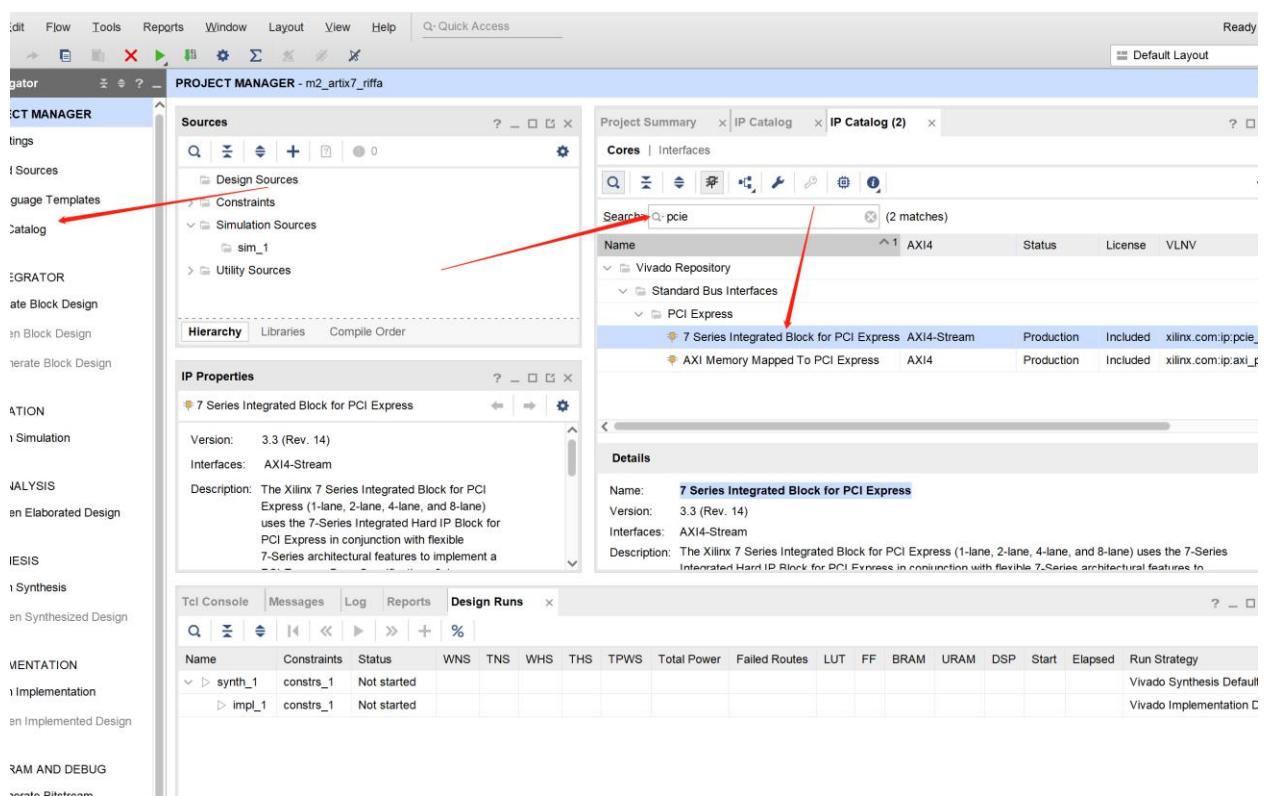


图 3-15 找到并添加 PCIE IP

3.6 PCIE IP 配置

本节详细介绍 PCIE IP 的配置内容和配置项对应的含义和作用。

7 Series Integrated Block for PCI Express 是 XILINX 在 7 系列 FPGA 上一种可扩展、高带宽和可靠的串行互连构建块，用于构建 PCIE 应用，包含了完整的事务层，数据链路层与物理层。7 系列 PCI Express 集成块（PCle）解决方案支持最高 5gb/s（Gen2）速度下的 1 通道、2 通道、4 通道和 8 通道端点和根端口配置，接口使用 AMBA 的 axi4 stream 接口。

3.6.1 Basic

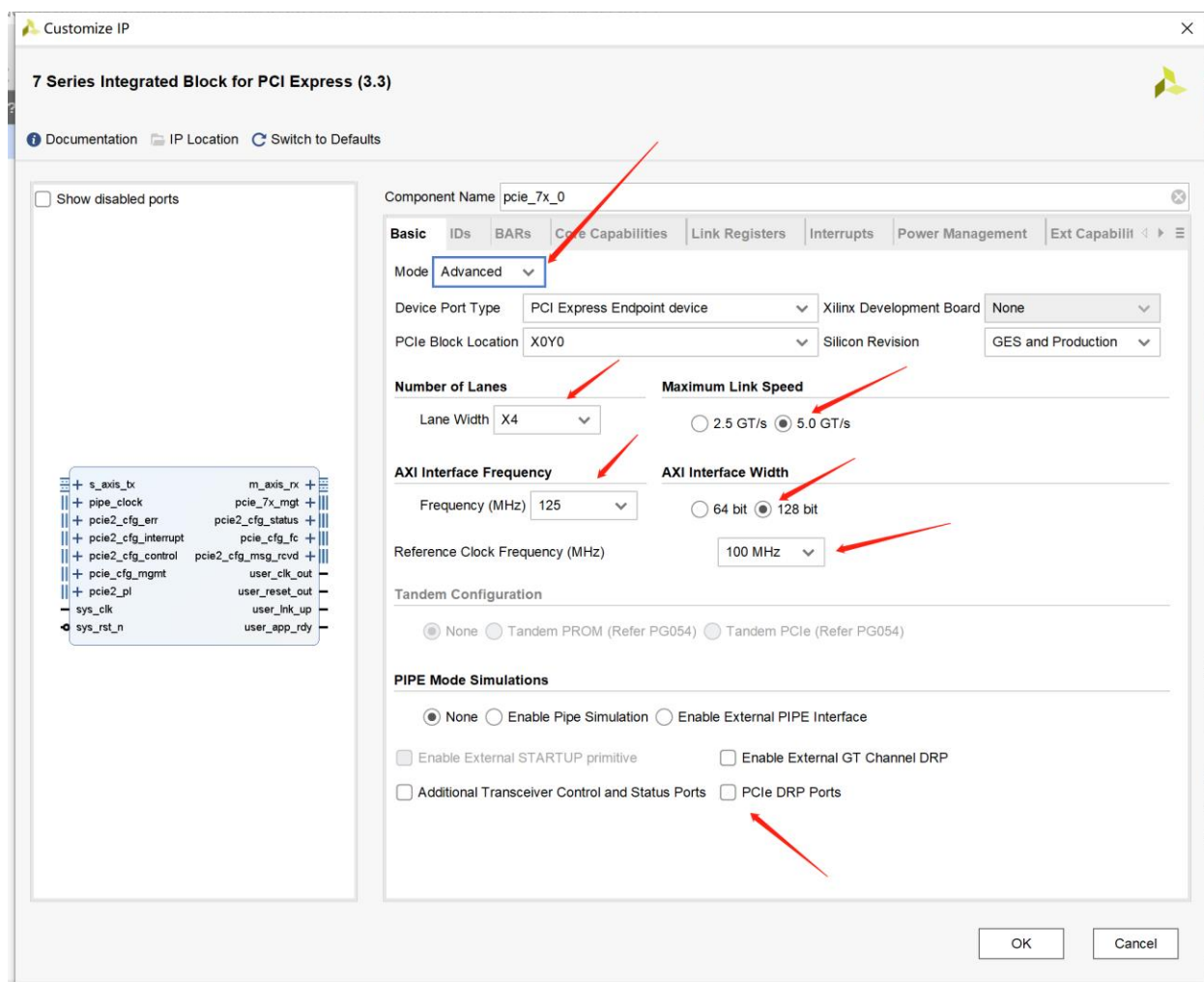


图 3-16 Basic 配置项

如图 3-16 所示为 Basic 配置项，包含以下主要内容

1. Mode: 可选择为 Advance 或者 base，高级或者基础模式，这里选择为 Advance。
2. Device Port Type: 可选择为 Endpoint device, LegacyEndpoint device, Root Complex, 也就是端点设备或者根复合节点。
3. Xilinx Development Board: 如果是官方的开发板可以选择这个进行一键配置。

4. PCIe Block Location: 选择 PCIE 的位置, 对于 artix 的芯片只能有一个选择。
5. Number of Lanes: Lane Width: 选择 PCIE 的通道数量, 选择 X4。
6. Maximun Link Speed: 最大连接速率, PCIE1.0 是 2.5GT/s, PCIE2.0 是 5GT/s, 选择 5GT/s。
7. AXI Interface Frequency: 用户端接口的 AXI Stream 接口的时钟, 这里选择 125MHz。
8. AXI Interface Width: 用户端接口的 AXI Stream 接口的数据位宽, 这里选择 128bit。
9. Reference Clock Frequency: 外部 PCIE 的参考时钟, 对于端点设备, 有电脑主板提供供给 FPGA 板卡, 这里选择 100MHz。
10. PCIE DRP Ports: 控制和检测 PCIE 物理层, 可以改变速度、位宽等。这里不使用, 取消勾选。

3.6.2 IDs

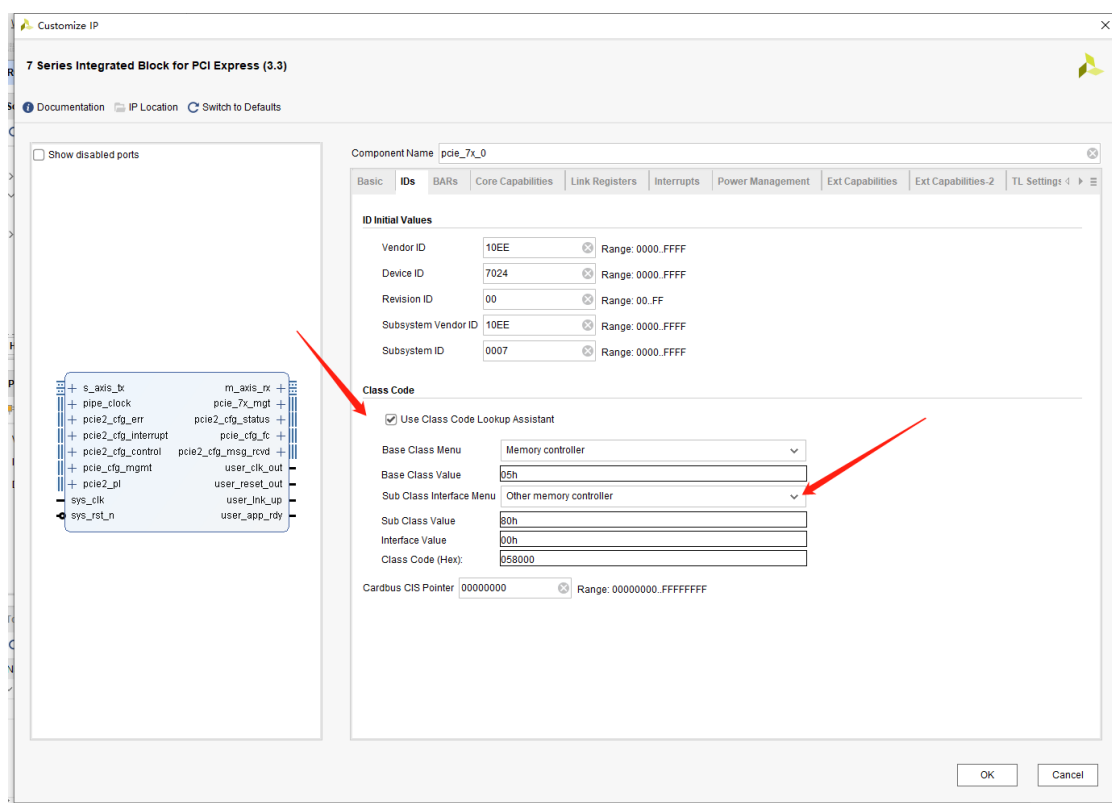


图 3-17 IDs 配置

IDs 页主要是关于用于初始化 Bar 空间的一些数据, 包括设备 ID, 厂商 ID, 类代码等, 与驱动程序有关, 一般不修改, 除非自己定制驱动, 否则修改后驱动程序将不识别 PCIE 设备。勾选 Use Class Code Lookup Assistant, Sub Class Interface Menu 选择 Other memory controller, 如图 3-17 所示。

1. Vendor ID: 厂商 ID。
2. Device ID: 设备 ID。
3. Revision ID: 修订版本 ID。

4. Subsystem Vendor ID: 与厂商 ID 类似。
5. Subsystem ID: 与设备 ID 类似。
6. Class Code: PCIE 类代码, 与驱动程序有关。

3.6.3 BARs

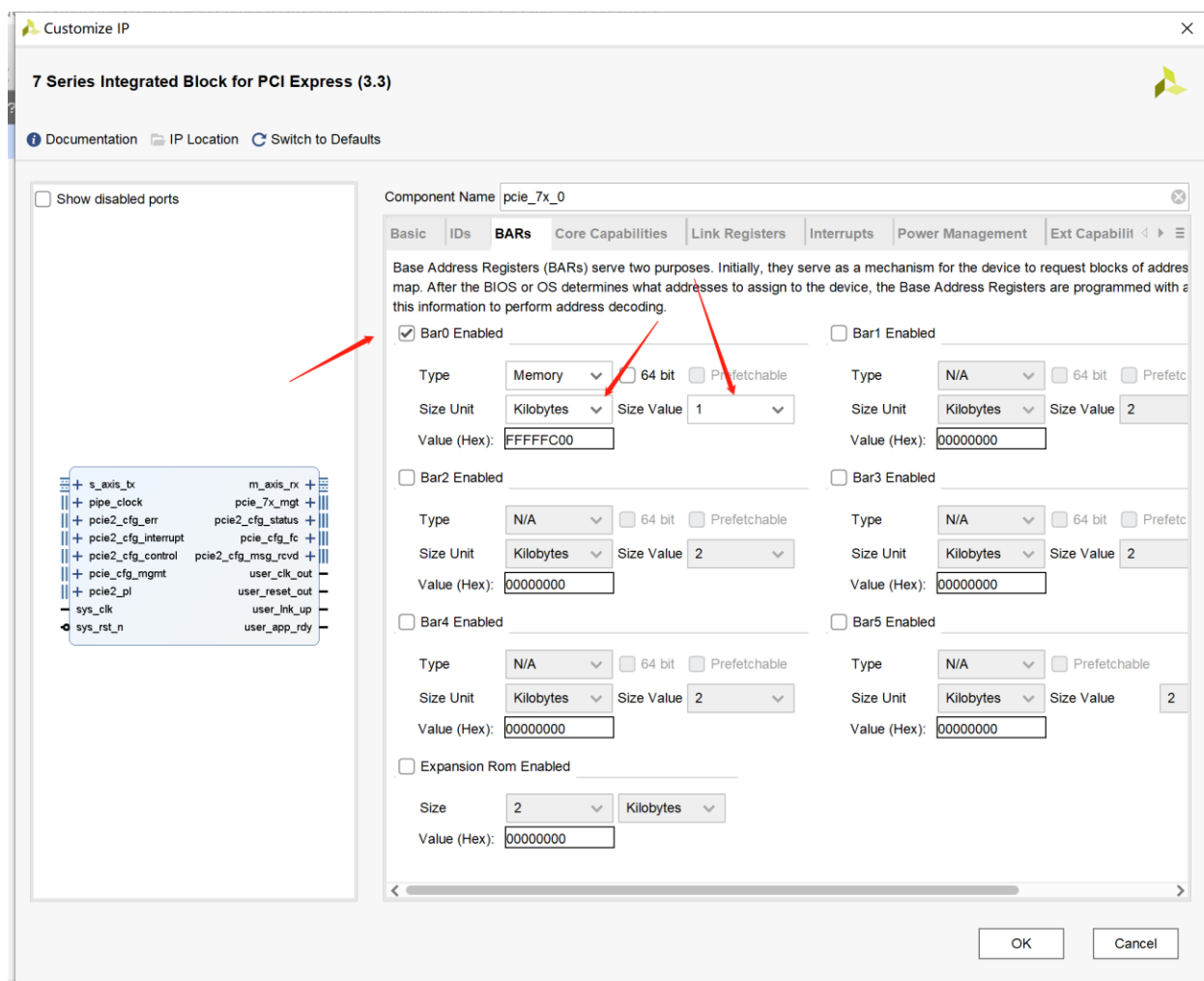


图 3-18 BARs 配置

BAR 页是用于配置 Bar 空间的, 主要起地址映射作用, 将电脑地址空间与 PCIE 地址空间完成映射。此页可开启多个 BAR 空间, 这里我们设置只使用 BAR0, 大小选择为 1Kbyte。

3.6.4 Core Capabilities

Core Capability 页是关于 PCIE 性能的配置, 其中包括事务层的有效负载数据最大值, 以及数据缓冲等等。配置如图 3-19 所示。

1. Max Payload Size: 有效负载数据最大值, 可设置为 128,256,512,1024 字节, 此值大可提高 PCIE 效率, 不过也更加耗费 FPGA 的资源, 主要是 BRAM 等资源, 这里我们设置为 256 字节大小。

2. BRAM Configuration Options: 配置 PCIE 的缓冲区大小, 可选为 Good 与 High, Good 耗费 BRAM 资源少, High 耗费 BRAM 资源。这里我们选择 High, 同时可勾选缓冲优化选项。

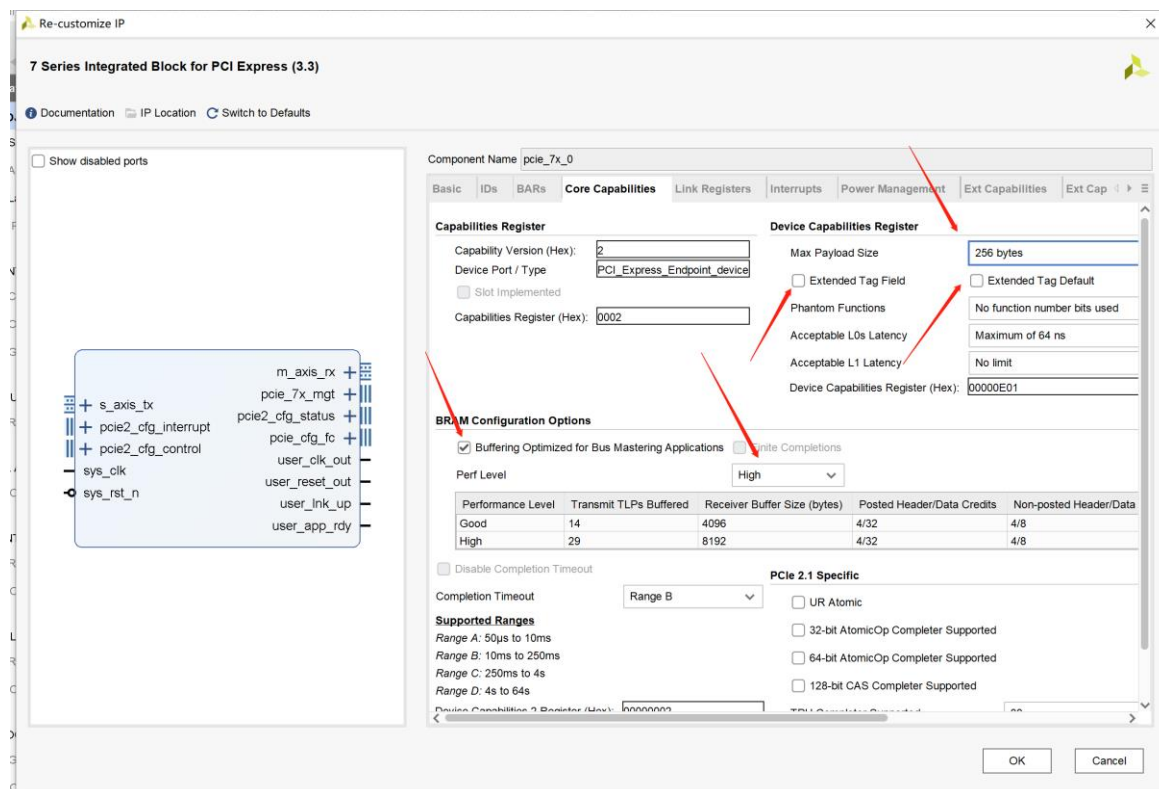


图 3-19 Core Capabilities 设置

其他值保持默认即可。

3.6.5 Link Registers

此页我们不做任何配置, 保持默认即可, 如图 3-20 所示。

Link Register 是连接寄存器, 用于告知电脑 FPGA 板卡速度与通道, 电脑识别为 PCIE1.0 还是 2.0, 3.0, 或者 X1, X2, X4 等都是电脑访问这个地址得到的, 在 PCIE 枚举过程会访问此地址。

1. Support Link Speed: 链路速度, 如果是 PCIE1.0, 此值为 1, PCIE2.0 为 2, PCIE3.0 为 3。
2. Maximum Link Width: 链路通道, X1, X2, X4, X8 分别是 1, 2, 4, 8 等。
3. Link Control Register: 连接地址控制寄存器。
4. Hardware Autonomous Speed Disable: 是否禁止链路自匹配功能。

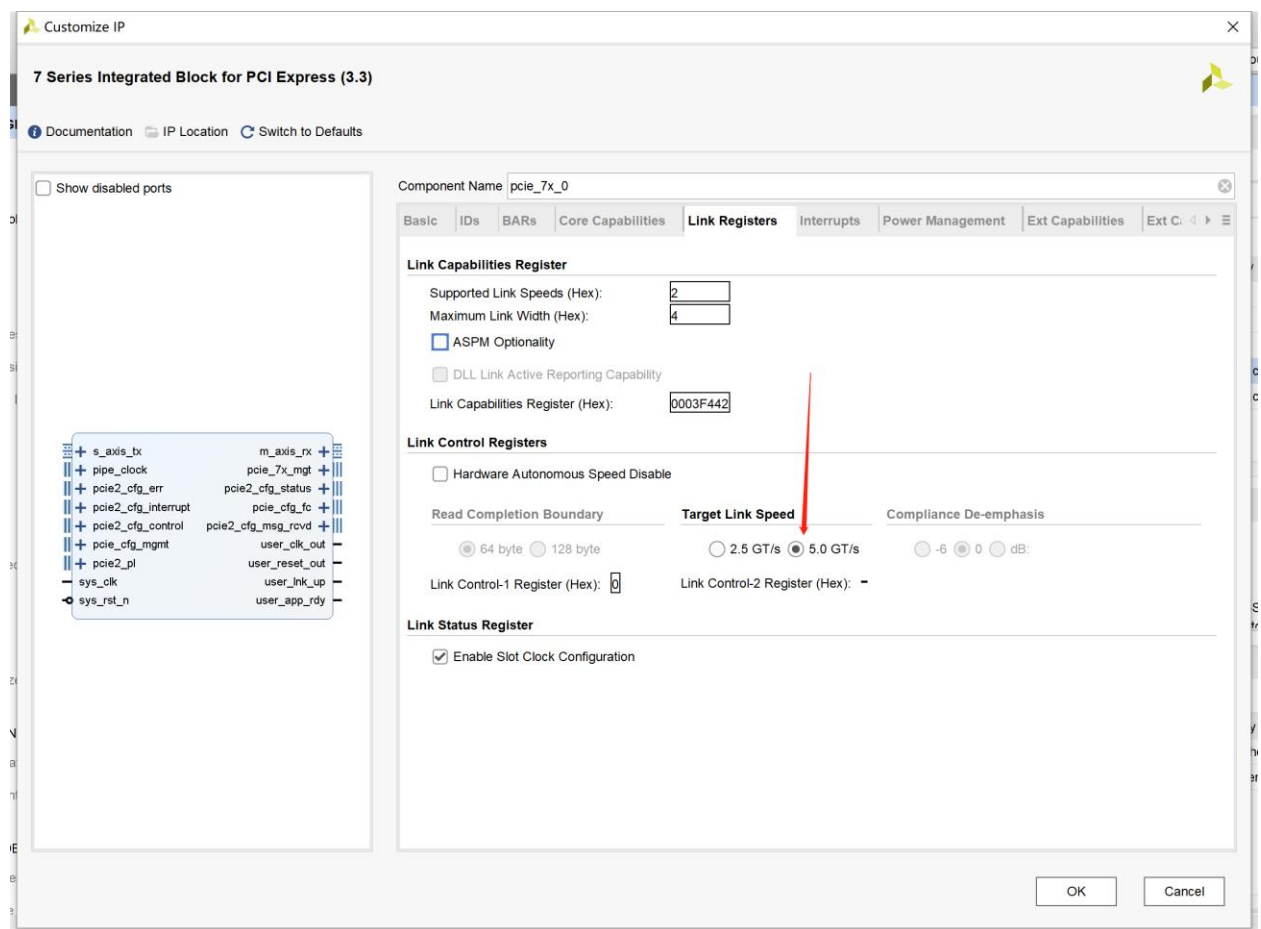


图 3-20 Link Registers 配置

3.6.6 Interrupts

Interrupts 是有关 PCIE 中断的，配置如图 3-21 所示。

1. Enable IntX: 这是引脚中断，现在不常用，在 FPGA 板卡也没有引出此功能，所以我们选择不勾选以关闭。

2. MSI Capability: 消息中断，通过数据包发送中断的一种机制。

3. MSIx Capability: 消息中断的另外一种，不可与 MSI Capability 同时开启，两种只能开启一个。

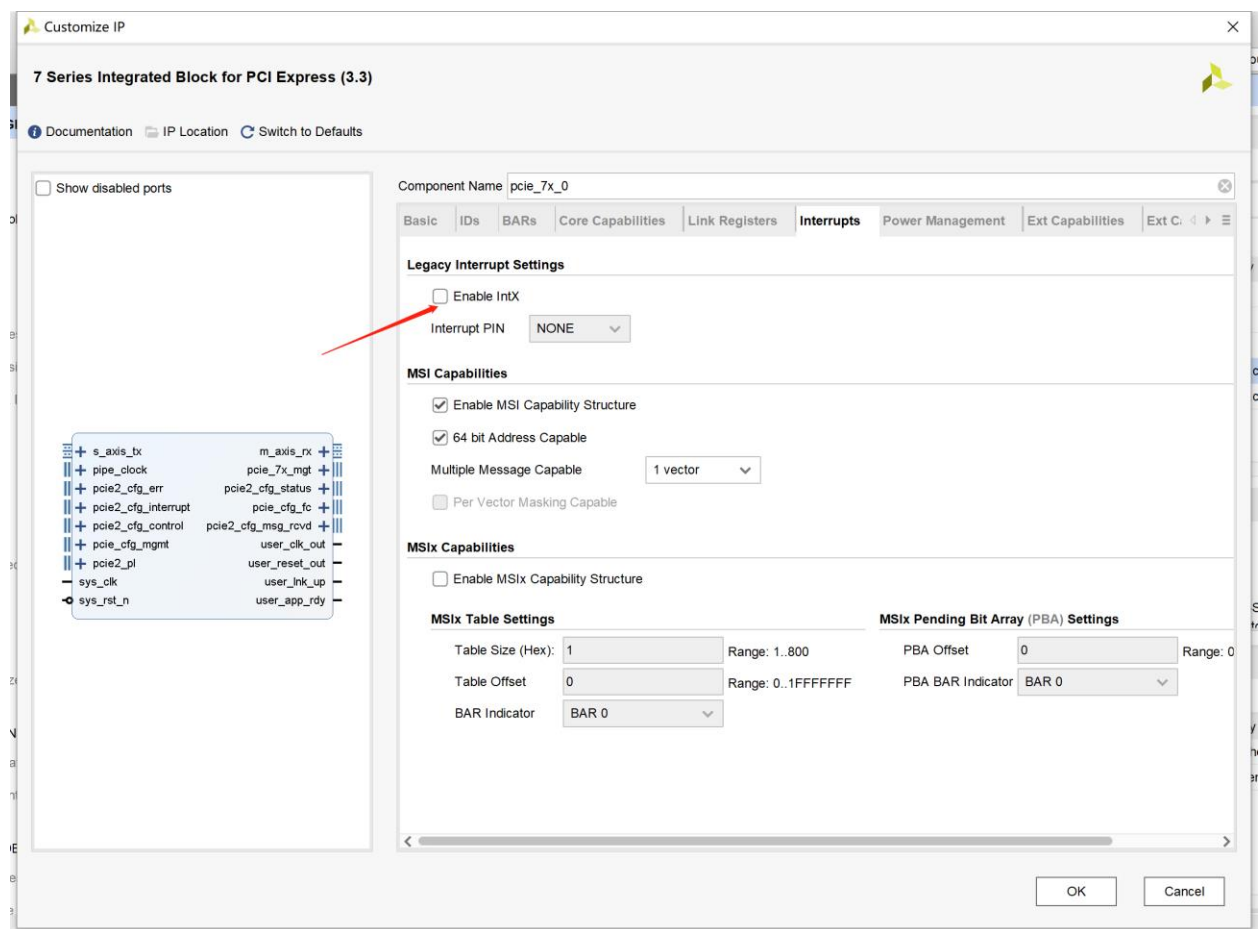


图 3-21 Interrupts 配置

3.6.7 Power Management

Power Management Register 页是关于电源管理寄存器的，其中包括热插拔，软复位等。这里我们暂时不做修改。如图 3-22 所示。

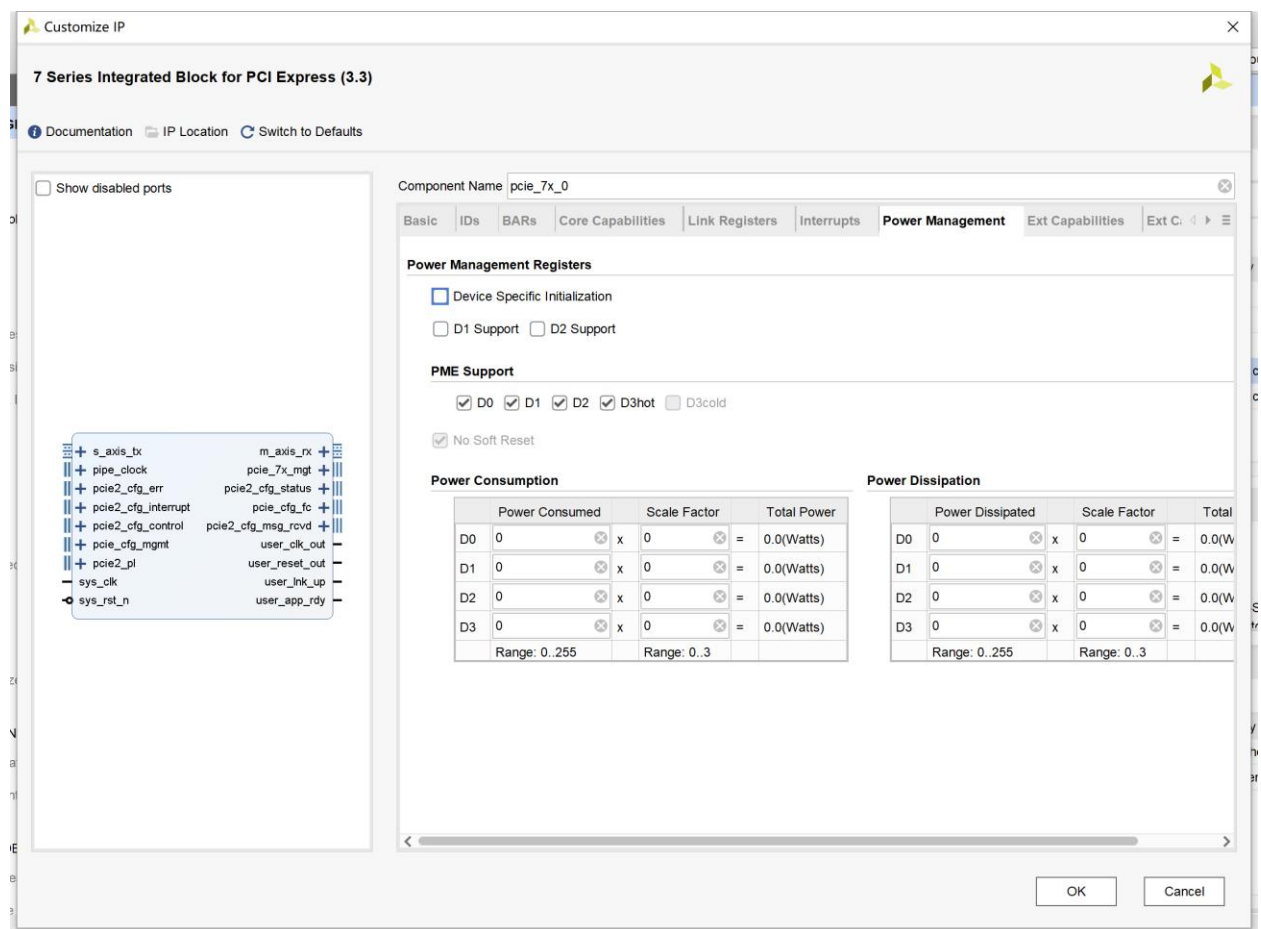


图 3-22 Power Management 配置

3.6.8 Ext Capabilities

此页具体内容不做介绍，保持默认即可，如图 3-23 所示。

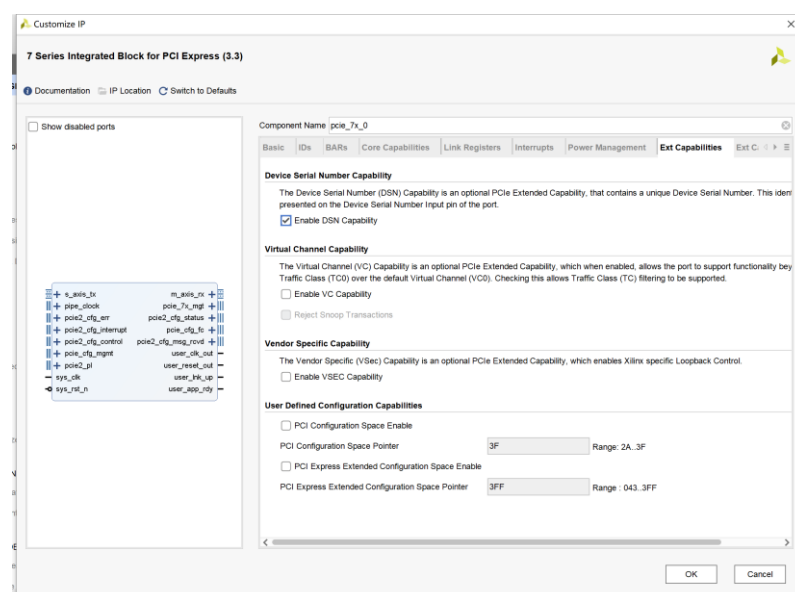


图 3-24 Ext Capabilities 配置保持默认

3.6.9 Ext Capabilities-2

此页具体内容不做介绍，保持默认即可，如图 3-24 所示。

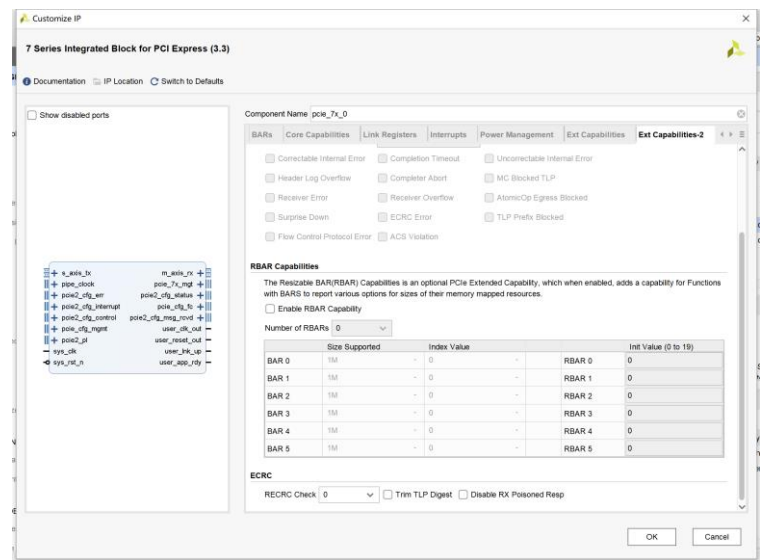


图 3-24 Ext Capabilities-2 配置保持默认

3.6.10 TL Settings

TL Settings 也是事务层的一些高级设定，保持默认不修改，如图 3-25 所示。

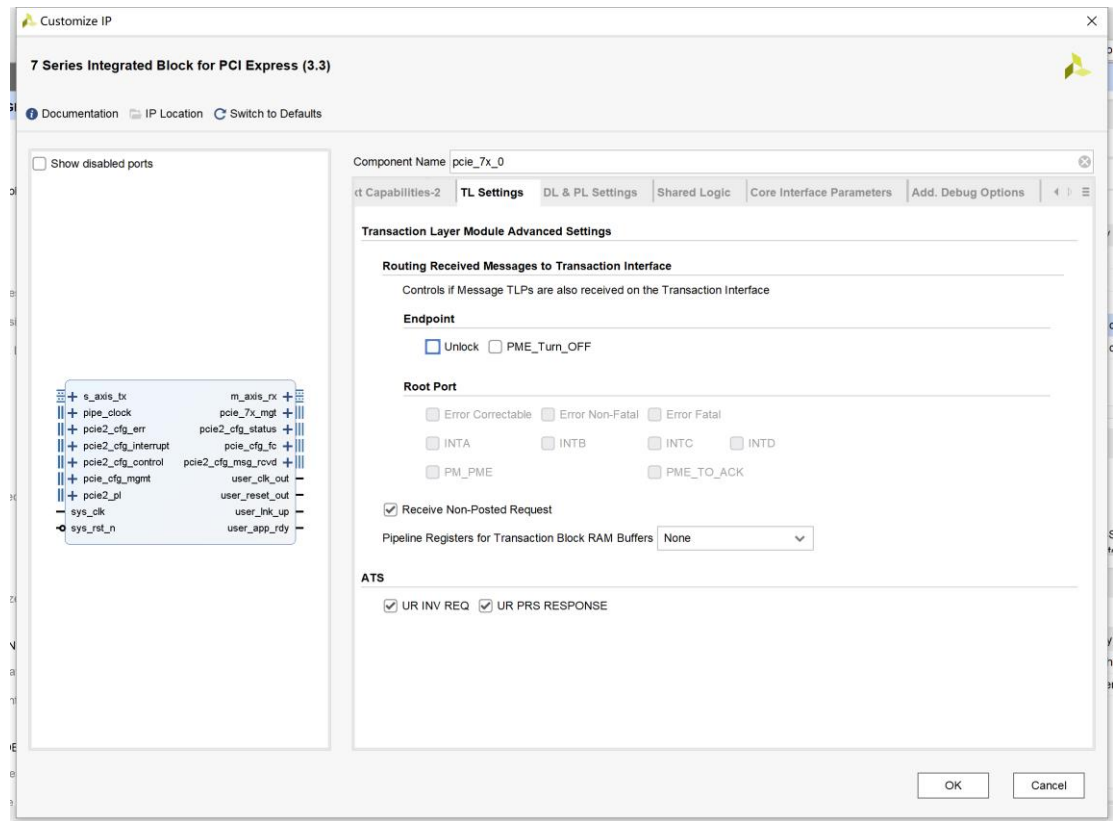


图 3-25 TL Settings 保持默认

3.6.11 DL&PL Settings

DL&PL Settings 也是链路层的一些高级设定，保持默认不修改，如图 3-26 所示。

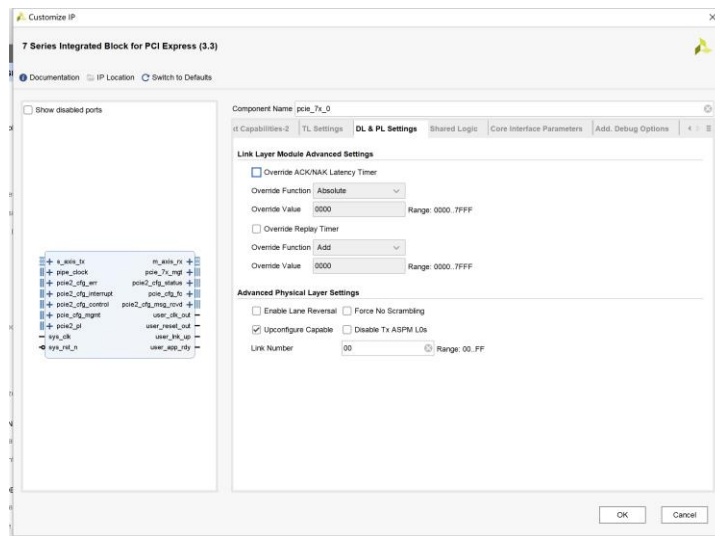


图 3-26 DL&PL Settings 保持默认

3.6.12 Shared Logic

Shared Logic 共享逻辑是有关 IP 核生成的某些修改，是生成的例子设计，对于 RIFFA 框架，取消所有勾选，如图 3-27 所示。

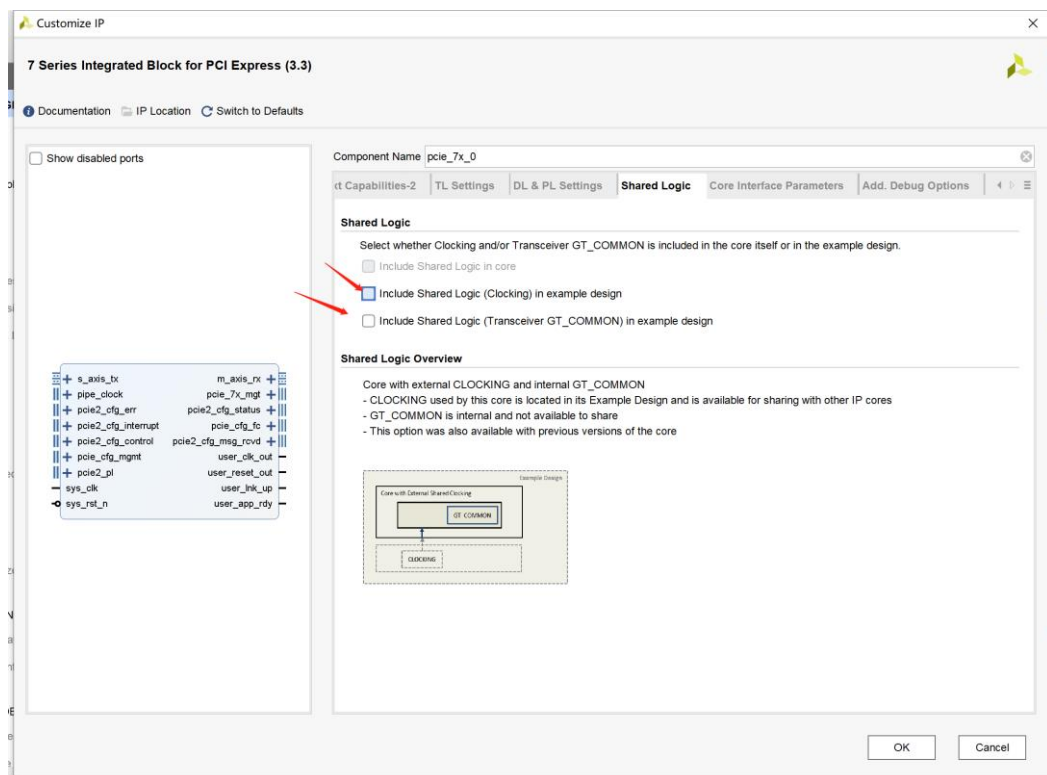


图 3-27 取消所有 Shared Logic

3.6.13 Core Interface Parameters

Core Interface Parameters 页有关一些接口的开启。配置如图 3-28 所示。

1. PL Interface: 编程的接口, 可以进行某些配置, 不使用。
2. Error RePorting: 错误报告接口, 不使用。
3. Config Management Interface: 电源管理接口, 不使用。
4. Config CTRL Interface: 配置配置 CTRL 的接口, riffa 需要, 此处勾选。
5. Config Status Interface: 配置标志接口, riffa 需要, 此处勾选。
6. Receive Msg Interface: 消息中断接口, 不使用。
7. Config FC Interface: 配置 FC 接口, riffa 需要, 此处勾选。

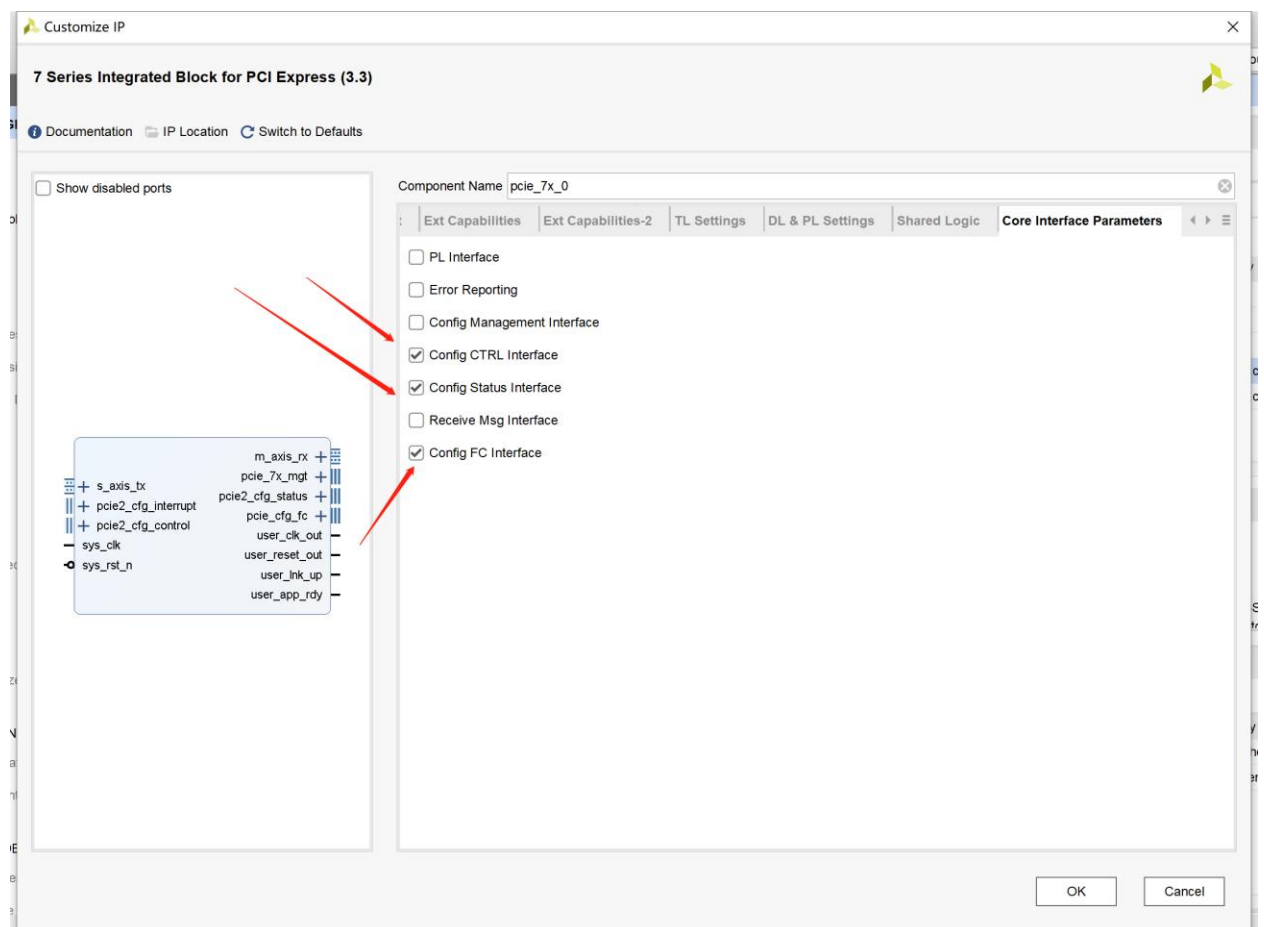


图 3-28 Core Interface Parameters 配置

3.6.14 Add.Debug Options

调试接口, PCIE 是具备 JTAG 调试接口的, 此页关于是否开启 JTAG 调试接口, 这里我们不使用, 保持不勾选, 如图 3-29 所示。

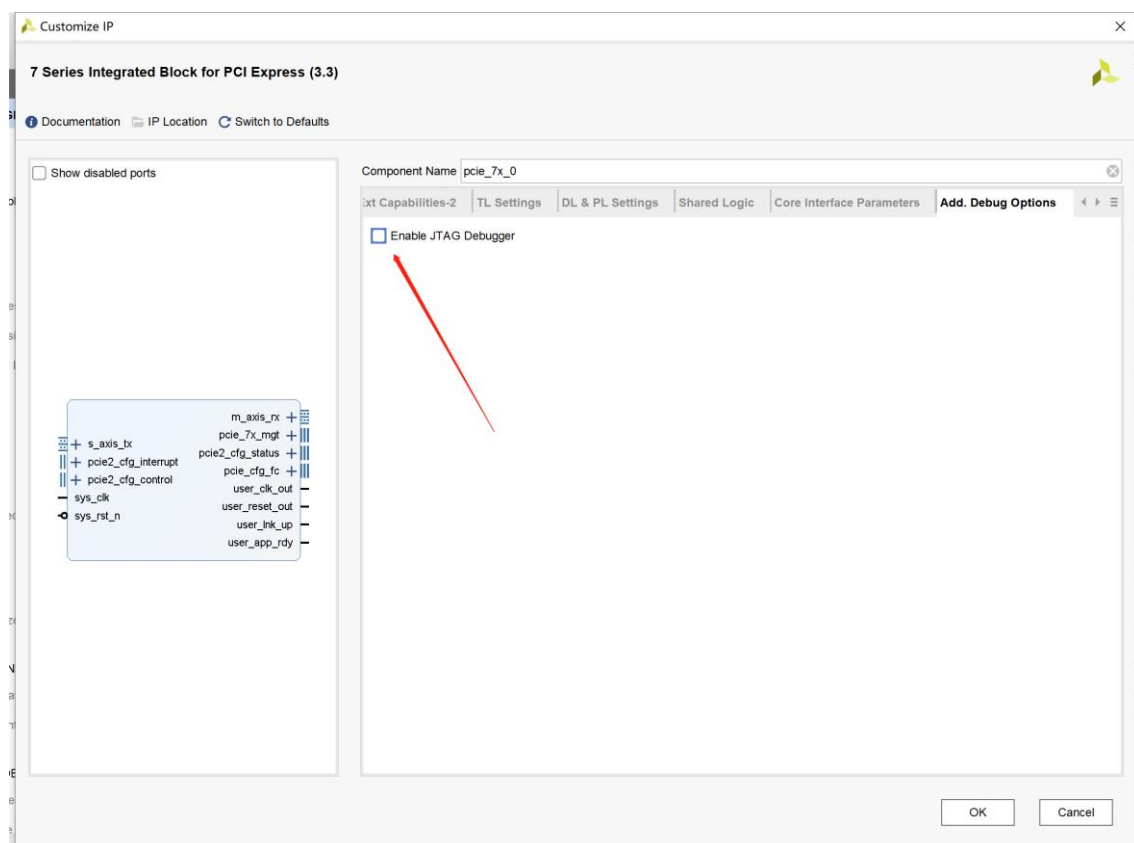


图 3-29 JTAG 调试选项

至此 PCIE IP 配置完成，点击 OK 完成。在弹出的生成输出文件对话框中，选择确定。如图 3-30 所示。

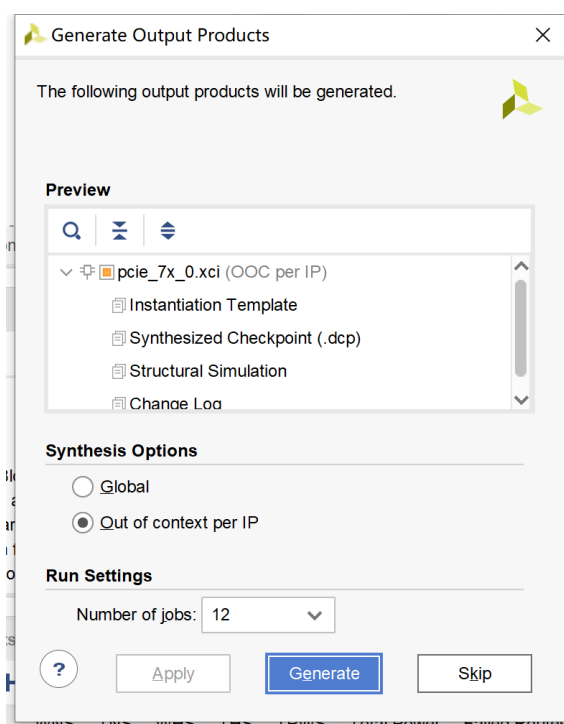


图 3-30 生成输出文件

3.6.15 Soft Reset 配置

PCIE IP 按照上述配置后,主机重启会导致卡不识别,因为主机重启了加速卡中的 PCIE IP 没有重启。这是因为没有配置软复位导致的, 这里我们在项目资源管理器中双击生成的 IP, 打开到 Power Management 选项, 取消勾选 No soft Reset, 如图 3-31 所示。这个设置在刚开始配置 IP 的时候不能配置, 不知道为什么。添加完成 Soft Reset 后, 重新生成 IP。

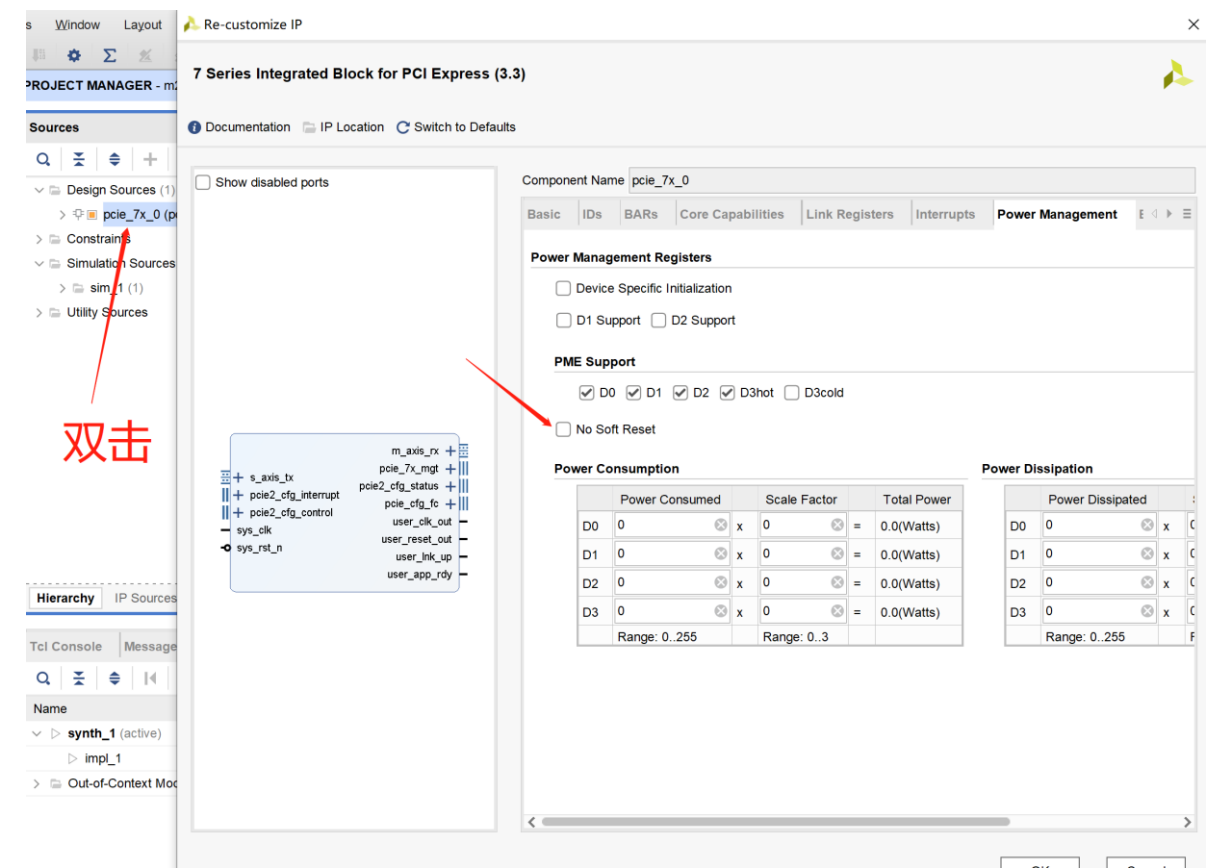


图 3-31 添加软复位

3.6.16 PCIE 通道修改

Xilinx 的 PCIE IP 生成完成后,默认会使用官方推荐的 PCIE 对应 GT 通道顺序, 由于加速卡硬件顺序和官方推荐顺序不一样, 在重新约束通道后, Vivado 软件布局布线时会报 Critical Warning, 虽然不影响使用, 但这里还是介绍一下如何消除。

打开如图 3-32 所示的 IP 配置完成后生成的约束文件, 将箭头所指的 4 处 PCIE Lane 所使用的 GT Lane 序号进行修改。**(A7-35T-100T 的 GTP 通道是 Lane0-Lane3 对应 X0Y0-X0Y3, A7-200T 的 GTP 通道是 Lane0-Lane3 对应 X0Y4-X0Y7)** 修改完成后保存即可。

```
pcie_7x_0-PCIE_X0Y0.xdc
> m2_artix7_riffa > m2_artix7_riffa.gen > sources_1 > ip > pcie_7x_0 > source > pcie_7x_0-PCIE_X0Y0.xdc
85 #####
86 #
87 # Transceiver instance placement. This constraint selects the
88 # transceivers to be used, which also dictates the pinout for the
89 # transmit and receive differential pairs. Please refer to the
90 # Virtex-7 GT Transceiver User Guide (UG) for more information.
91 #
92 #
93 # PCIe Lane 0
94 set_property LOC GTPE2_CHANNEL_X0Y0 [get_cells {inst/gt_top_i/pipe_wrapper_i/pipe_lane[0].gt_wrapper_i/gtp_channel.gtpe2_channel_i}]
95 # PCIe Lane 1
96 set_property LOC GTPE2_CHANNEL_X0Y1 [get_cells {inst/gt_top_i/pipe_wrapper_i/pipe_lane[1].gt_wrapper_i/gtp_channel.gtpe2_channel_i}]
97 # PCIe Lane 2
98 set_property LOC GTPE2_CHANNEL_X0Y2 [get_cells {inst/gt_top_i/pipe_wrapper_i/pipe_lane[2].gt_wrapper_i/gtp_channel.gtpe2_channel_i}]
99 # PCIe Lane 3
100 set_property LOC GTPE2_CHANNEL_X0Y3 [get_cells {inst/gt_top_i/pipe_wrapper_i/pipe_lane[3].gt_wrapper_i/gtp_channel.gtpe2_channel_i}]
101
102 # GTP Common Placement
103 set_property LOC GTPE2_COMMON_X0Y0 [get_cells {inst/gt_top_i/pipe_wrapper_i/pipe_lane[0].pipe_quad.gt_common_enabled.gt_common_int.gt_common_i}]
104
105 #
106 # PCI Express Block placement. This constraint selects the PCI Express
107 # Block to be used.
108 #
109
110 set_property LOC PCIE_X0Y0 [get_cells inst/pcie_top_i/pcie_7x_i/pcie_block_i]
```

图 3-32 修改 PCIE Lane 顺序

至此 PCIE IP 配置完成。

3.7 Riffa RTL 源文件及约束文件添加

这里使用本教程提供的已经修改好的 Riffa RTL 代码和约束文件。将 Riffa_Source 文件夹，如图 3-33 所示。复制到工程目录的 srcs 文件夹下，如图 3-34 所示。

修改的代码主要添加了 LED 灯的逻辑以及修改了官方的部分内容以适配 IP 设置。主要内容是，顶层源码中需要保持如下参数和 IP 配置一致。

```
module pcie_riffa
#(// Number of RIFFA Channels
parameter C_NUM_CHNL = 1,
// Number of PCIe Lanes
parameter C_NUM_LANES = 4,
// Settings from Vivado IP Generator
parameter C_PCI_DATA_WIDTH = 128,
parameter C_MAX_PAYLOAD_BYTES = 256,
parameter C_LOG_NUM_TAGS = 5
)
```

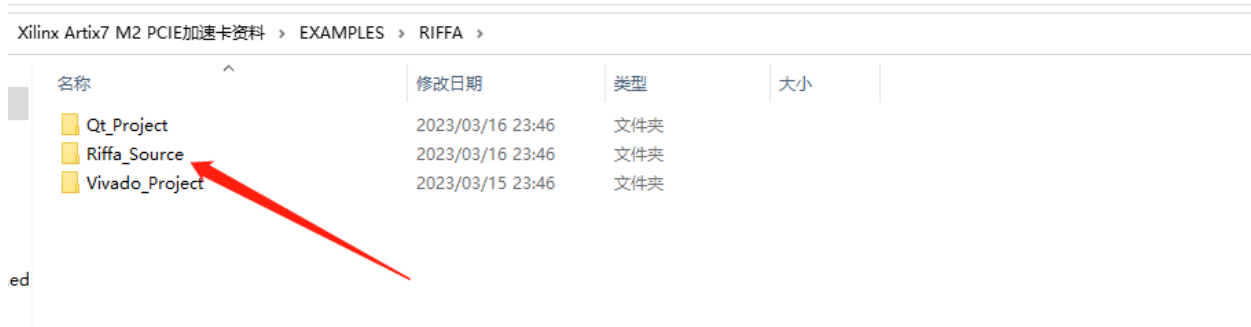


图 3-33 Riffa_Source 文件夹

m2_artix7_riffa > m2_artix7_riffa.srcs >			
名称	修改日期	类型	大小
Riffa_Source	2023/03/14 20:26	文件夹	
sources_1	2023/03/14 20:01	文件夹	

图 3-34 Riffa_Source 文件夹复制到工程源文件目录下

在 Vivado 软件的工程管理窗口中，右键工程 Design Source 选项，选择添加源文件，如图 3-35 所示。

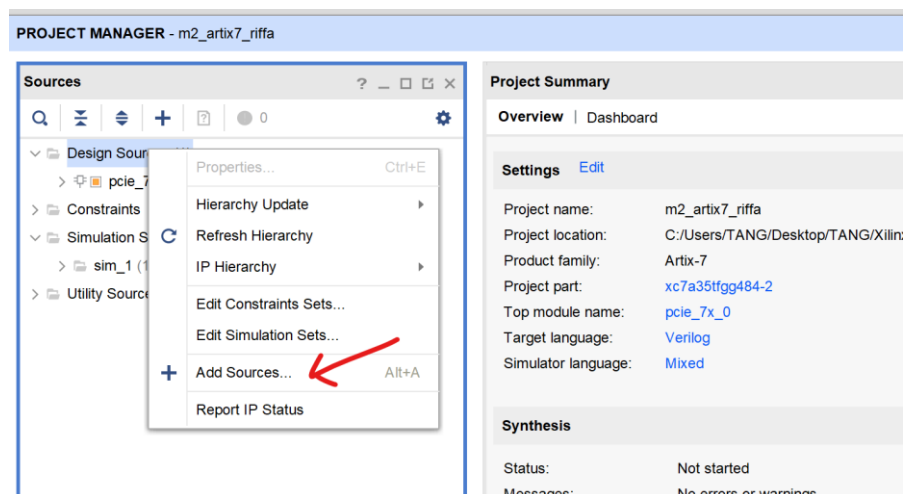


图 3-35 添加源文件

点击 Next。

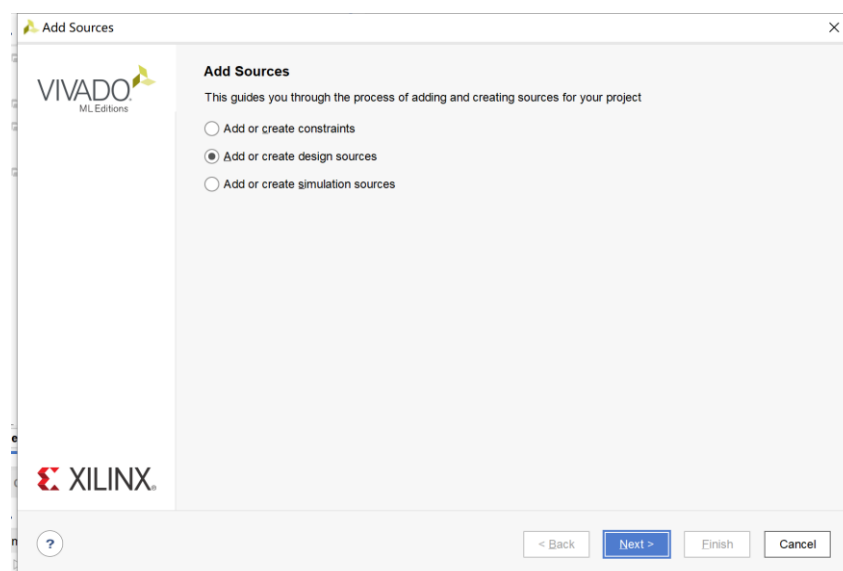


图 3-36 添加文件

在窗口中选择添加路径，如图 3-37 所示。

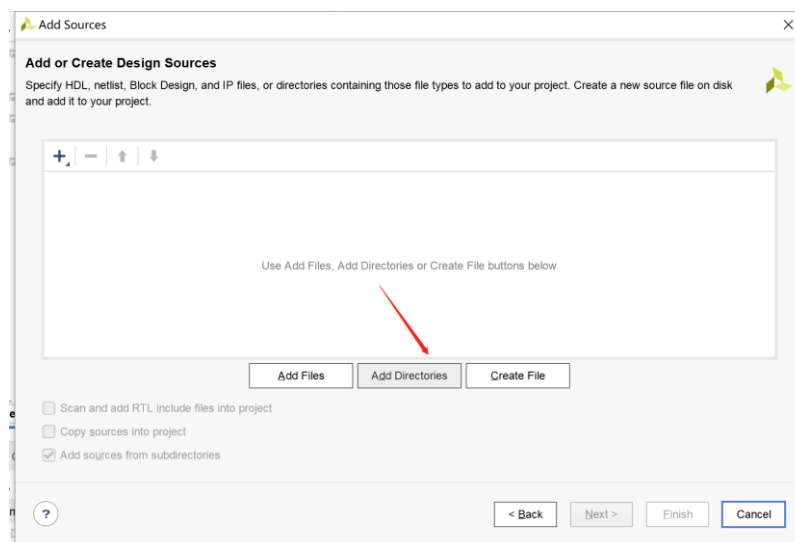


图 3-37 选择添加路径

将 Riffa_Source 文件夹下的 source_1 文件夹选中后点击 Select，然后点击 Finish，如图 3-38 所示。

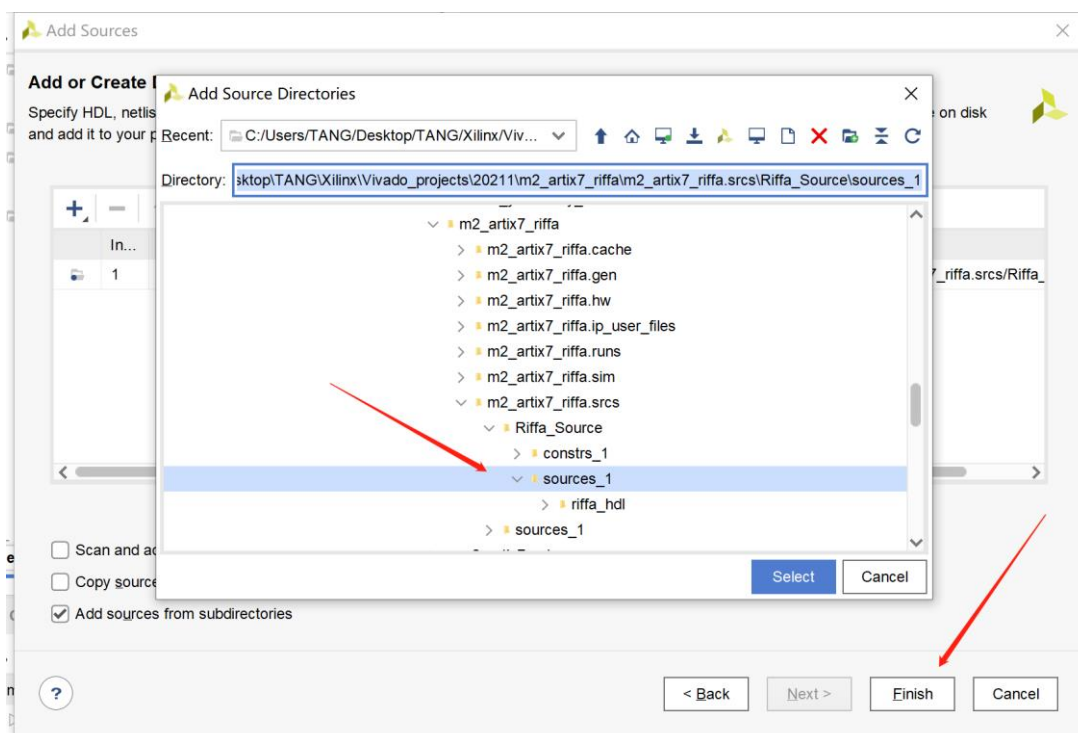


图 3-38 添加提供的 Riffa_Source 文件夹下的 source_1

完成路径添加后，工程管理目录中会自动出现所有的文件和结构。如图 3-39 所示。

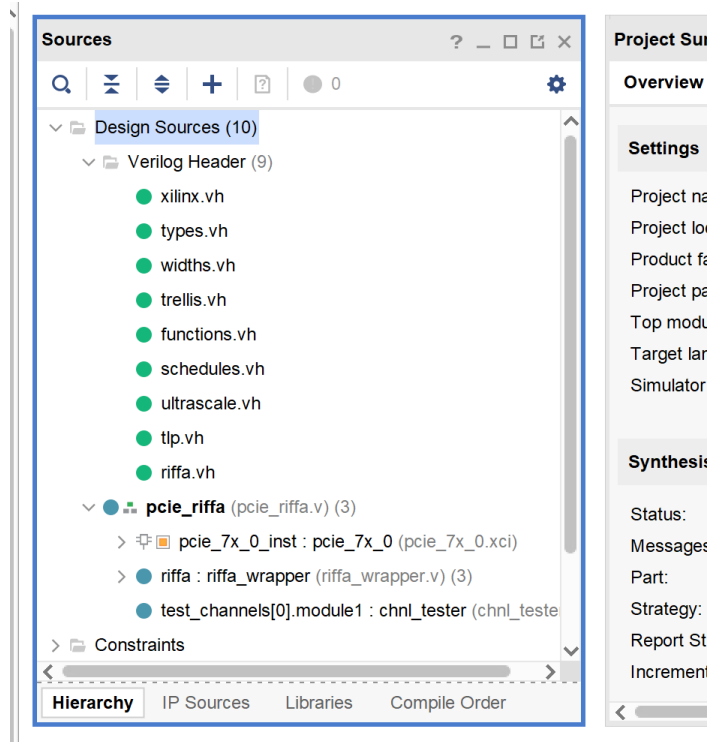


图 3-39 Riffa 文件结构

将 functions.vh 右键，在菜单中设置为 Global include，如图 3-40 所示。

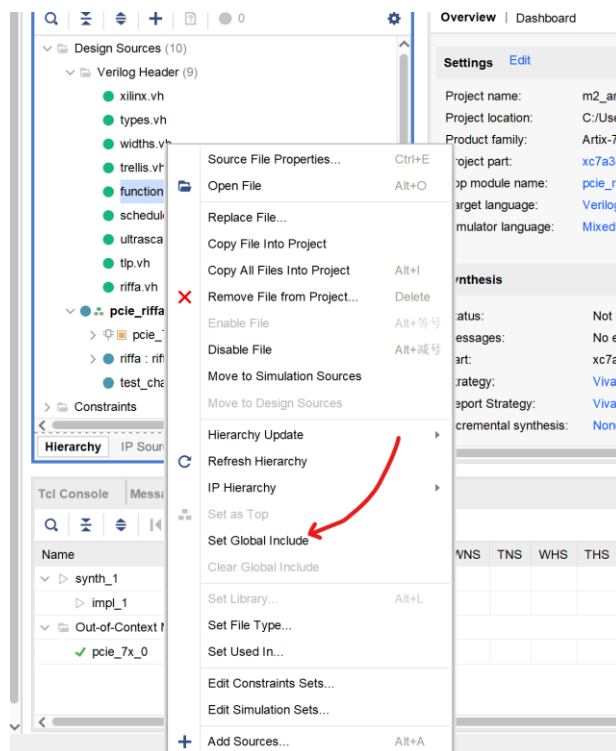


图 3-40 设置 functions.vh 为 Global include

在 Constraints 选项上右键，选择添加约束文件，添加 Riffa_Source 文件夹下 constrs_1 文件夹下的 pcie_riffa.xdc 约束文件，点击 Finish。如图 3-41 所示。

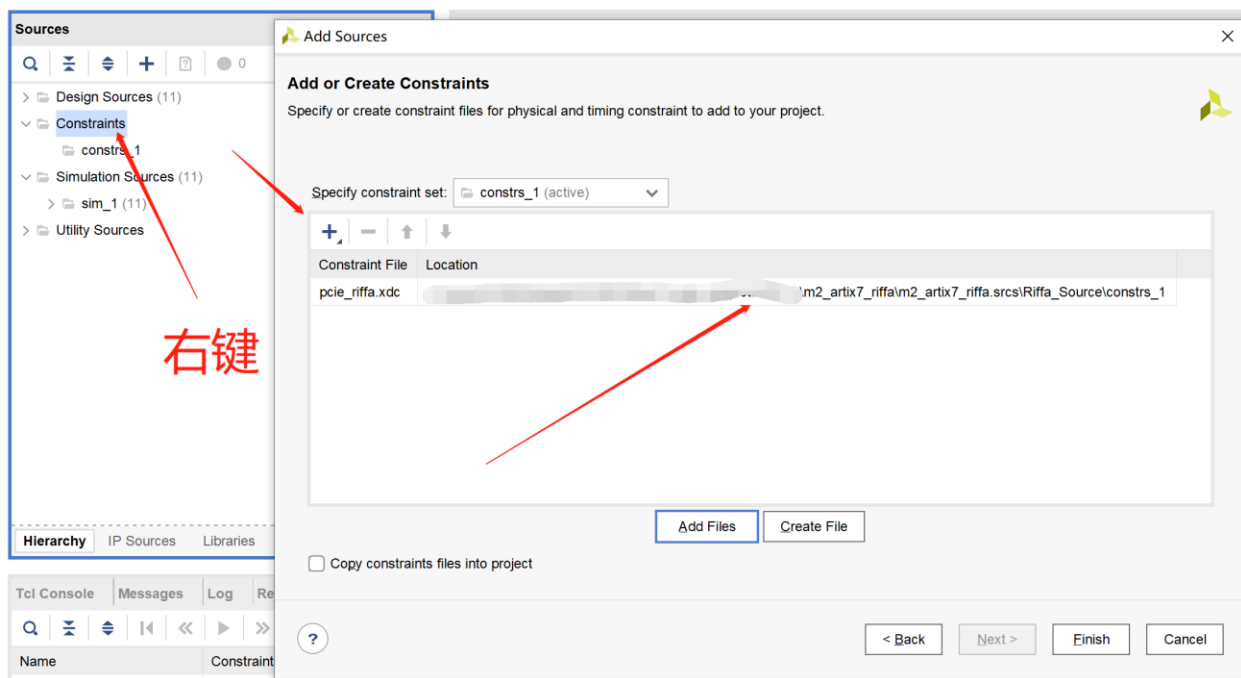


图 3-41 Riffa 约束文件添加

添加完成后，即完成工程创建，点击 **Generate Bitstream** 生成比特流文件。生成完成后可以看到 Riffa 的资源占用比 XDMA 占用少很多。如图 3-42 所示。

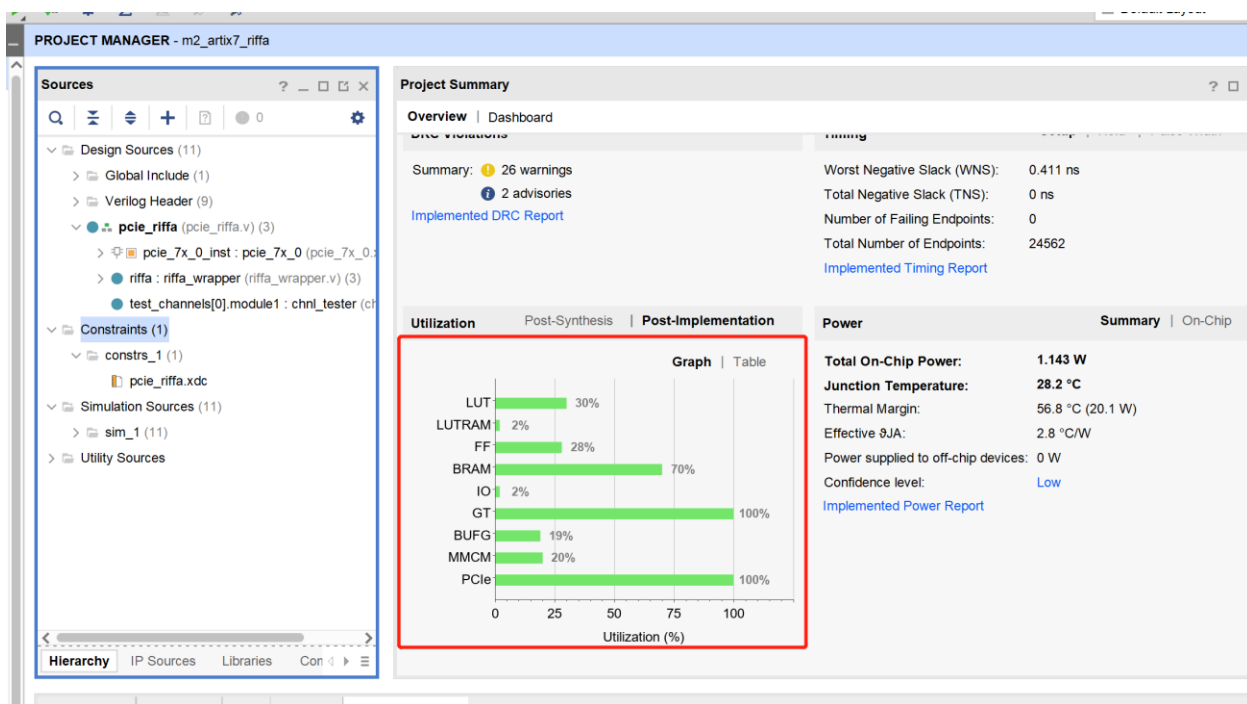


图 3-42 Riffa 综合结果

3.8 上机测试

3.8.1 Flash 烧录

将 Riffa 工程生成的 Bin 文件烧录进 Flash 中，具体操作按照第 1-3 节生成 Bin 文件并烧录进 Flash 中的流程，这里不再赘述 Flash 固化过程。

3.8.2 安装加速卡

将烧录好 Flash 的 M2 加速卡安装至电脑的 M2 接口上，注意是要 M2 接口 NVME 协议的，安装完成后电脑开机，加速卡上电灯亮，且绿色 LED 亮。如图 3-43 所示。

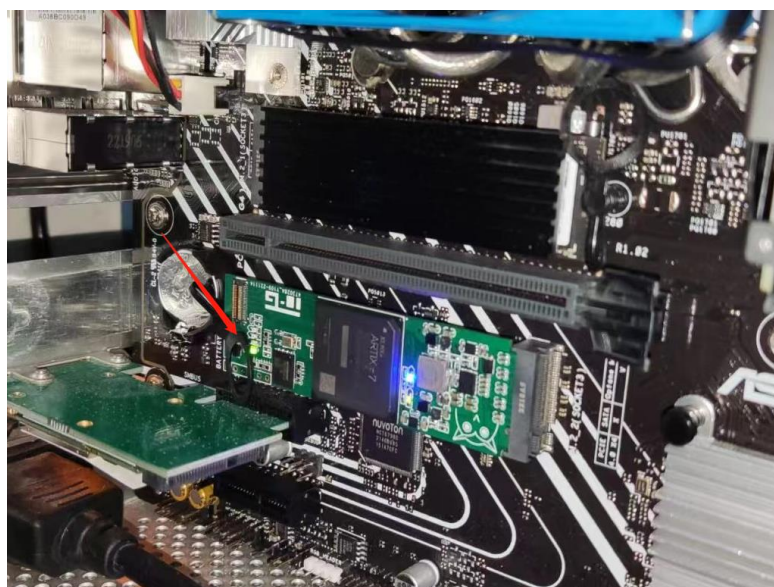


图 3-43 安装加速卡并开机

在 Windows 系统中的设备管理器中，也可识别到 Riffa Device 设备，如图 3-44 所示。



图 3-44 设备管理器中识别到 Riffa 设备

3.8.3 Qt 软件工程编译和运行测试

打开 Qt Creator，选择打开工程，打开提供的 test_speed 工程文件，如图 3-45 所示。

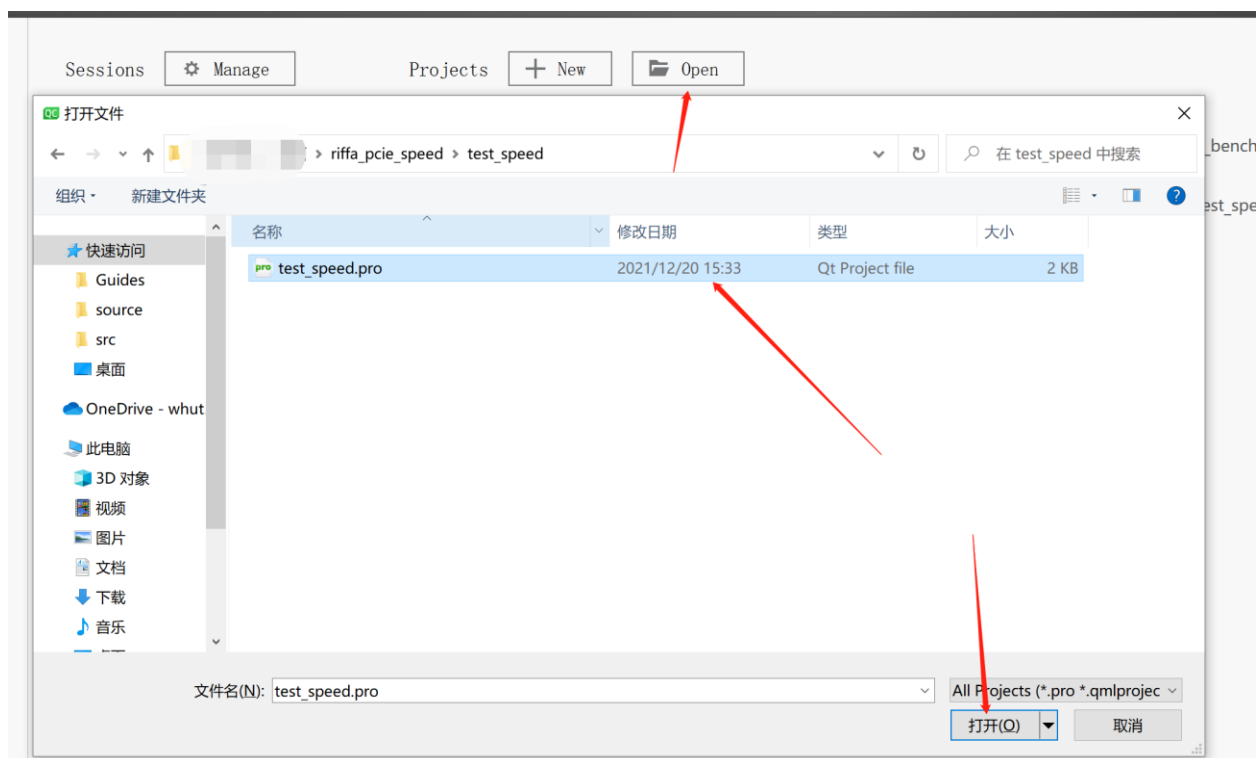


图 3-45 打开 Qt test_speed 工程

打开测速工程，其中 pcie_func 为主要文件，代码可自行阅读，如图 3-46 所示。

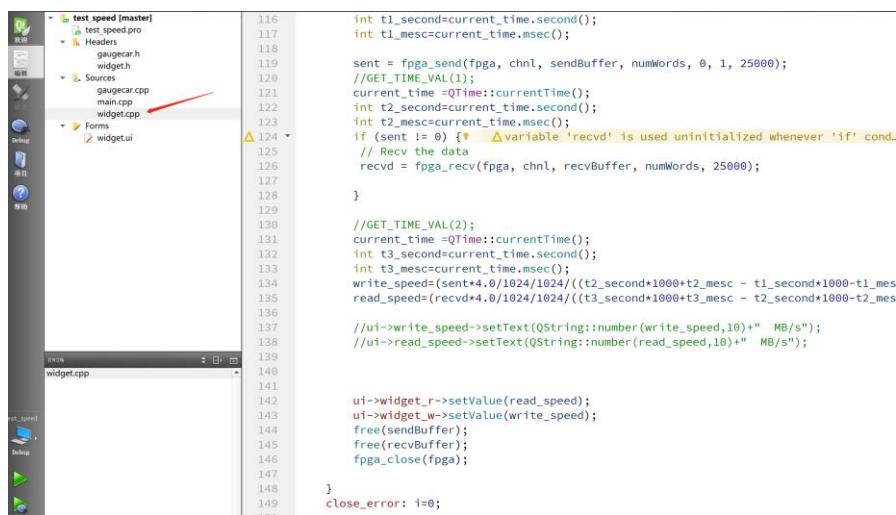


图 3-46 PCIE 测速工程

点击左下角锤子图标编译工程，编译完成弹出一些警告，可以忽视，如图 3-47 所示。

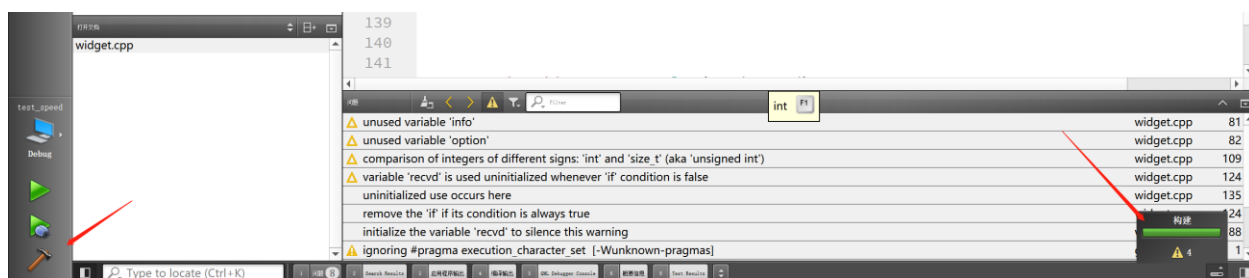


图 3-47 编译测速工程

点击左下角的运行按钮，会开始 Riffa 的环回测速，并以码表的 GUI 形式展现出来。点击 GUI 界面中的左侧下拉菜单测试模式后，点击右边的 Start 开始测试，即开始进行 Riffa 环回测试并显示速度，如图 3-48 所示。

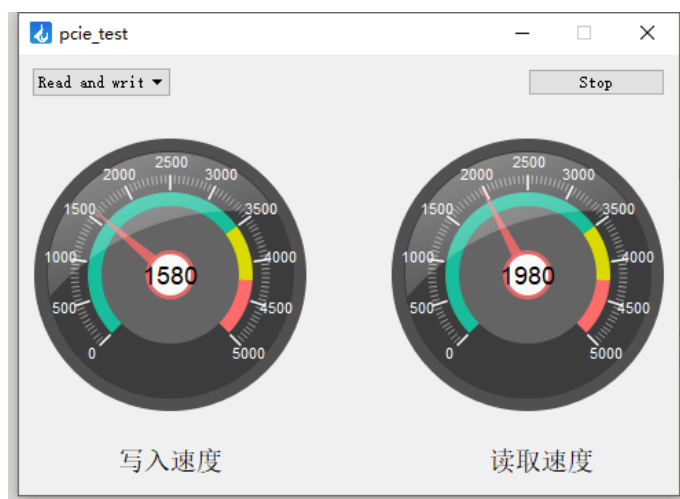


图 3-48 Riffa 环回测试结果

至此基于 Riffa 的环回速度测试结束。

PCIE2.0*4 的理论带宽是 $(5GT/s*4)*(8/10)*(1/8)=2000MB/s$ ，其中 8/10 是 8b/10b 编码，1/8 是比特到字节转换。

可以看到 Riffa 测速结果是写入 1580MB/s，读取 1980MB/s 很接近理论带宽了。

3.9 后记

本 Riffa 教程参考野火的 PCIE 修炼秘籍教程修改而来，野火的 PCIE 教程提供了 Riffa 的更多实例，有兴趣可参阅附录中的野火 PCIE 教程。在实际使用过程中，Riffa 的效率和速度相较于 XDMA 确实存在不少优势，且由于开源和驱动经过了签名，可以有更多的操作空间。

第四章 光通信 ibert 测试

4.1 硬件准备

准备光通信扩展底板，将光模块安装至插槽中，并确保时钟芯片输出频率配置拨动开关选择到 125MHz 时钟输出对应位置，即 11010，如图 4-1 所示。

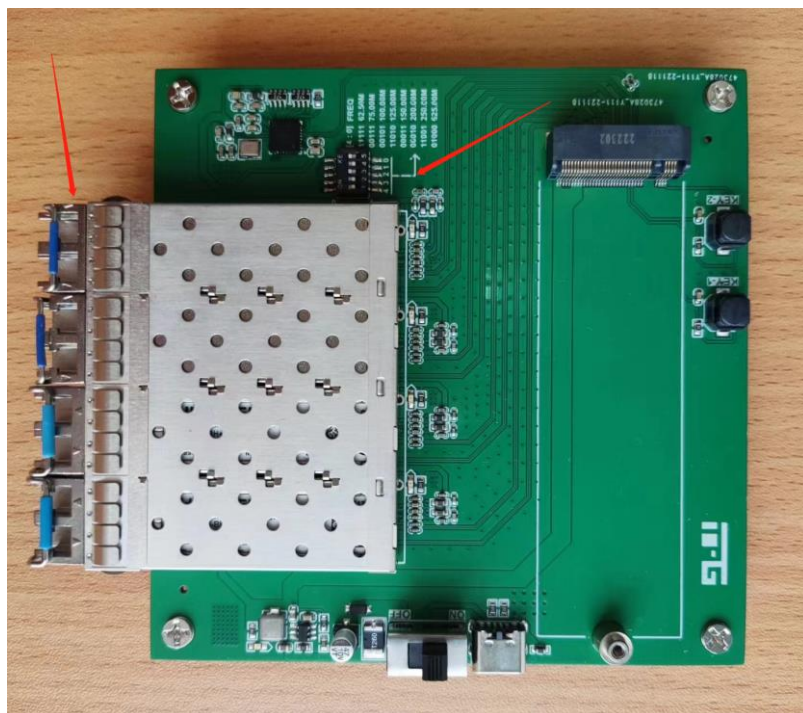


图 4-1 光模块安装与时钟芯片配置

并将 M2 FPGA 核心板安装至底板的 M2 安装槽中并使用螺丝固定，如图 4-2 所示。

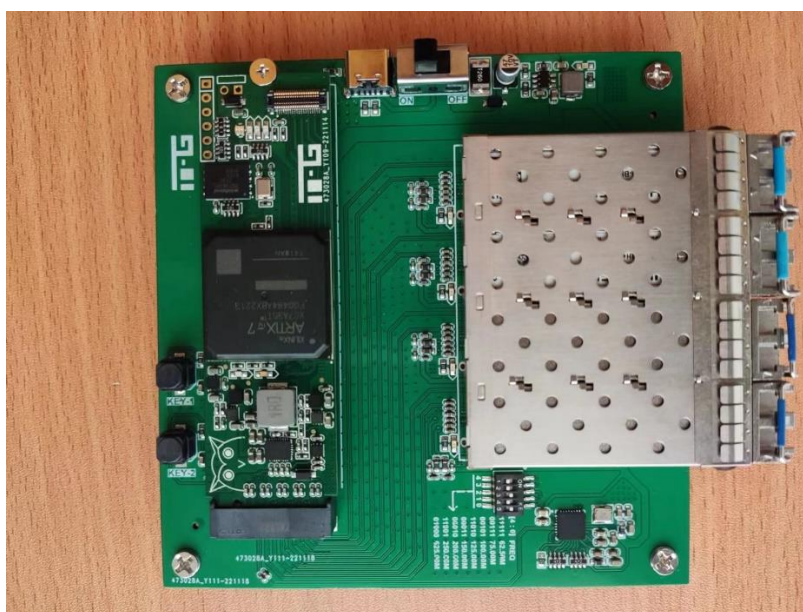


图 4-2 FPGA 核心板安装

4.2 创建 Vivado 测试工程

打开 Vivado 软件，选择 Create Project 并点击 Next。如图 4-3 所示。

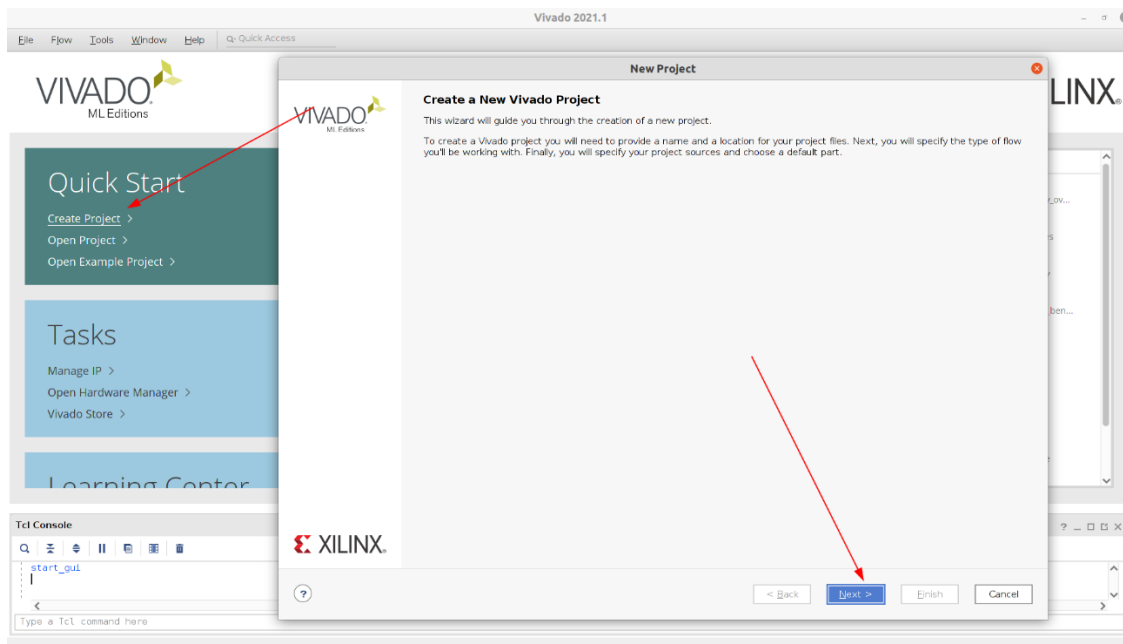


图 4-3 创建 Vivado 工程

输入工程名称和工程存放路径,以 m2_artix7_ibert 为例,如图 4-4 所示.

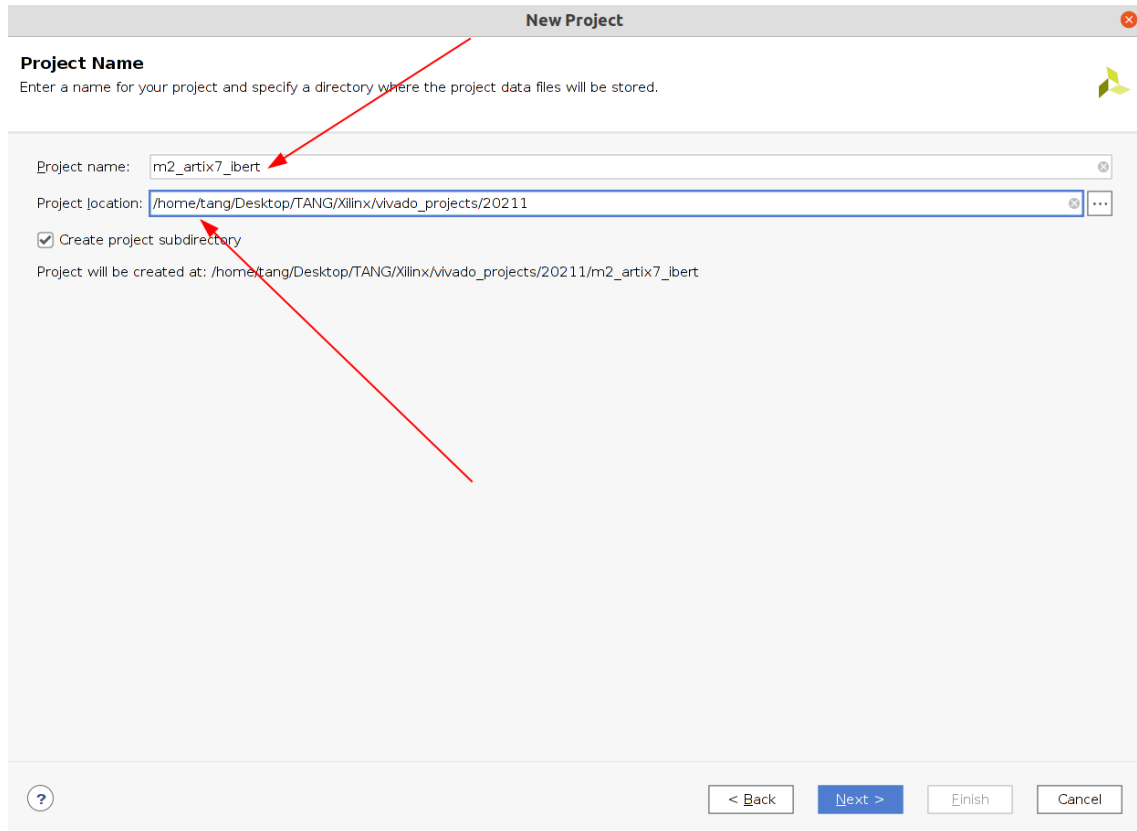


图 4-4 输入工程名称和路径

选择 RTL Project 并且勾选”当前不指定源文件”,如图 4-5 所示.

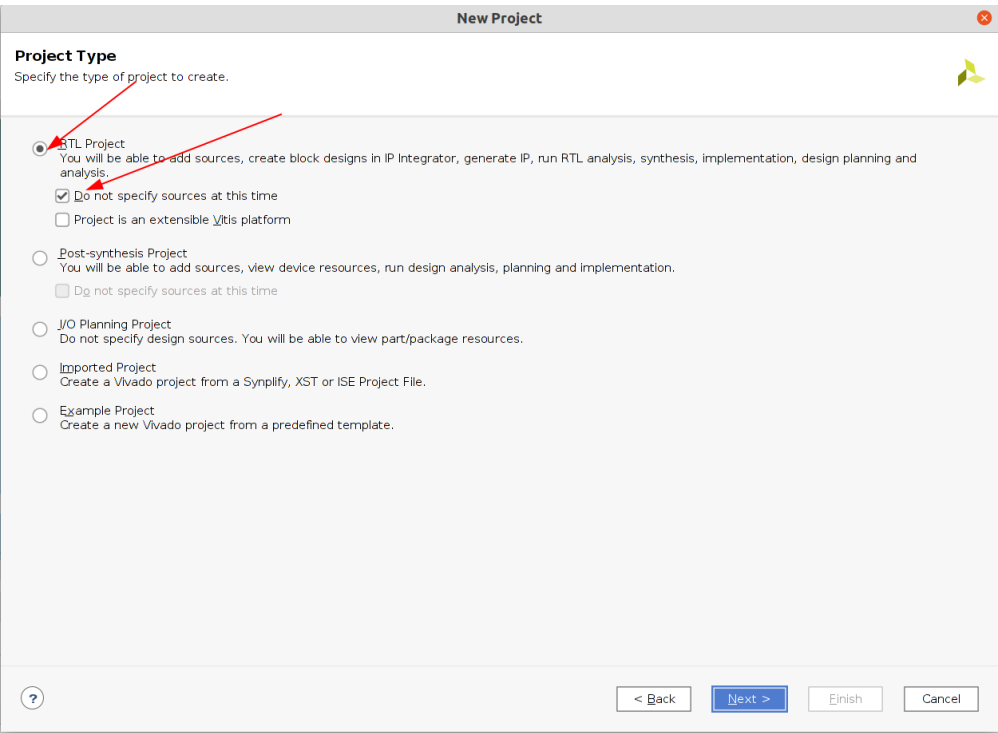


图 4-5 选择 RTL Project

根据核心板芯片型号选择芯片后点击 Next,以 XC7A35T-2FGG484I 为例,如图 4-6 所示.

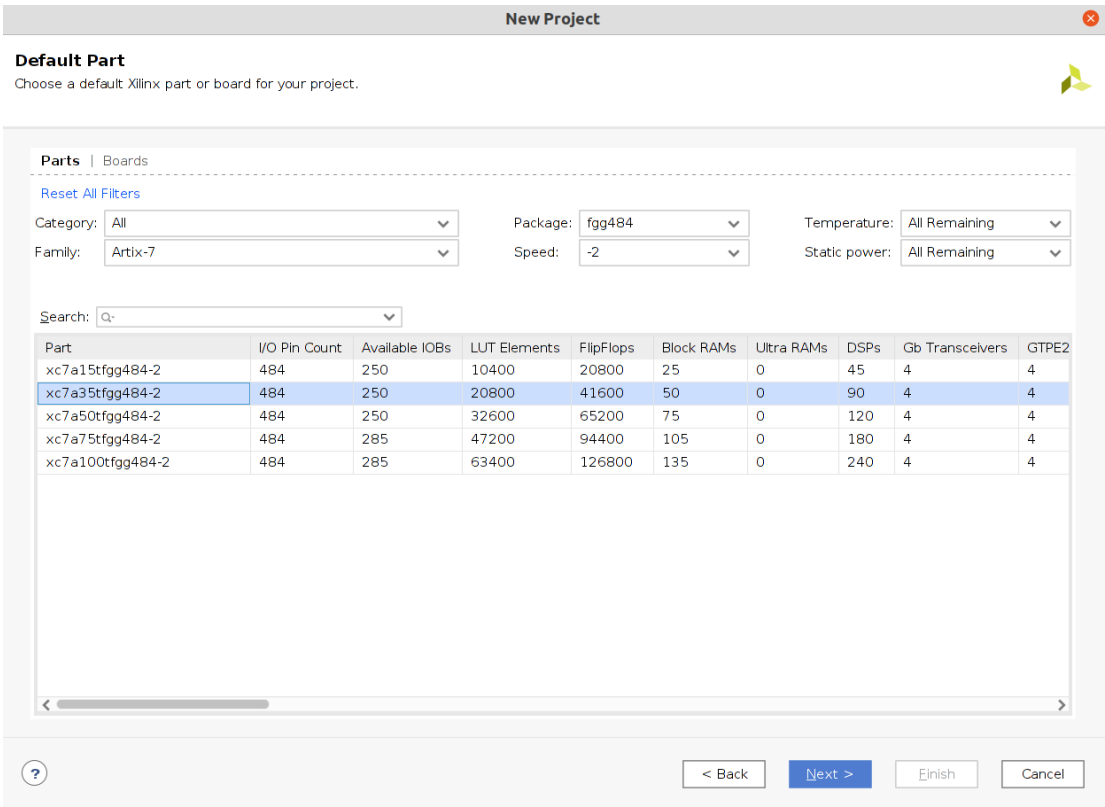


图 4-6 选择芯片型号

在新窗口中点击 Finish 完成工程创建,进入工程主界面,选择 IP catalog 如图 4-7 所示.

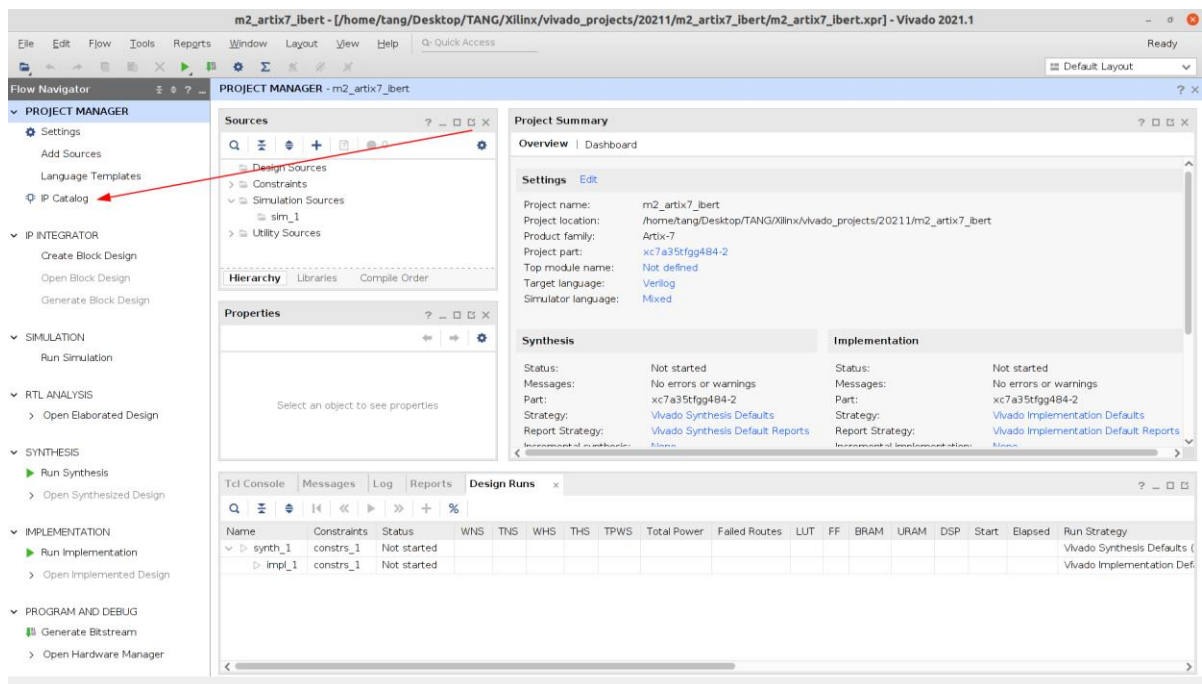


图 4-7 工程主界面选择 IP Catalog

在 IP Catalog 中的 Search 框中输入 ibert,找到 IBERT 7 Series GTP 并双击进入配置界面. 如图 4-8 所示.

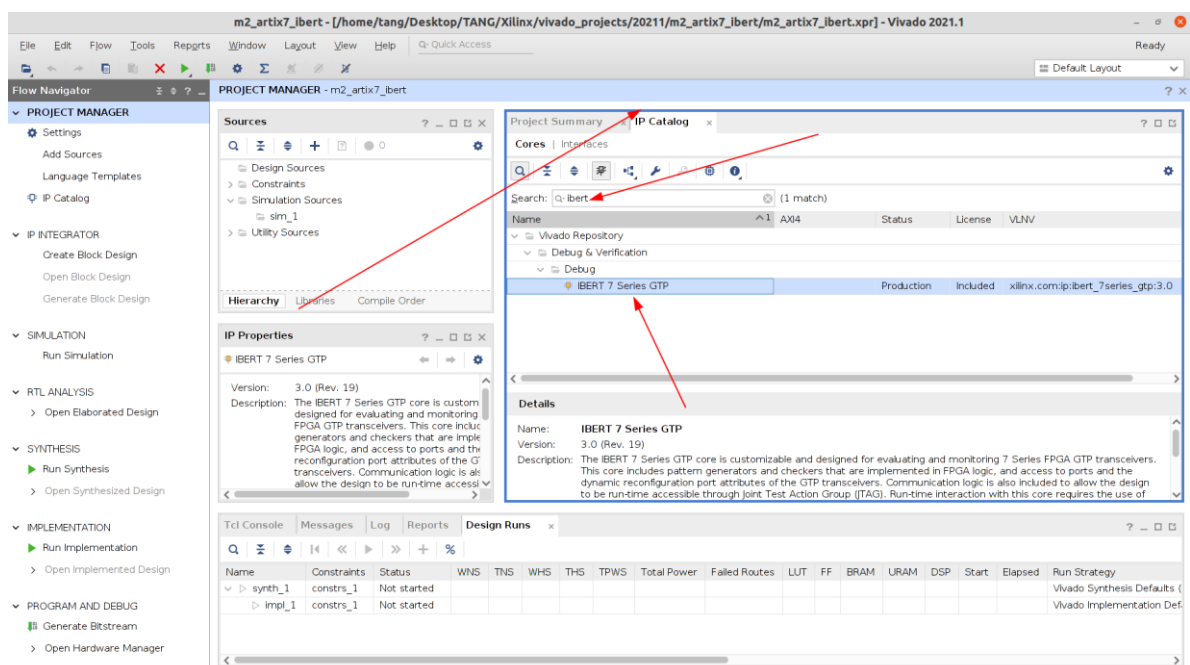


图 4-8 找到并双击进入 ibert IP 配置

在 IP 配置界面中,选择 Protocol Definition ,LineRate 填入为 6.25Gbps,Refclk 选择 125MHz,如图 4-9 所示.

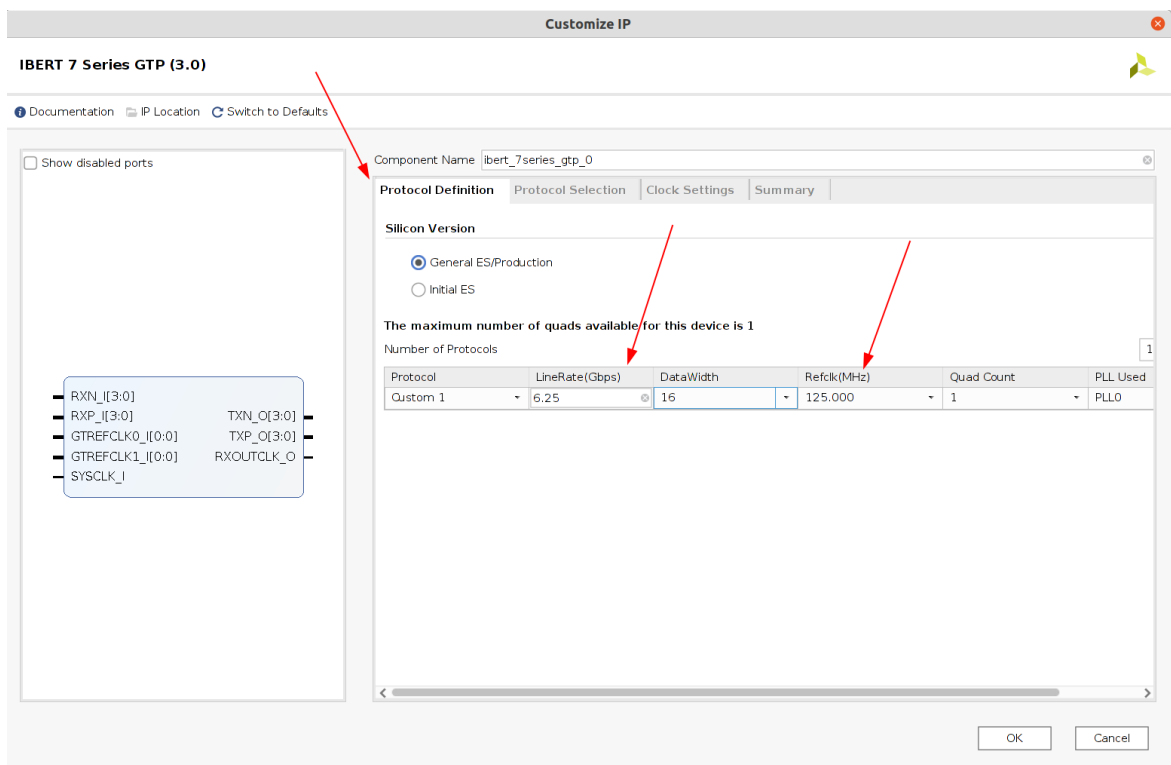


图 4-9 IP 协议指定

选择 Protocol Selection, 按照图 4-10 选择 6.25Gbps 协议, 参考时钟输入选择 MGTREFCLK1_216, Source 选择 Channel0.

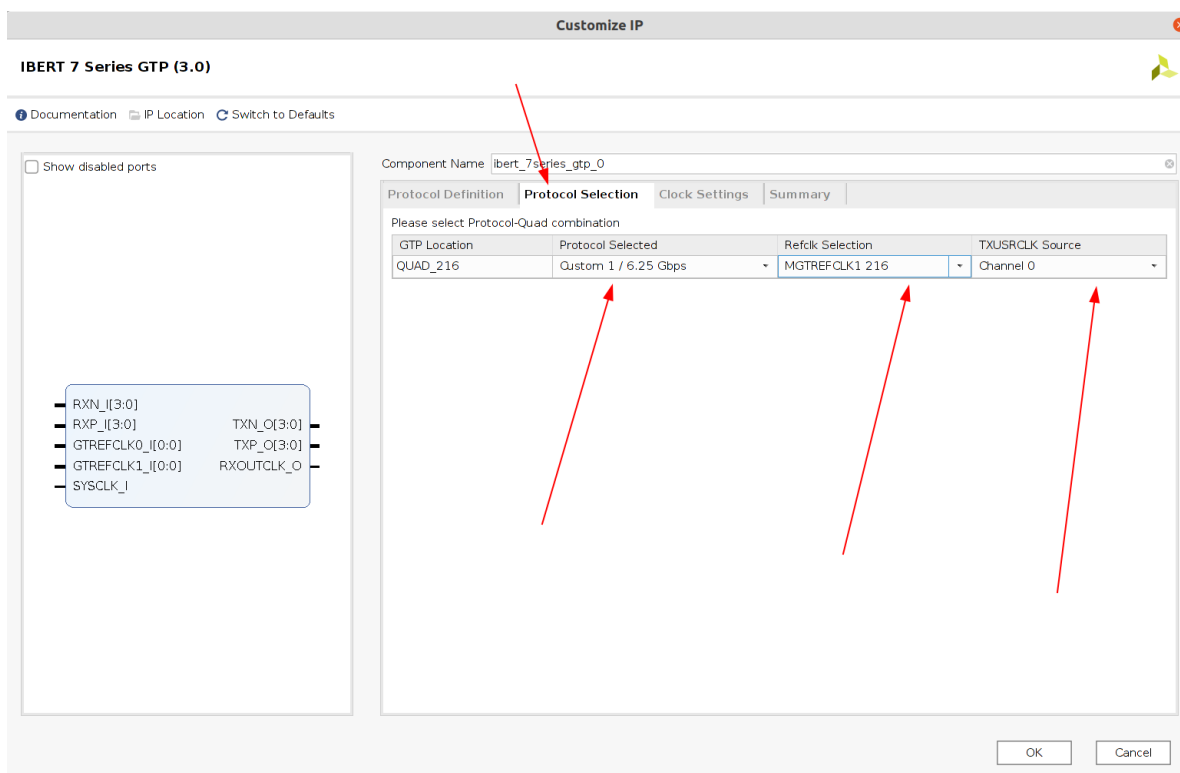


图 4-10 IP 协议选择

选择 Clock Setting, Source 选择 QUAD216_1,如图 4-11 所示.

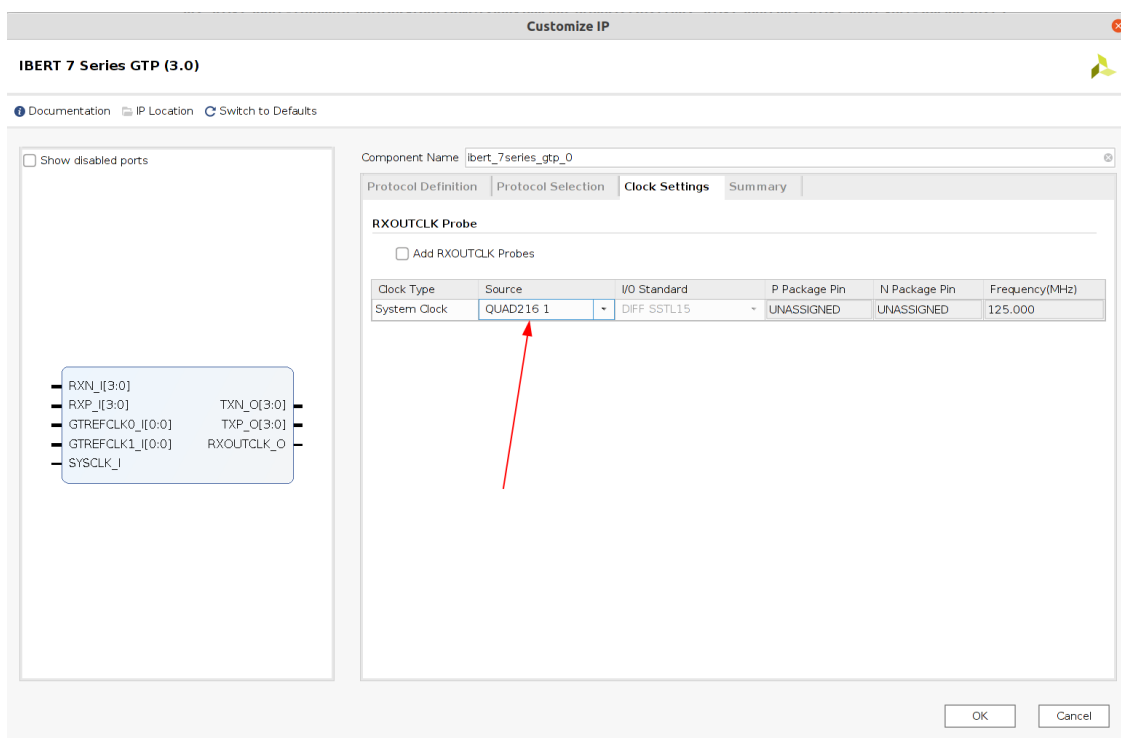


图 4-11 IP 时钟设置

在 Summary 选项中确认配置无误后,点击 OK 完成 IP 设置,如图 4-12 所示.

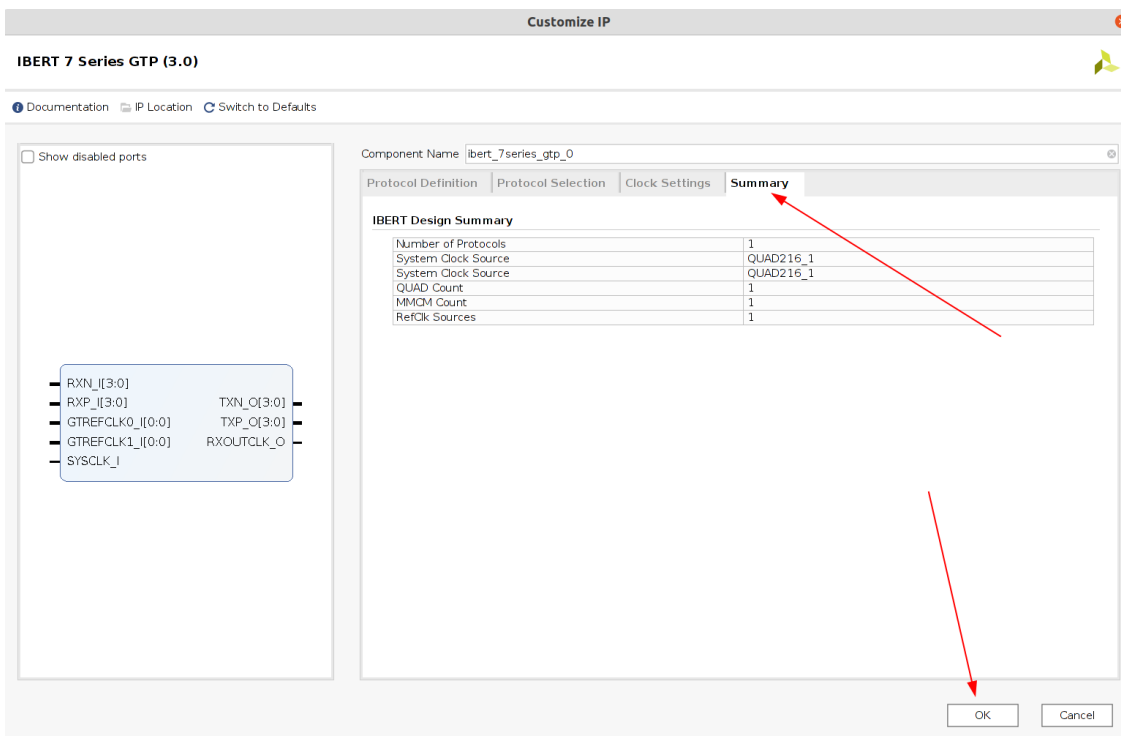


图 4-12 完成 IP 设置

在弹出的界面中点击 Skip,暂时不生成输出文件.如图 4-13 所示.

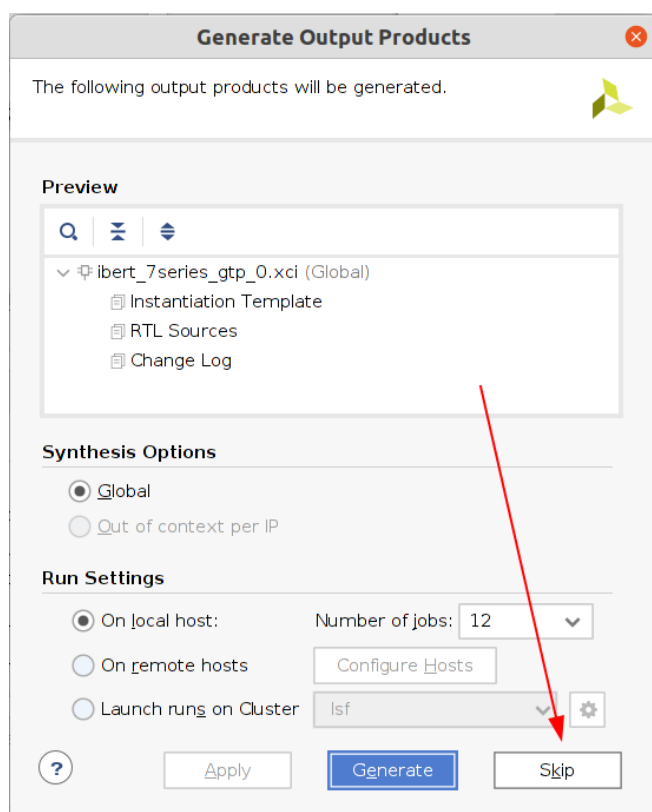


图 4-13 跳过输出文件生成

在 Design Source 下, **右键单击** ibert IP, 选择 Open IP Example Design ,如图 4-14 所示。

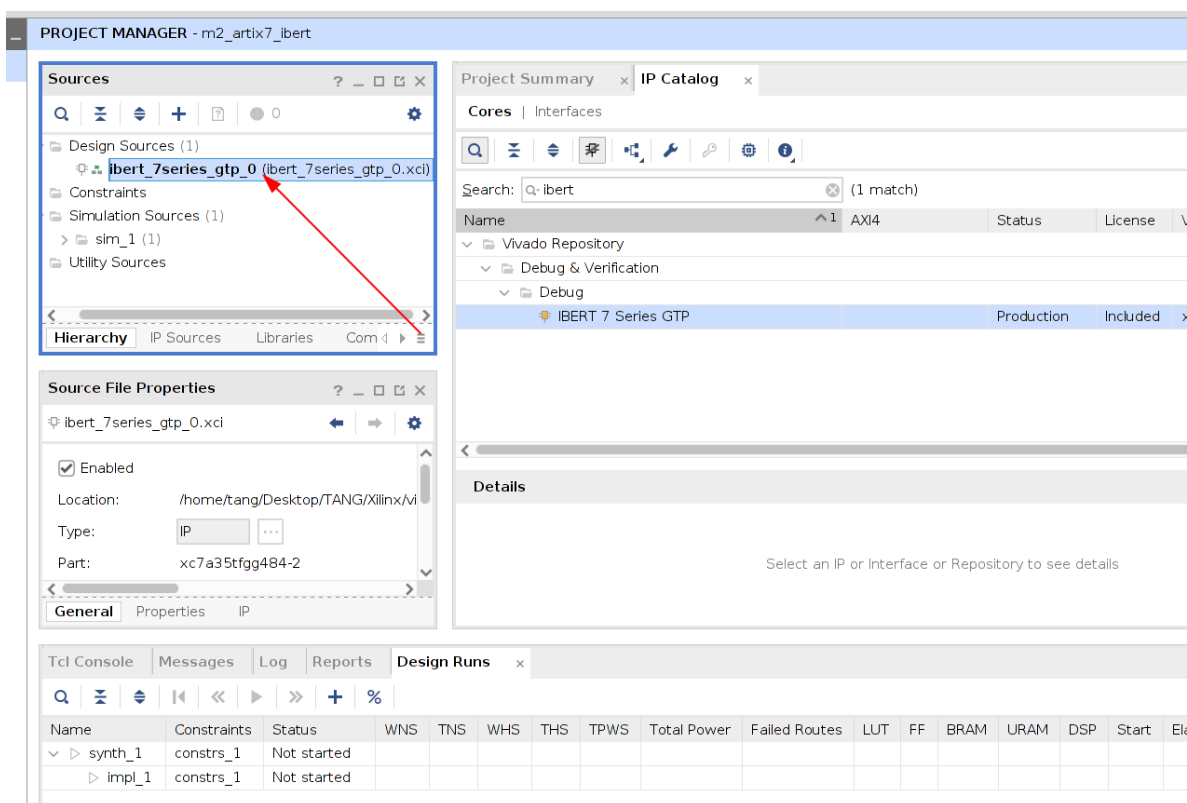


图 4-14 选择输出 Example Design

选择示例工程输出目录,如图 4-15 所示.

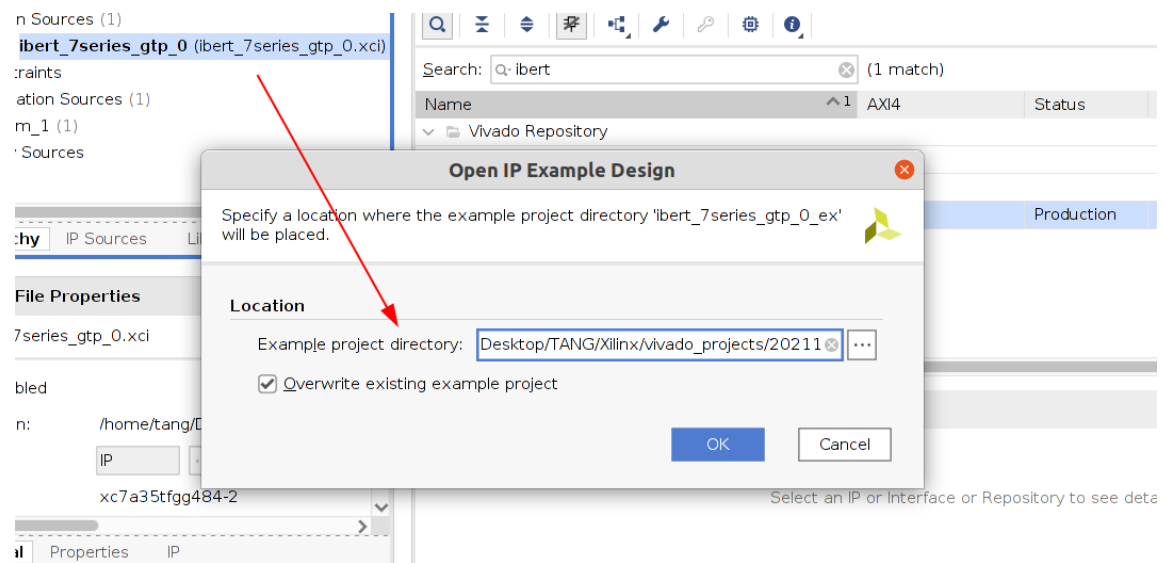


图 4-15 选择 Example Design 生成目录

点击 OK 后,Vivado 会自动生成一个新的 Example 工程并自动打开,如图 4-16 所示.由于之前已经配置好了,直接点击 Generate Bitstream 直接生成比特流.

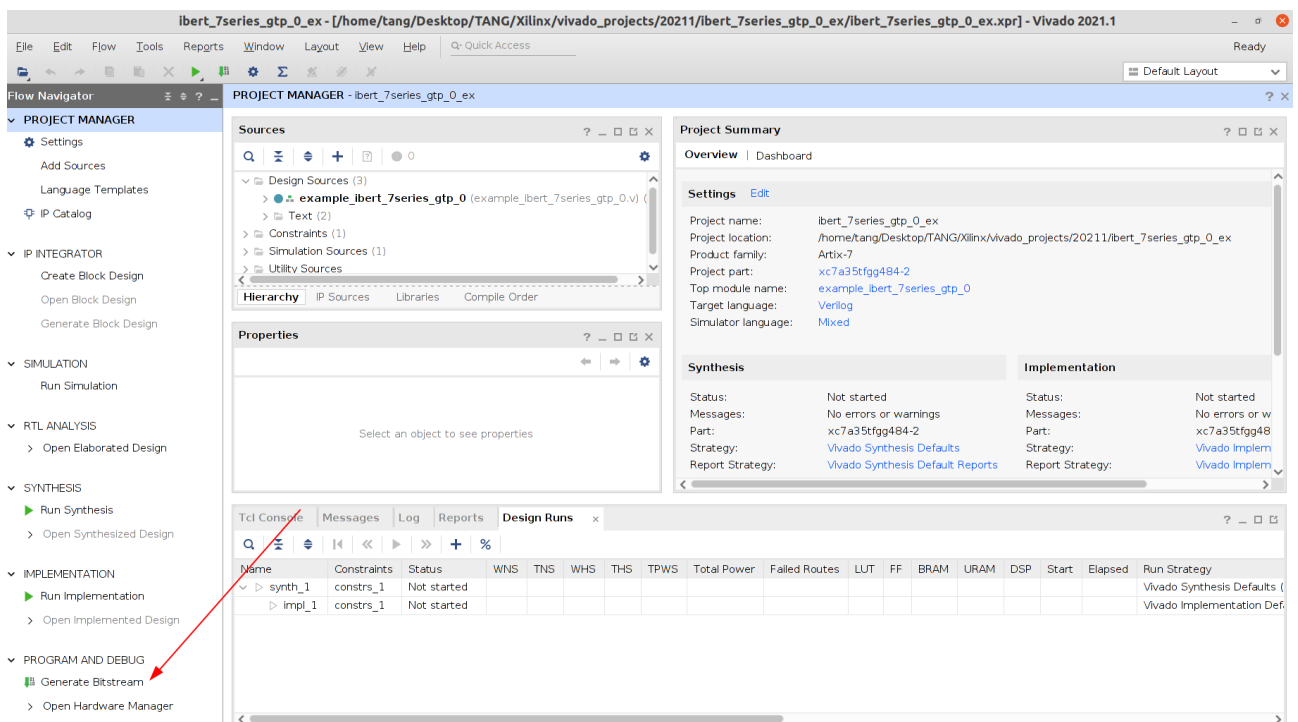


图 4-16 ibert 示例工程

4.3 ibert 测试

Bitstream 生成完成后,连接好 JTAG 和底板供电,并用光纤连接光模块,打开电源,如图 4-17 所示.

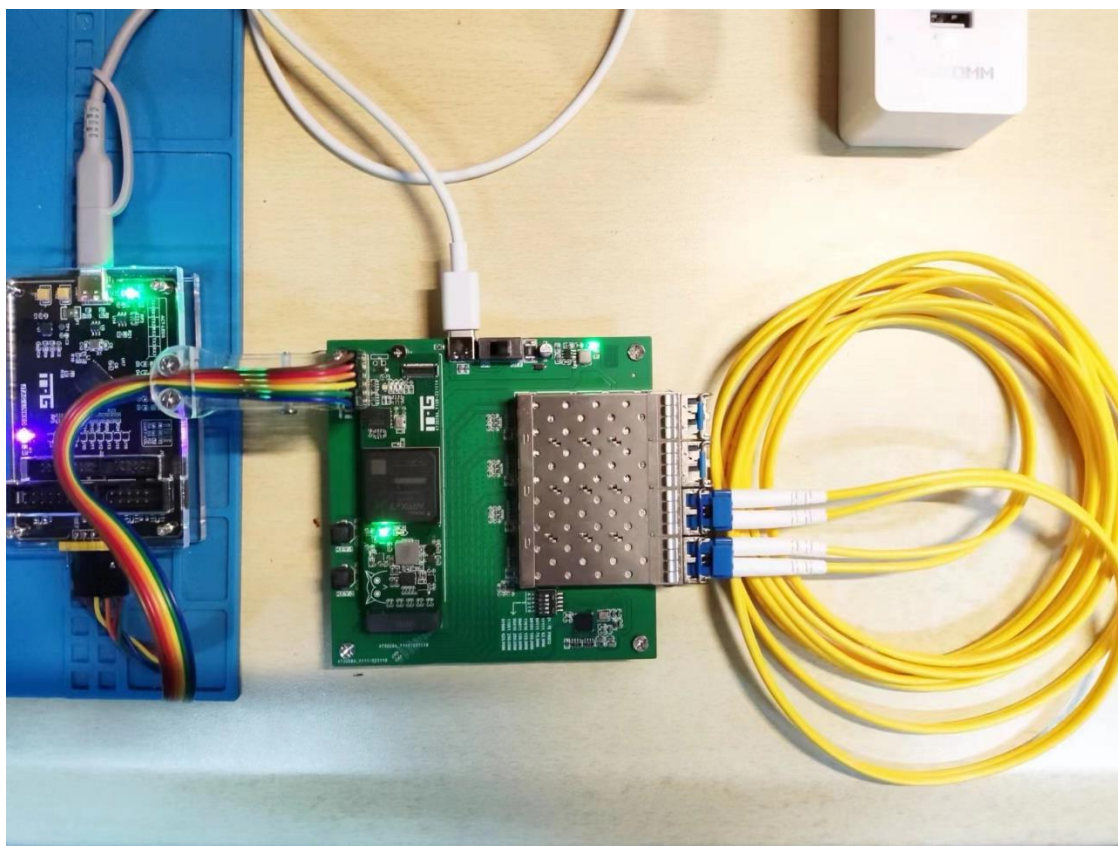


图 4-17 连接光纤和下载器，并给开发板上电

选择 Vivado 中 Open Hardware Manager,并点击 Open Target 选择 Auto Connect,连接到开发板,识别到芯片,如图 4-18 所示.

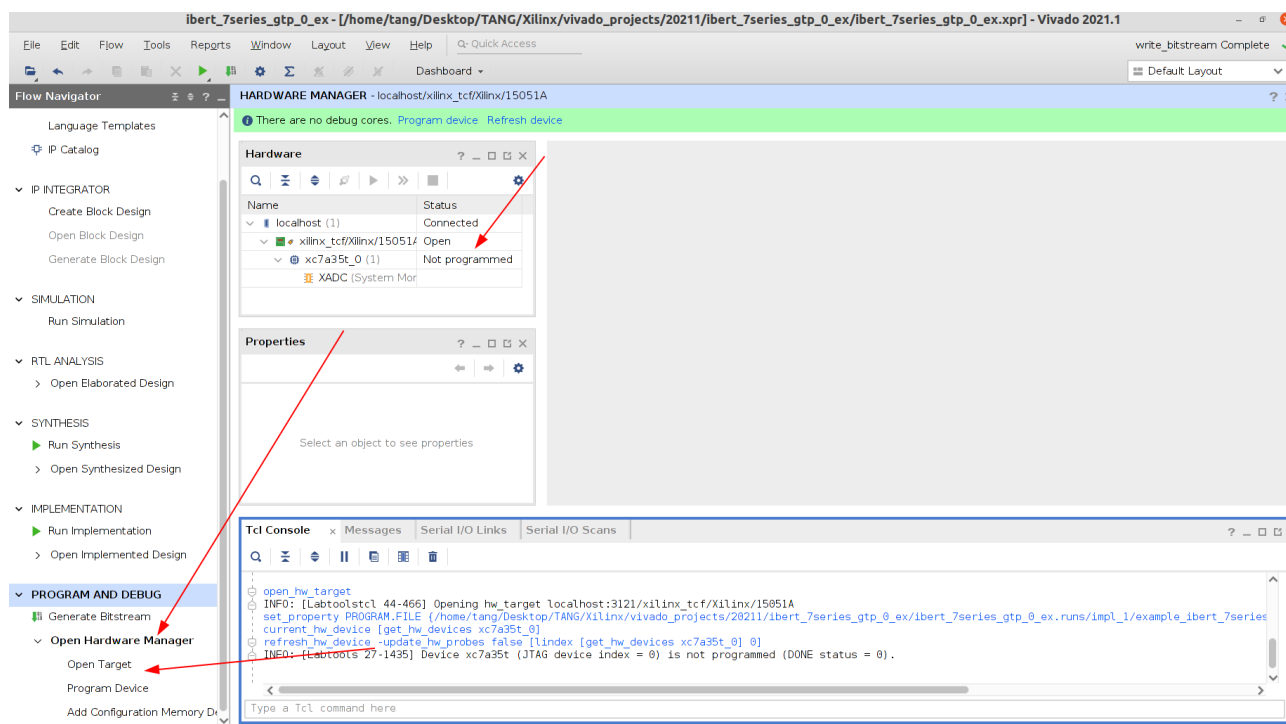


图 4-18 JTAG 连接和识别芯片

烧录刚刚生成的 Bitstream,ibert 开始运行,会自动出现如图 4-19 的 PLL 锁定.在下方的 Serial IO Link 窗口中,点击 Auto-detect links,会自动扫描到进行环回的两个通道。

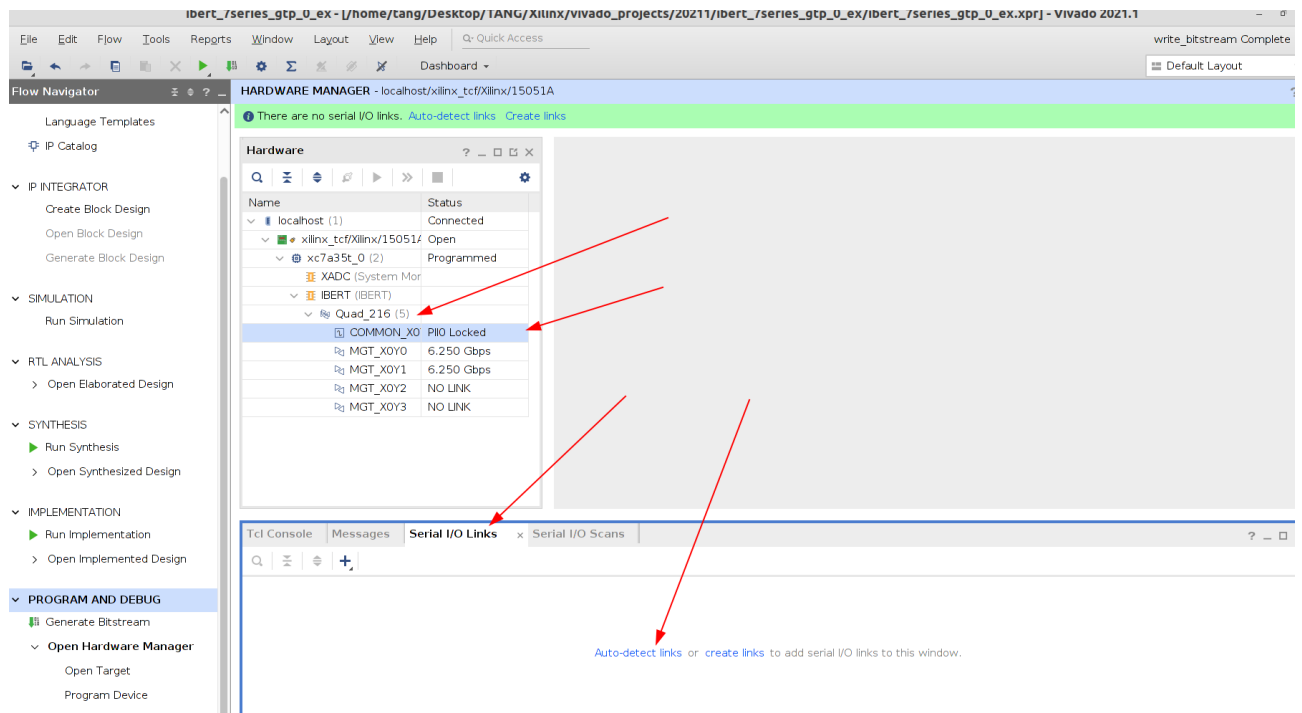


图 4-19 ibert 运行

在任意 link 上右键点击,选择 Create Scan,使用默认参数点击 OK,如图 4-20 所示。

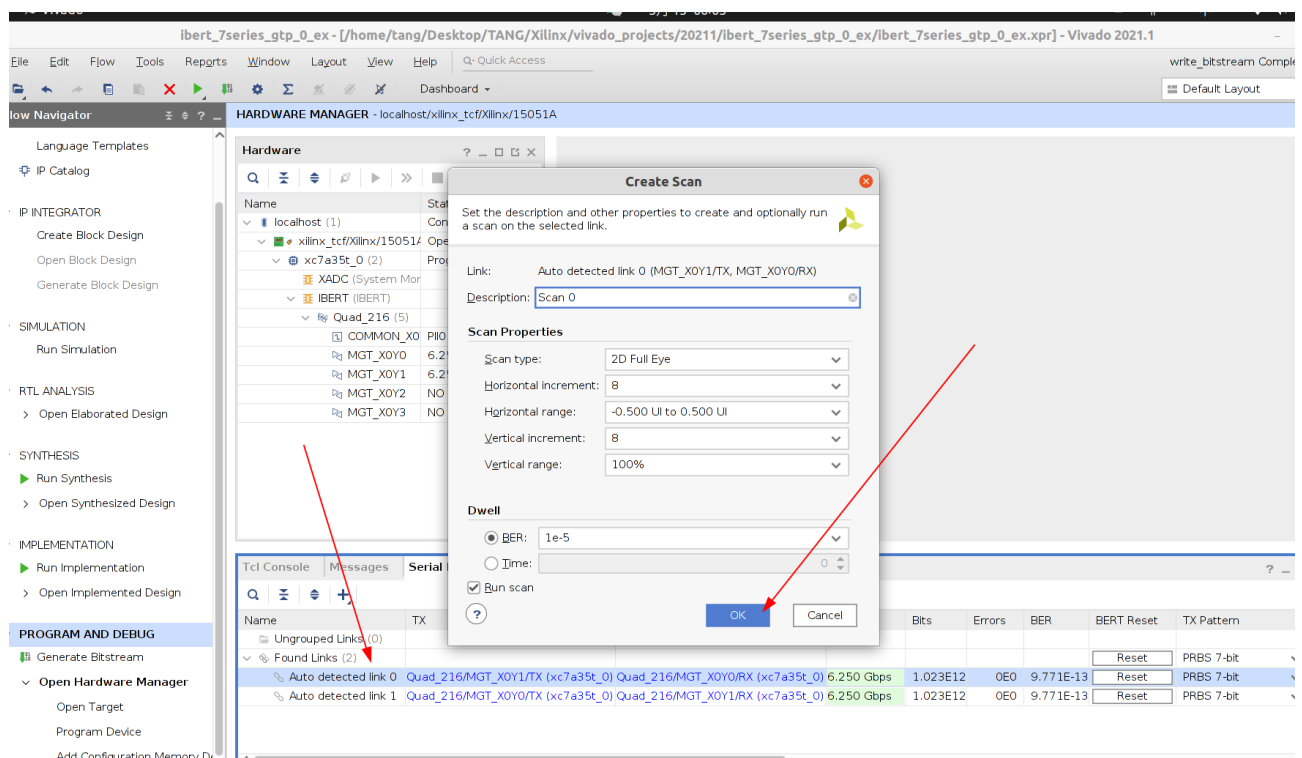


图 4-20 Create Scan

在完成一次扫描后,会自动生成眼图信息,如图 4-21 所示。

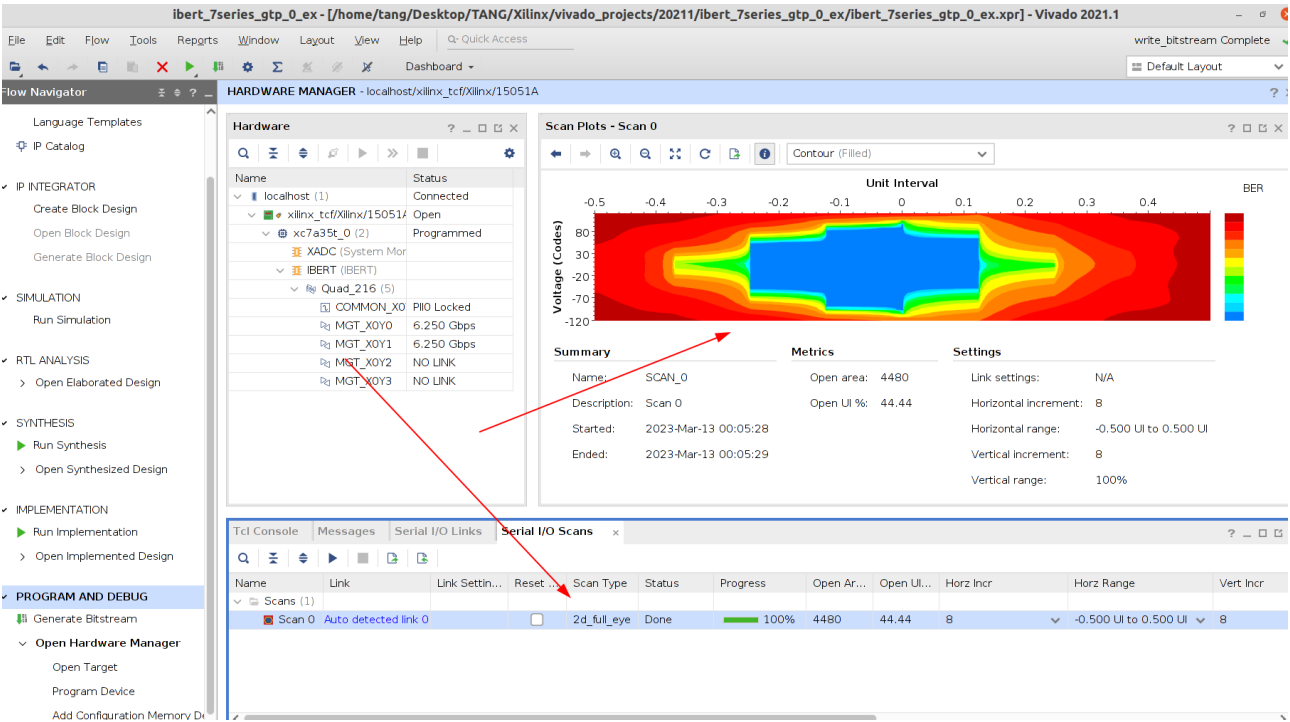


图 4-21 ibert 眼图信息

读者可根据参考资料中的 Xilinx 数据手册 PG132 Integrated Bit Error Ratio Tester 7 Series GTX Transceivers 进一步发掘更多相关测试信息。